Training Deep Neural Nets

CLOUD x LAB

Training Deep Neural Nets – Index

- Vanishing or Exploding gradients
- He Initialization
- Activation Functions
  - SELU > ELU > Leaky ReLU > ReLU > tanh > logistic
- Batch Normalization
- Gradient Clipping
- Reusing Pretrained Layers
- Faster Optimizers
- Learning Rate Scheduling
- Regularisation
  - Early Stopping, L1 & L2, DropOut
- Data Augmentation
- Practical Guidelines

Training Deep Neural Nets

- In previous sessions
  - We introduced **Artificial Neural Networks** and
  - Trained our first deep neural network
  - It was a **shallow NN**
    - With only two hidden layers
  - This shallow neural network will not work if
    - We have to deal with **complex problems** such as
    - Detecting hundreds of objects in high-resolution images

Training Deep Neural Nets

- In that case, we may need to train a deeper neural network containing
  - Many layers
  - Each layer containing hundred of neurons
  - Connected by hundreds of thousands of connections

Question

What will be the challenges in training such a deep neural network?

Training Deep Neural Nets

- We may face problem of **vanishing gradients** (which we will cover shortly)
- Training such a large network will take a **lot of time**
- Such model with millions of parameters may be prone to **overfitting**

Training Deep Neural Nets

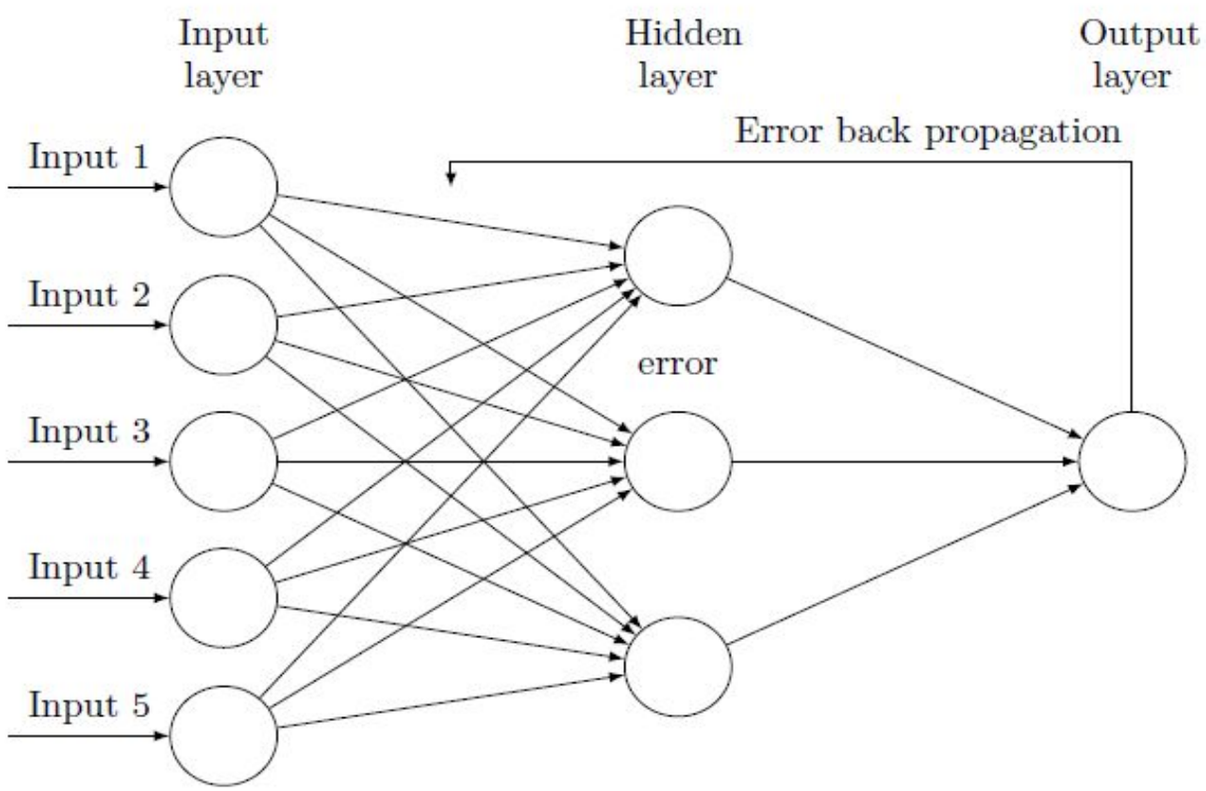
- In this chapter we will
  - Go through the vanishing gradients problem
    - And explore solutions to it
  - Look at various **optimizers** that can speed up training large models
- We will also look at
  - Popular **regularization techniques** for large neural networks

CLOUD x LAB

Vanishing / Exploding Gradients Problem

Vanishing / Exploding Gradients Problem

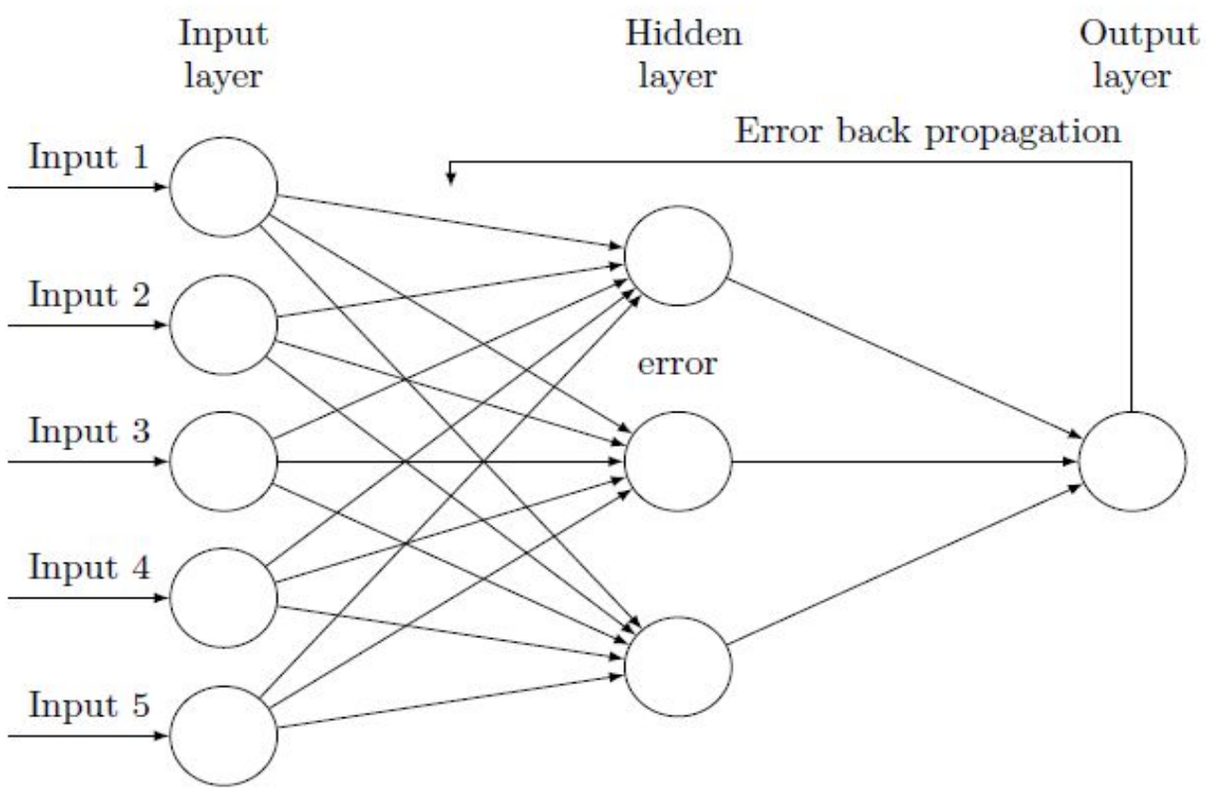
- As discussed earlier
  - Backpropagation algorithm works by going from the
  - Output layer to the input layer
  - Propagating the error gradient on the way



CLOUD x LAB

Vanishing / Exploding Gradients Problem

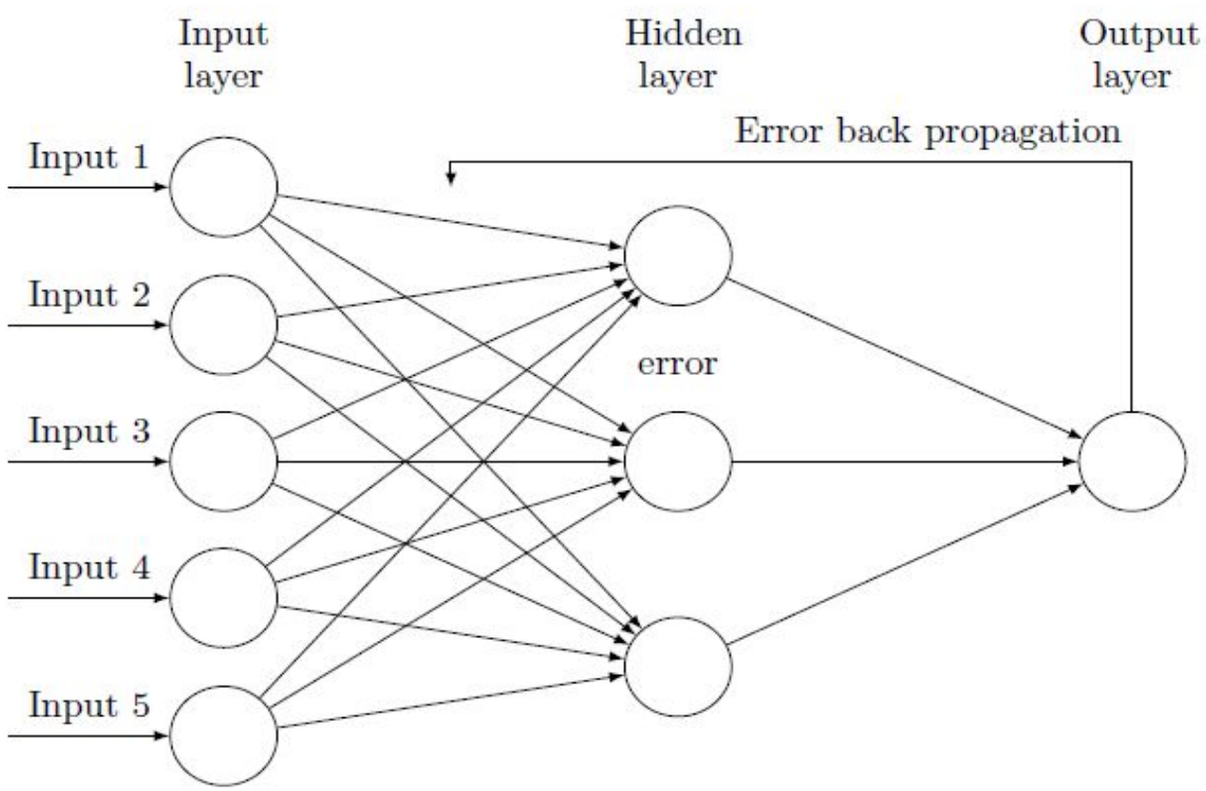
- Once the algorithm computes the gradient of the cost function
  - With regards to each parameter in the network
  - Then it uses these gradients to update each parameter



CLOUD x LAB

Vanishing / Exploding Gradients Problem

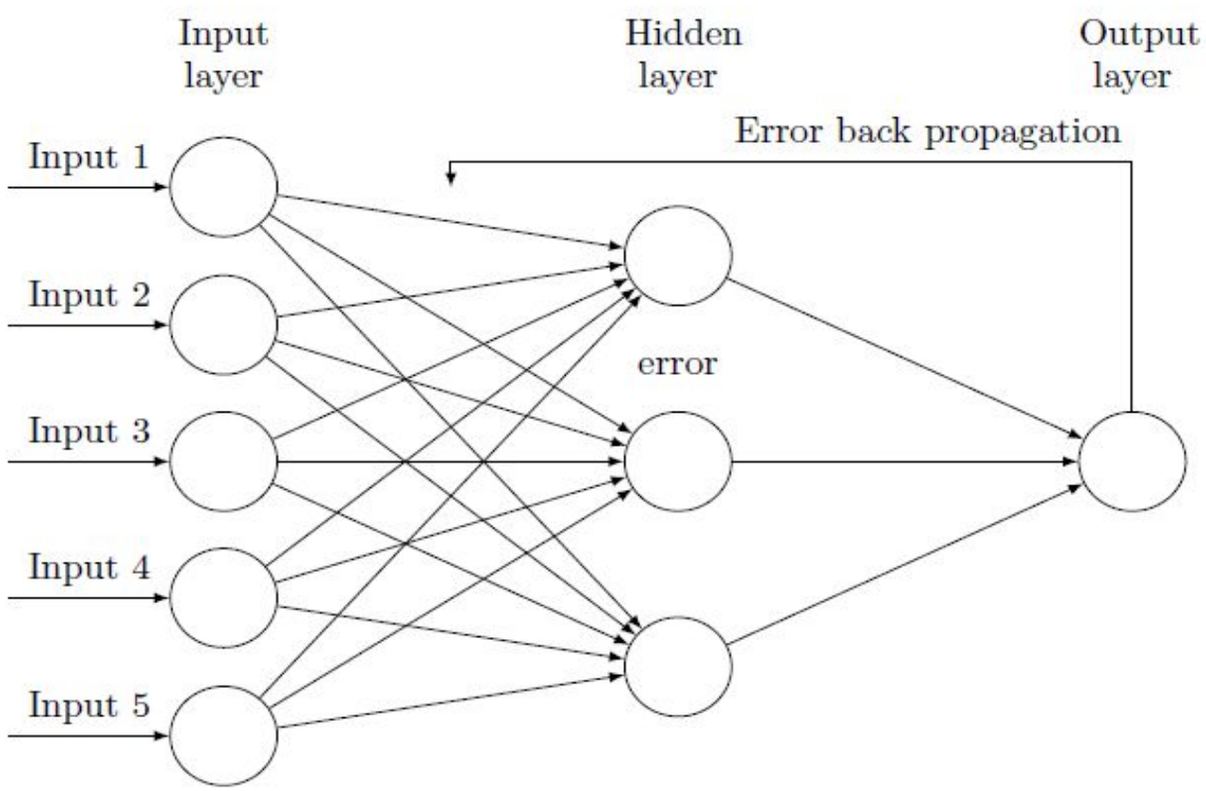
- Here the problem is that
  - Gradients often get smaller and smaller
  - As the algorithm progresses down to the early layers



CLOUD x LAB

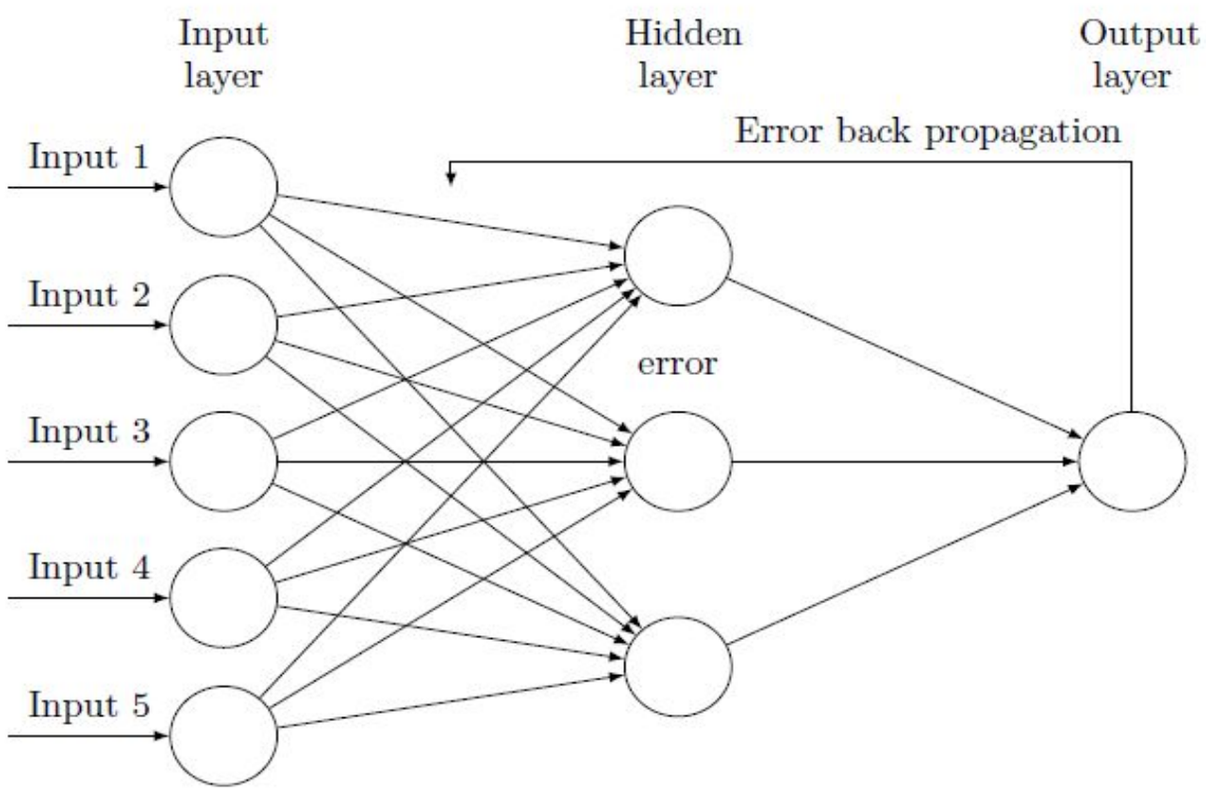
Vanishing / Exploding Gradients Problem

- Because of this,
  - The lower layer connection weights virtually remains unchanged
  - And training never converges to a good solution
  - This is called the **vanishing gradients** problem



Vanishing / Exploding Gradients Problem

- Let's understand Vanishing Gradient Problem with an example

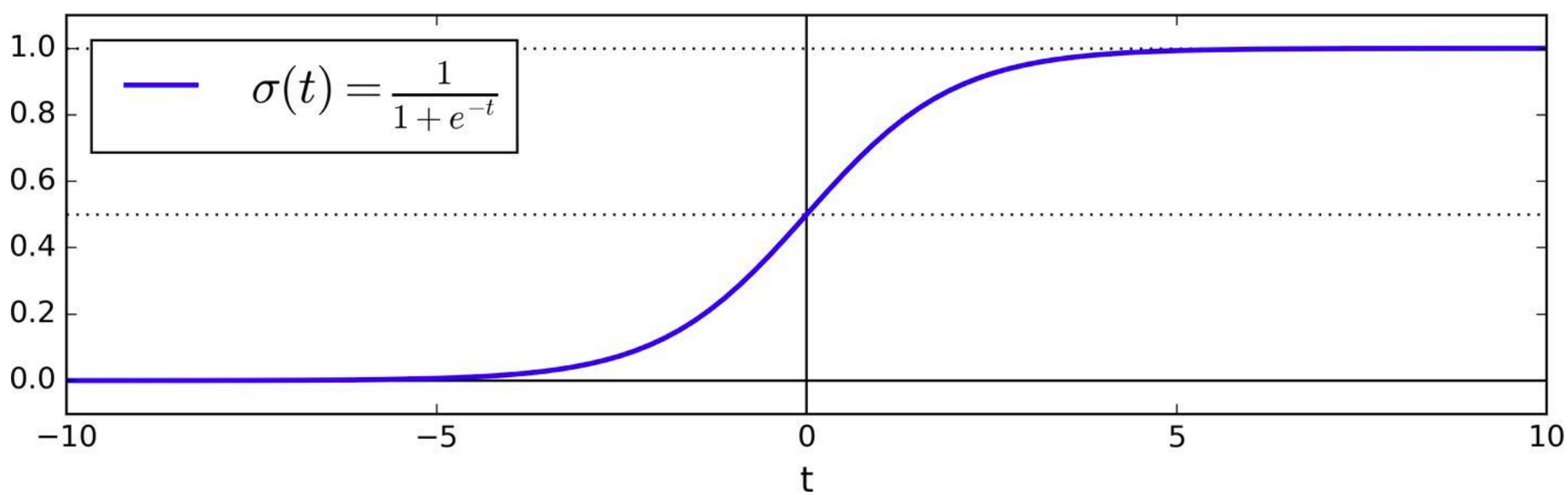


CLOUD x LAB

Vanishing / Exploding Gradients Problem

- Let's recall **sigmoid function**
  - Popular activation function for ANN in classification context
  - Its output is in range of 0 to 1

$$Sigmoid = S(\alpha) = \frac{1}{1+e^{-\alpha}}$$



Check the code to plot sigmoid function in the notebook

Vanishing / Exploding Gradients Problem

```
# Logit / Sigmoid function plot


>>> def logit(z):
>>>     return 1 / (1 + np.exp(-z))


>>> z = np.linspace(-5, 5, 200)
>>> plt.plot([-5, 5], [0, 0], 'k-')
>>> plt.plot([-5, 5], [1, 1], 'k--')
>>> plt.plot([0, 0], [-0.2, 1.2], 'k-')
>>> plt.plot([-5, 5], [-3/4, 7/4], 'g--')
```

Vanishing / Exploding Gradients Problem

```
# Logit / Sigmoid function plot


>>> plt.plot(z, logit(z), "b-", linewidth=2)
>>> props = dict(facecolor='black', shrink=0.1)
>>> plt.annotate('Saturating', xytext=(3.5, 0.7), xy=(5,
1), arrowprops=props, fontsize=14, ha="center")
plt.annotate('Saturating', xytext=(-3.5, 0.3), xy=(-5,
0), arrowprops=props, fontsize=14, ha="center")
>>> plt.annotate('Linear', xytext=(2, 0.2), xy=(0, 0.5),
arrowprops=props, fontsize=14, ha="center")
```

Vanishing / Exploding Gradients Problem

```
# Logit / Sigmoid function plot


>>> plt.grid(True)
>>> plt.title("Sigmoid activation function",
fontsize=14)
>>> plt.axis([-5, 5, -0.2, 1.2])
>>> plt.show()
```

Vanishing / Exploding Gradients Problem

- Let's look at the derivative of sigmoid function

Sigmoid Function

$$Sigmoid = S(\alpha) = \frac{1}{1+e^{-\alpha}}$$

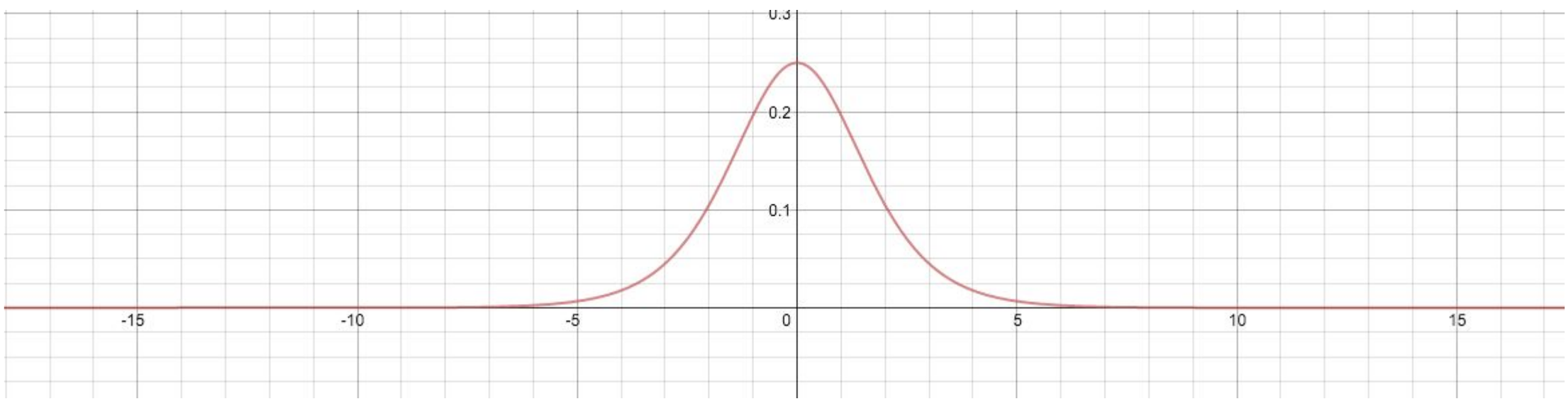
Derivative of Sigmoid
S (1 – S)

$$\frac{1}{1+e^{-\alpha}}\left[1-\frac{1}{1+e^{-\alpha}}\right]$$

Vanishing / Exploding Gradients Problem

- Let's plot the derivative of sigmoid function

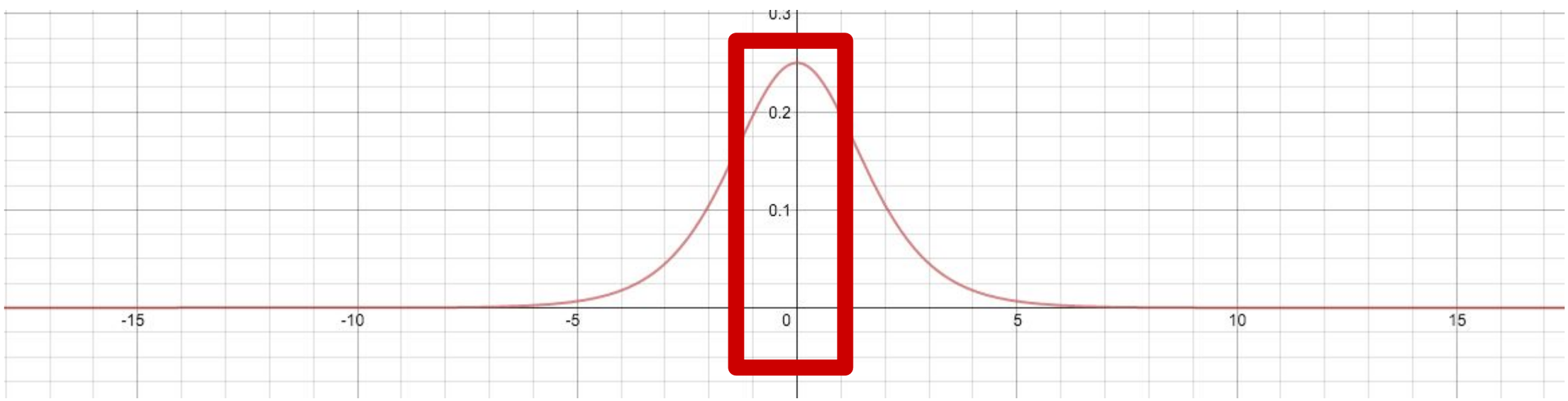
$$\frac{1}{1+e^{-\alpha}}\left[1-\frac{1}{1+e^{-\alpha}}\right]$$



Derivative of Sigmoid function

Vanishing / Exploding Gradients Problem

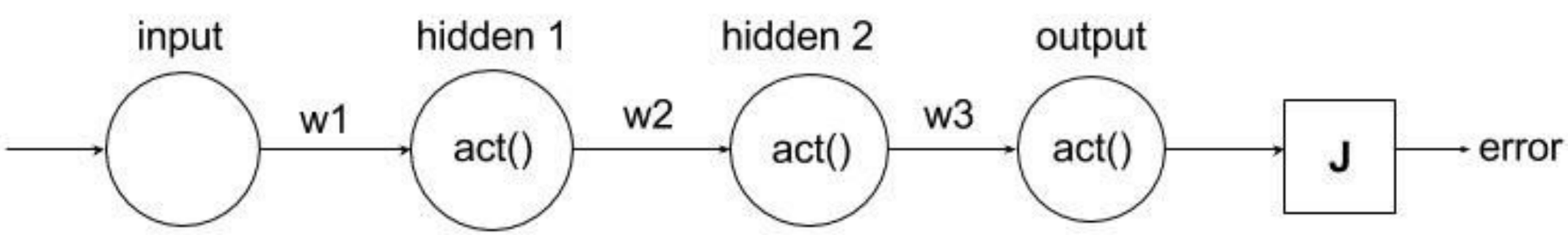
- Let's plot the derivative of sigmoid function
- As we can see
  - The output of the derivative of the Sigmoid function is
  - Always between 0 and ¼ (0.25)



Derivative of Sigmoid function

Vanishing / Exploding Gradients Problem

- Now let's look at the below univariate neural network
  - It has 2 hidden layers
  - ***act()*** is a sigmoid activation function
  - ***J*** returns the aggregate error of the model

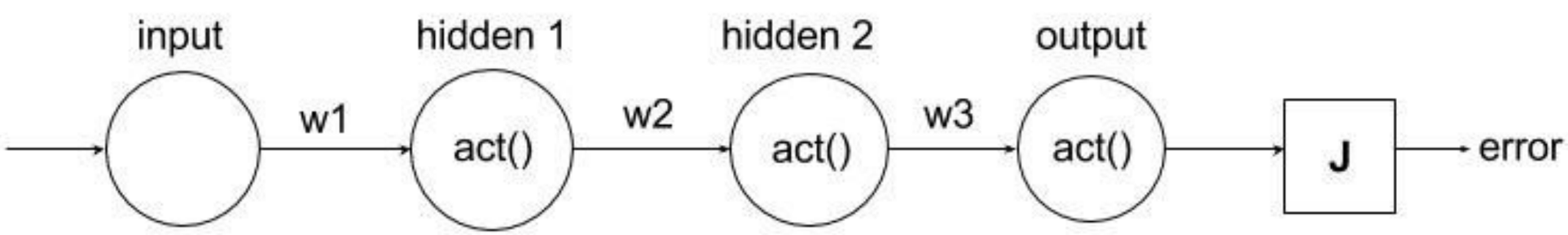


Univariate 2-layer Neural Network

Vanishing / Exploding Gradients Problem

- Now as per the chain rule in backpropagation
  - Rate of change in error because of weight w1 is

$$\frac{\partial error}{\partial w1} = \frac{\partial error}{\partial output} * \frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1} * \frac{\partial hidden1}{\partial w1}$$



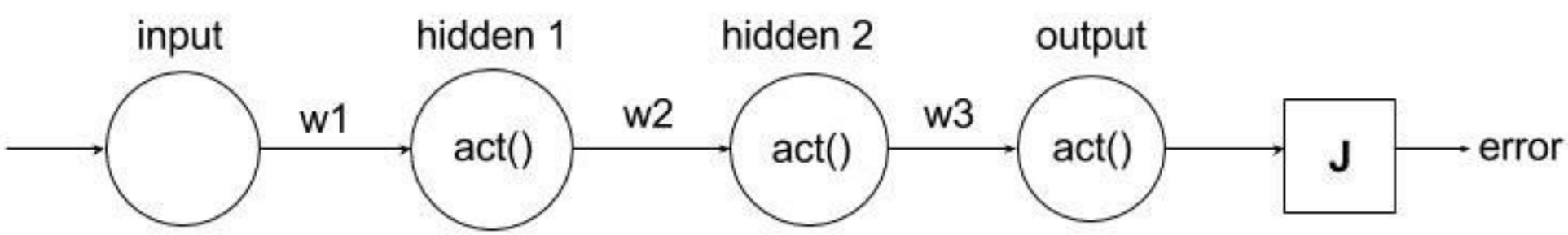
Univariate 2-layer Neural Network

Vanishing / Exploding Gradients Problem

- Let's focus on individual derivative for now

$$\frac{\partial error}{\partial w1} = \frac{\partial error}{\partial output} * \boxed{\frac{\partial output}{\partial hidden2}} * \frac{\partial hidden2}{\partial hidden1} * \frac{\partial hidden1}{\partial w1}$$

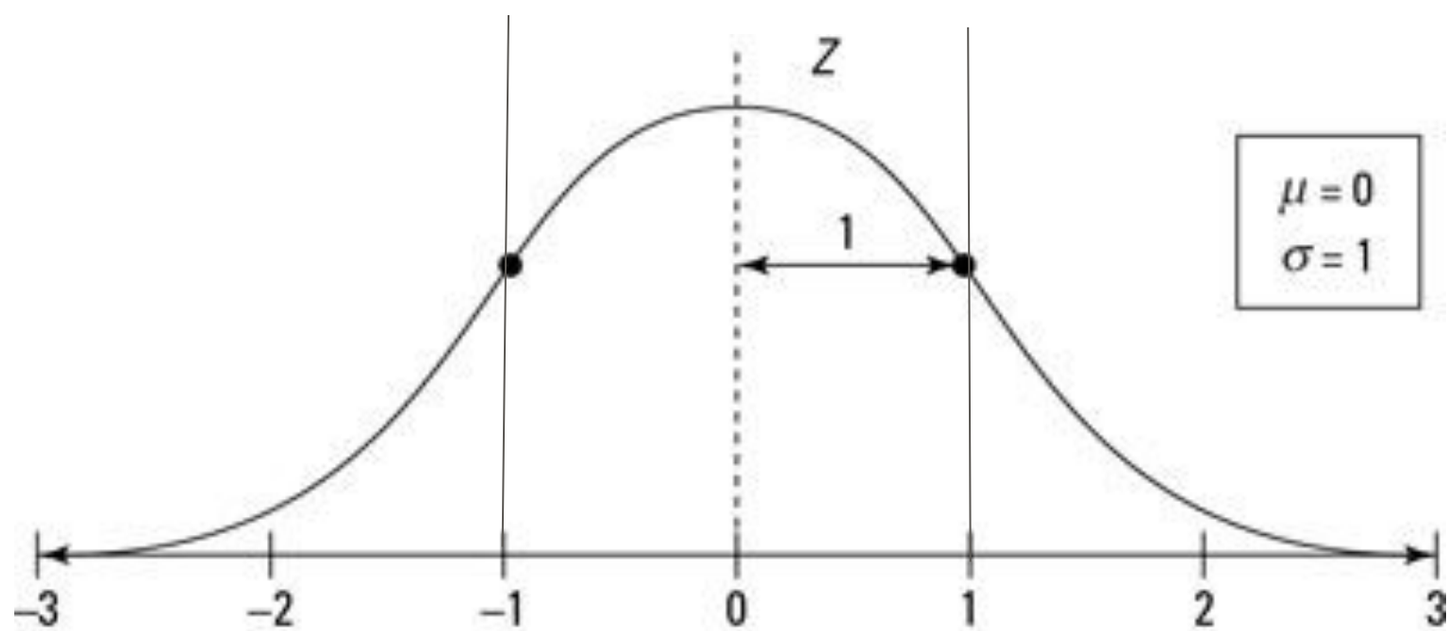
$$\frac{\partial output}{\partial hidden2} = \frac{\partial Sigmoid(hidden2)}{\partial hidden2} * w3$$



Univariate 2-layer Neural Network

Vanishing / Exploding Gradients Problem

- A typical approach of weight initialization in a neural network is to
  - Choose weights using a normal distribution with
    - Mean of 0 and
    - Standard deviation of 1
  - Hence, the weights in the neural network are usually
    - Between –1 and 1



Vanishing / Exploding Gradients Problem

- Now let's come back to our individual derivative
- As we have seen in the past that
  - Output of derivative of sigmoid function lies between 0 and ¼
- And we have just discussed that
  - Weights in the neural network are usually between -1 and 1

$$\frac{\partial output}{\partial hidden2} = \underbrace{\frac{\partial Sigmoid(hidden2)}{\partial hidden2}}_{< 1/4} * \underbrace{w3}_{< 1}$$

CLOUD x LAB

Vanishing / Exploding Gradients Problem

- Important – If we multiply two numbers between 0 and 1
  - Then the result will always be smaller
  - For example
  - ⅓ * ¼ = 1/12 (which is less than ⅓ and ¼)
- Here we are multiplying 2 values which are between 0 and 1
  - And the resulting gradient will be smaller

$< \frac{1}{4}$ < 1

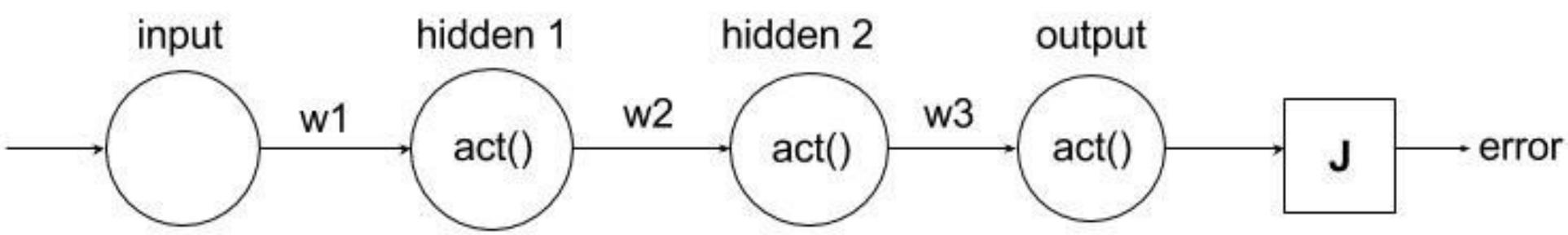
$$\frac{\partial output}{\partial hidden2} = \frac{\partial Sigmoid(hidden2)}{\partial hidden2} * w3$$

Vanishing / Exploding Gradients Problem

- Now let's take another individual derivative

$$\frac{\partial error}{\partial w1} = \frac{\partial error}{\partial output} * \boxed{\frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1}} * \frac{\partial hidden1}{\partial w1}$$

$$\frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1} = \frac{\partial Sigmoid(hidden2)}{\partial hidden2} w3 * \frac{\partial Sigmoid(hidden1)}{\partial hidden1} w2$$



Univariate 2-layer Neural Network

Vanishing / Exploding Gradients Problem

- This derivative has
  - Two sigmoid activation function
  - And here we multiply 4 values between 0 and 1
  - So this gradient will be really smaller than
  - The earlier derivative (∂output / ∂hidden2)

$$\frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1} = \overset{<\,1/4}{\frac{\partial Sigmoid(hidden2)}{\partial hidden2}} \overset{<\,1}{w3} * \overset{<\,1/4}{\frac{\partial Sigmoid(hidden1)}{\partial hidden1}} \overset{<\,1}{w2}$$

CLOUD x LAB

Vanishing / Exploding Gradients Problem

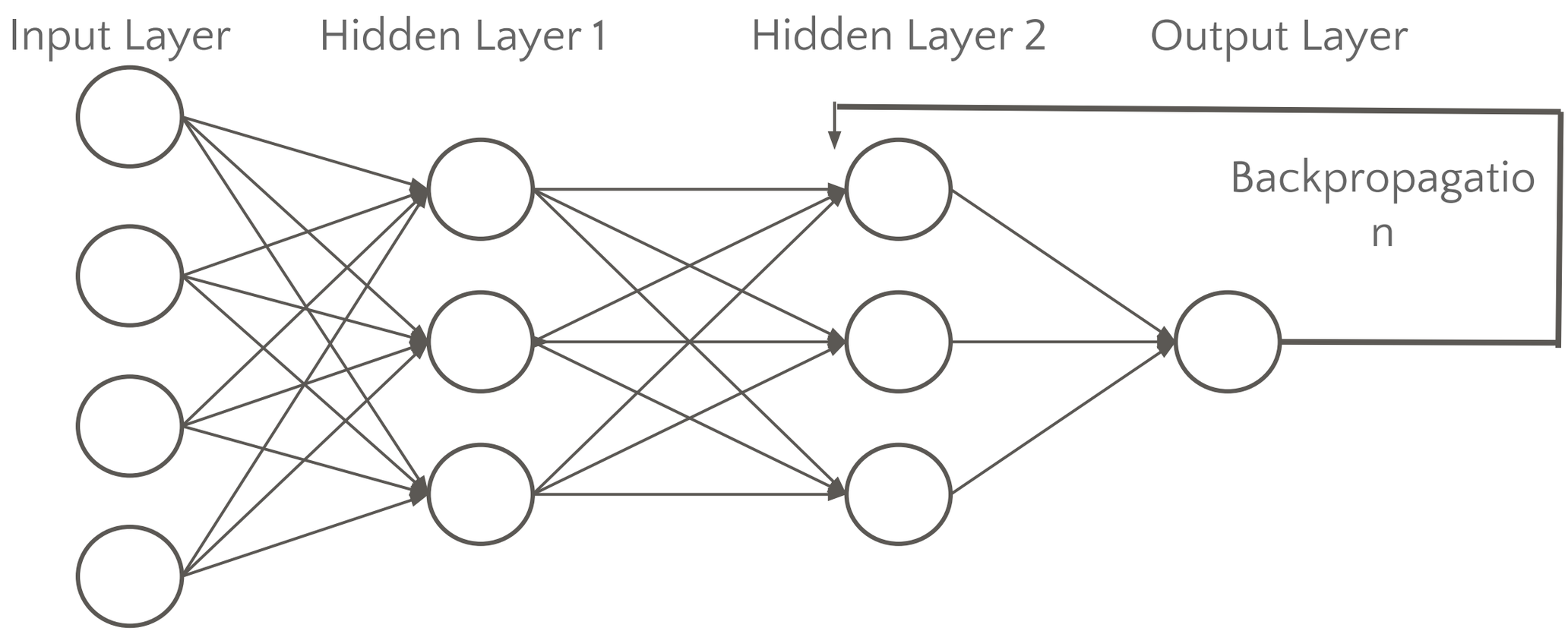
- So we can see that in the backpropagation as we move backward
  - Gradient just becomes smaller and smaller in every layer
  - And it becomes tiny in the early layers (input layers or the first layers)
  - This is called as **Vanishing Gradient Problem**

$$\frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1} = \overset{< 1/4}{\frac{\partial Sigmoid(hidden2)}{\partial hidden2}} \overset{< 1}{w3} * \overset{< 1/4}{\frac{\partial Sigmoid(hidden1)}{\partial hidden1}} \overset{< 1}{w2}$$

CLOUD x LAB

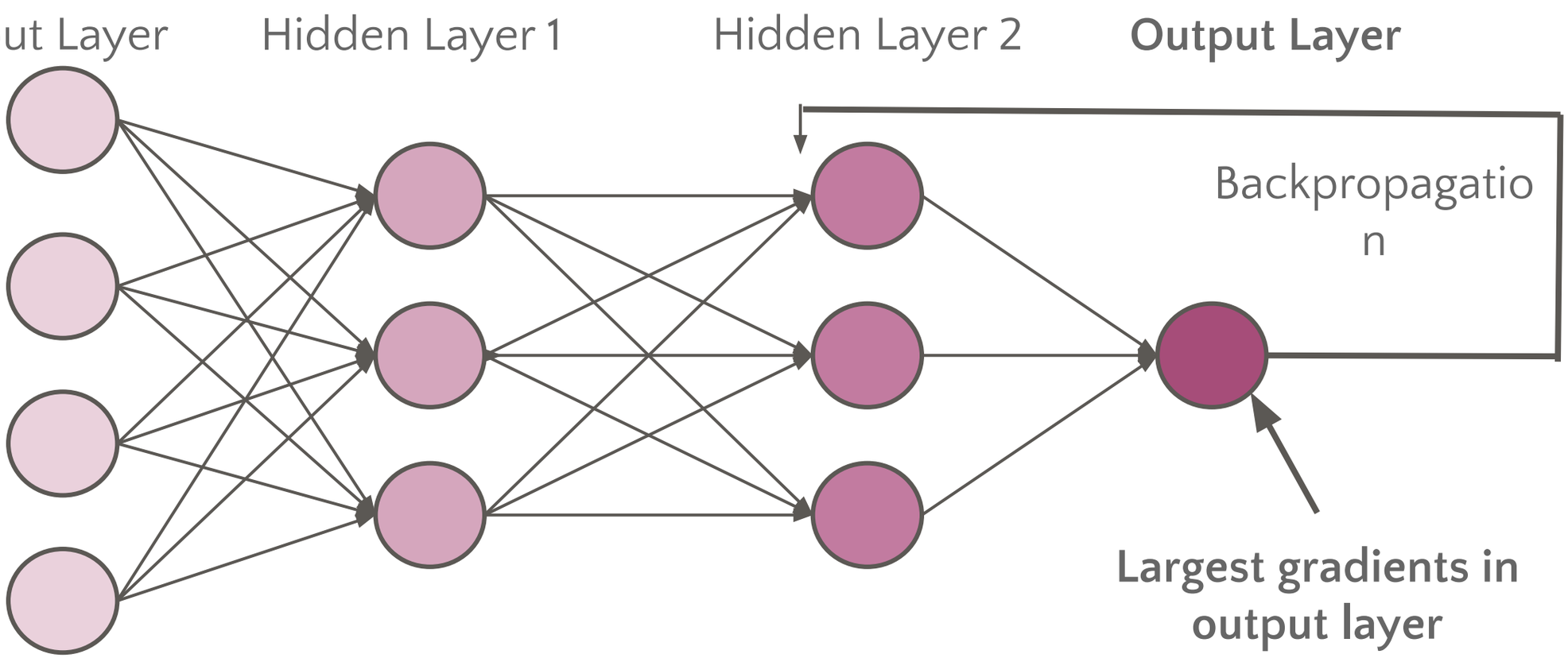
Vanishing / Exploding Gradients Problem

- Let's understand it once again
- Below is 2-layer neural network



Vanishing / Exploding Gradients Problem

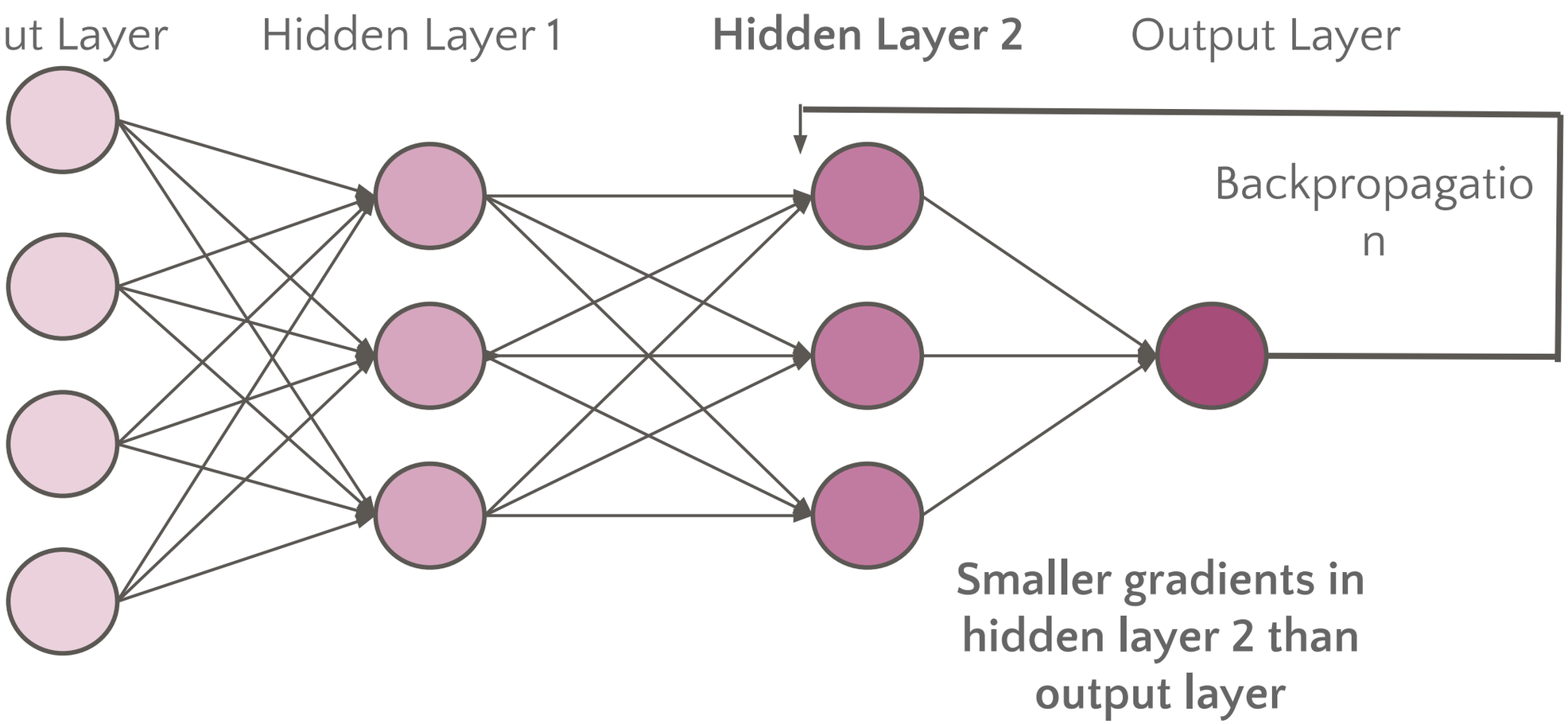
- Gradients will be largest in the output layer
  - Hence output layer is easiest to train



CLOUD x LAB

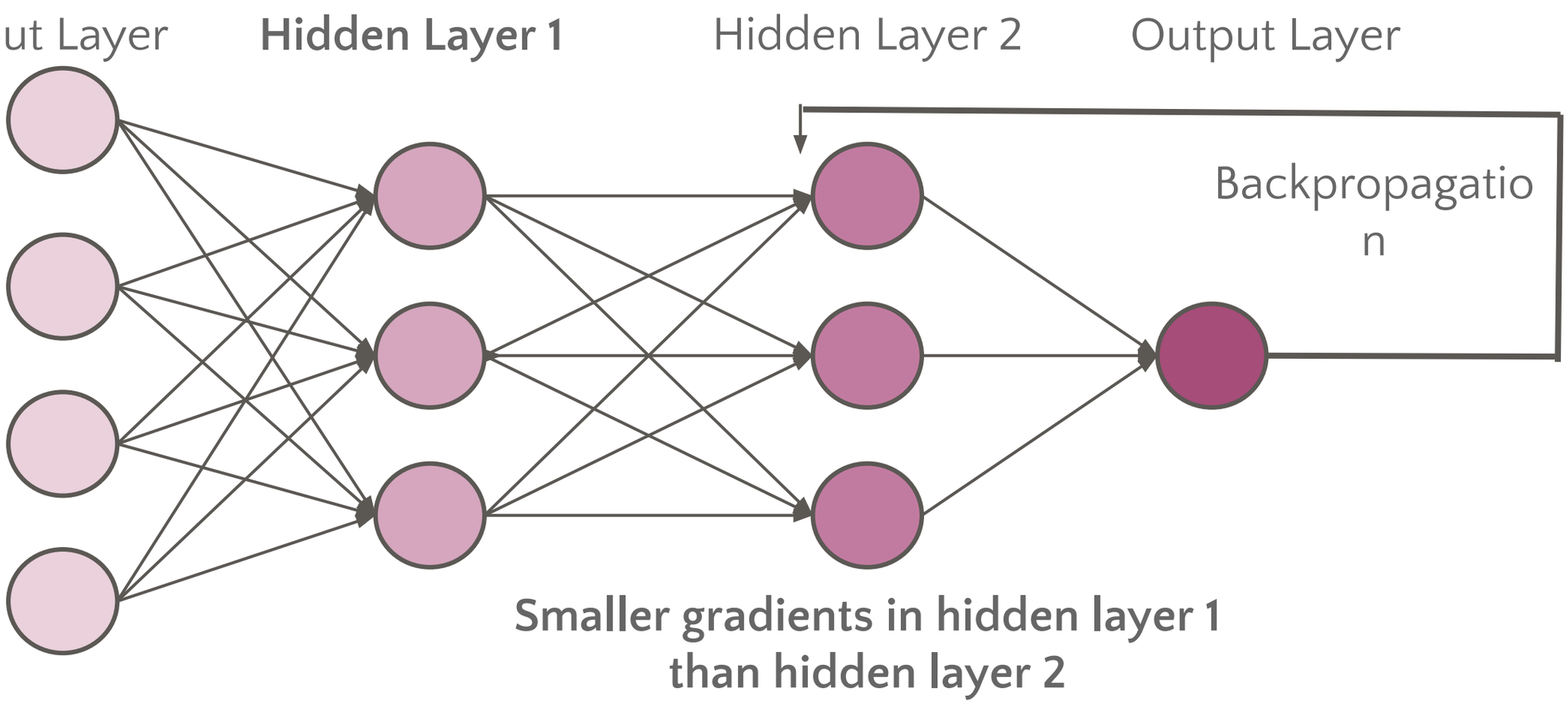
Vanishing / Exploding Gradients Problem

- Hidden layer 2 have
  - Smaller gradients than output layer



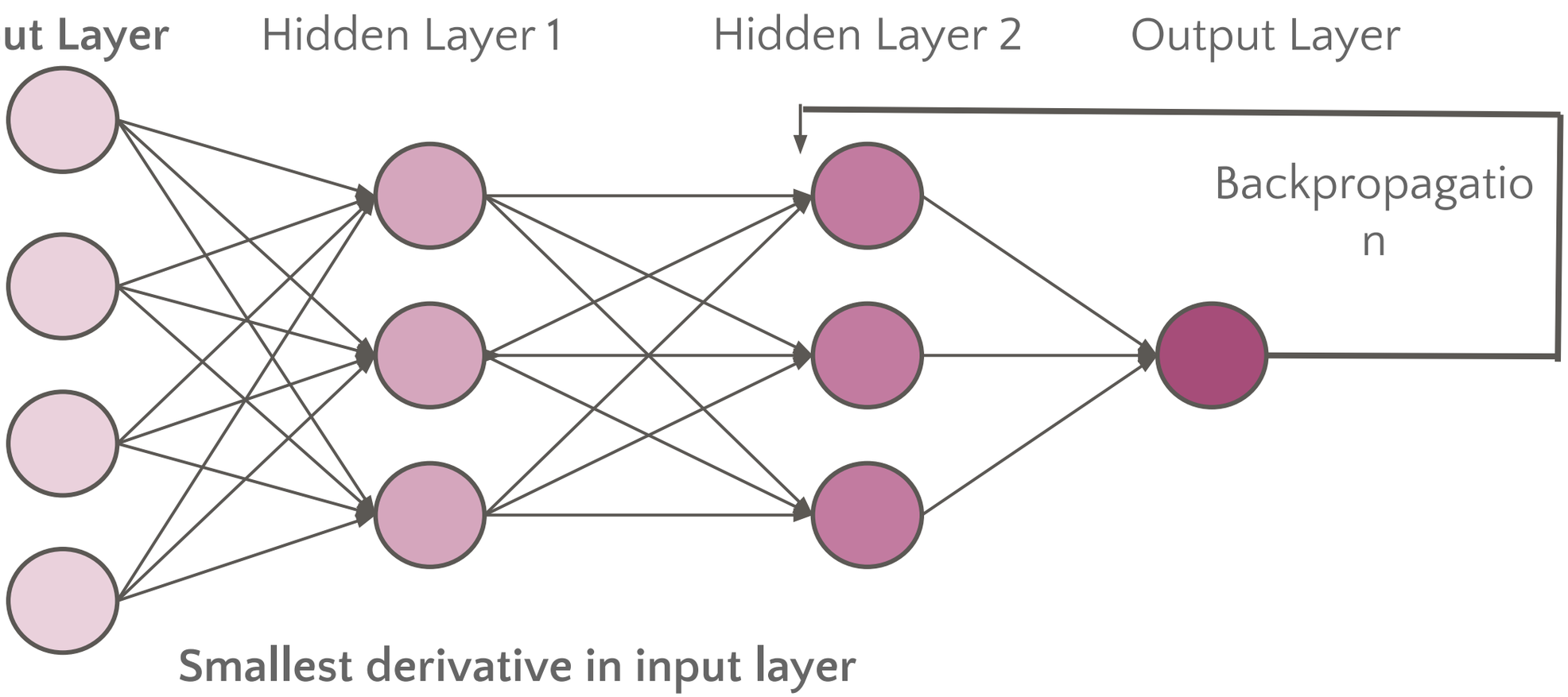
Vanishing / Exploding Gradients Problem

- Hidden layer 1 have
  - Smaller gradients than hidden layer 2



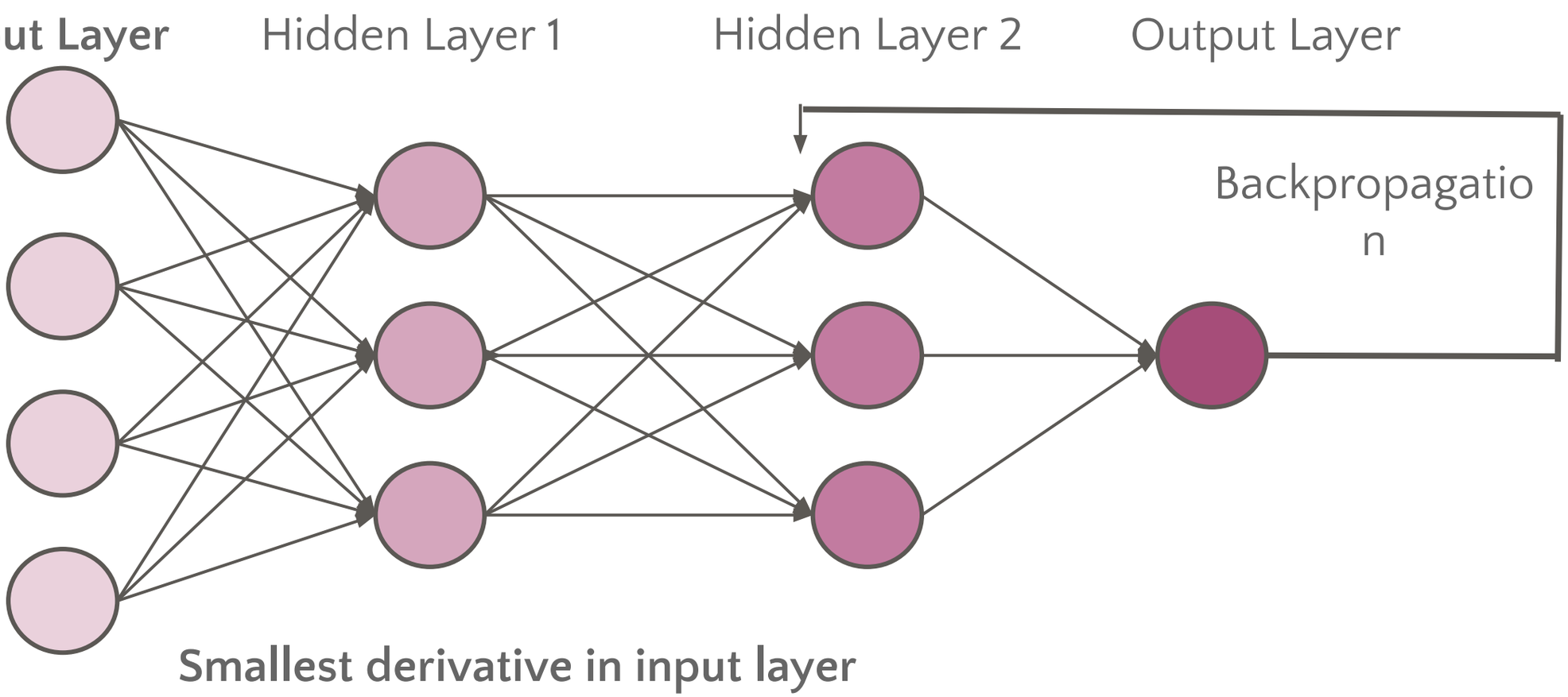
Vanishing / Exploding Gradients Problem

- As input layer is farthest from the output layer
  - Its derivative will be the longer expression (using chain rule)
  - Hence it will contain more sigmoid derivatives



Vanishing / Exploding Gradients Problem

- As input layer is farthest from the output layer
  - And it will have smallest derivative
  - This makes lower layers slowest to train



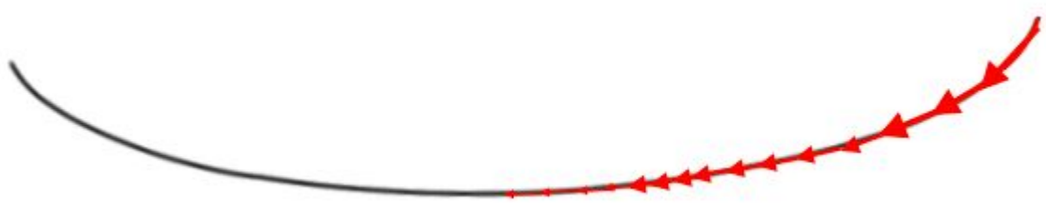
Question

So why Vanishing Gradient is a problem?

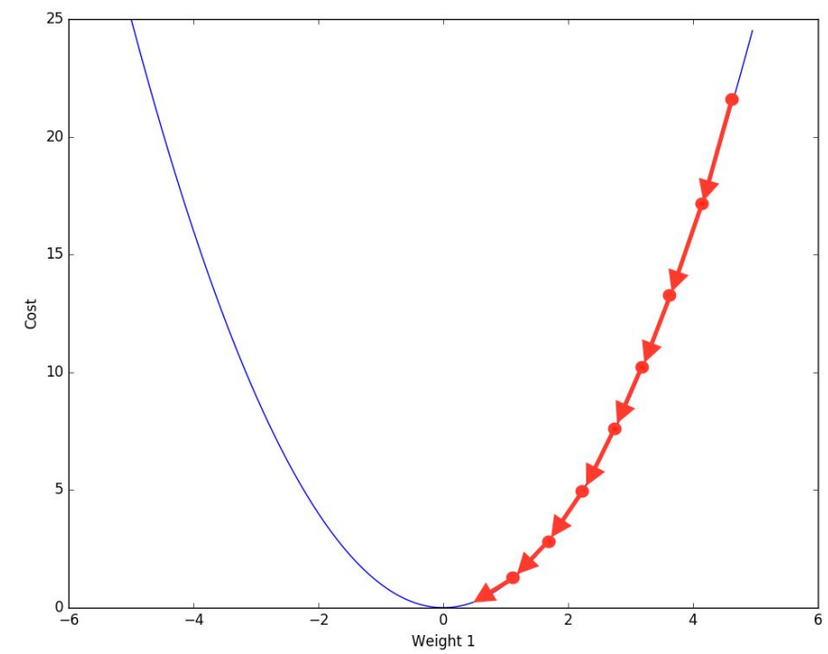
Vanishing / Exploding Gradients Problem

First problem

- Since gradient becomes really small in early layers (input layers)
  - It becomes **really slow** to train the early layers



Flat surface – small gradients. Gradient Descent converges slowly

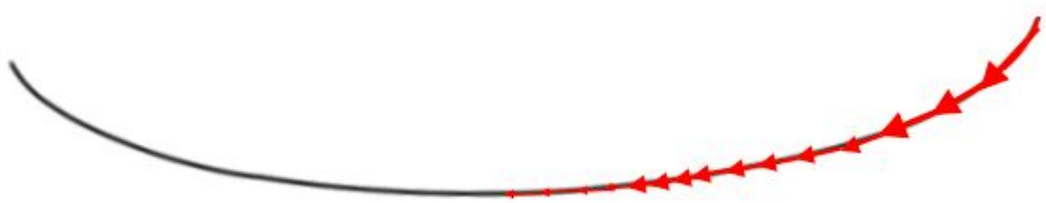


Larger gradients. Gradient Descent converges fast

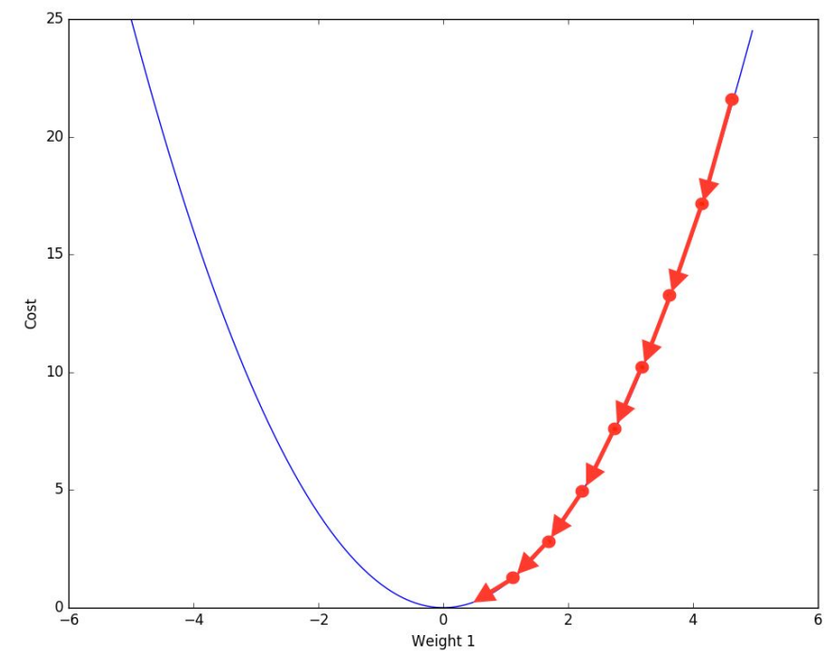
Vanishing / Exploding Gradients Problem

First problem

- Also because of small steps
  - May **converge at a local minimum** instead of global minimum



Flat surface – small gradients. Gradient Descent converges slowly



Larger gradients. Gradient Descent converges fast

CLOUD x LAB

Vanishing / Exploding Gradients Problem

Second problem

- Since the latter layers are dependent on the early layers
  - If early layers are not accurate
  - Then the latter or lower layers just build on this inaccuracy
  - And the entire neural net gets corrupted

Vanishing / Exploding Gradients Problem

Second problem

- Early layers are responsible for
  - Detecting simple patterns and are
  - Building blocks of the neural network
  - Hence it becomes important that early layers are accurate

Vanishing / Exploding Gradients Problem

Second problem

- For example, in face recognition
  - Early layers detects the edges
  - Which gets combined to form facial features later in the network
- And if early layers get it wrong
  - The result built up by the neural network will be wrong

Original Image

Image seen by neural network

Vanishing / Exploding Gradients Problem

Exploding Gradients Problem

- Like vanishing gradients problem
  - We can also have exploding gradients problem
  - If the gradients were bigger than 1 (multiplying numbers greater than 1 always gives huge result)
  - Because of this, some layers may get insanely large weights and
  - The algorithm diverges instead of converging
  - This is called **Exploding Gradients Problem**

Vanishing / Exploding Gradients Problem

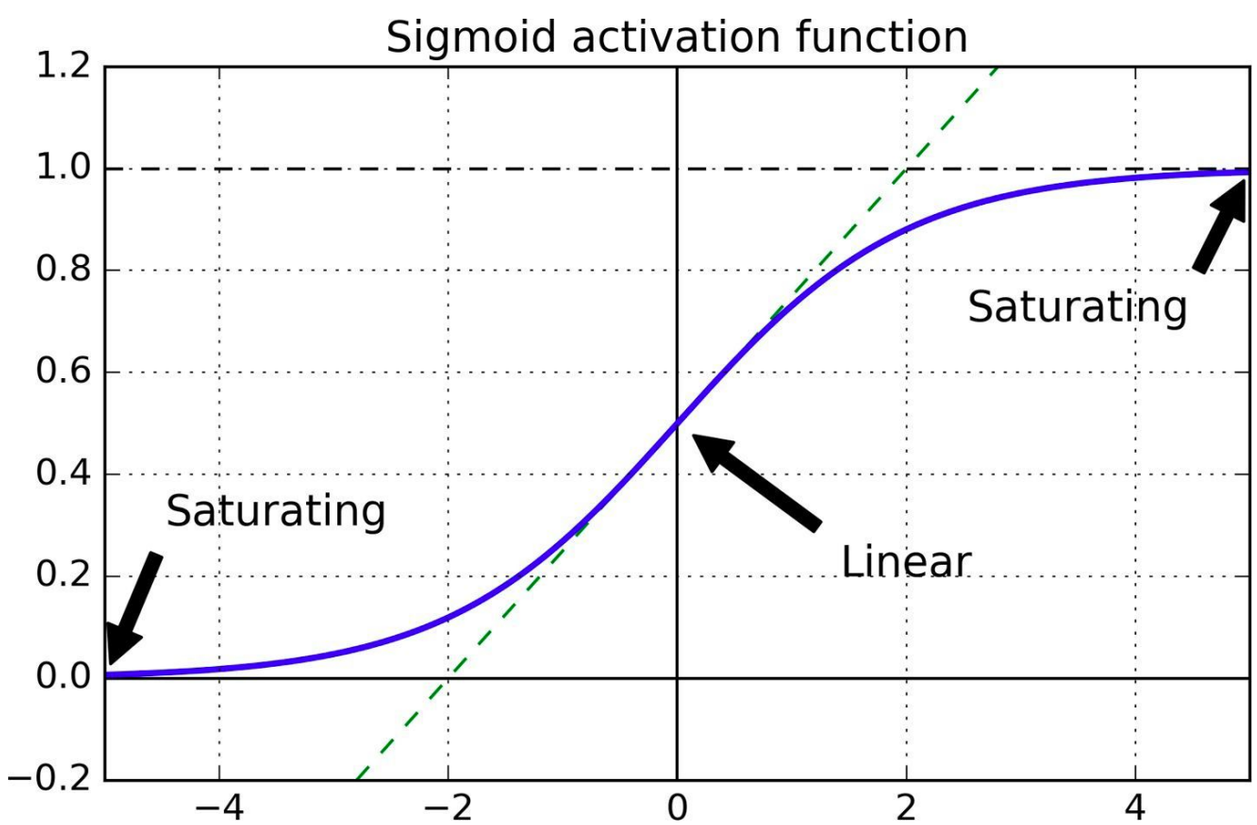
- As we have seen deep neural networks suffer from unstable gradients
  - Different layers may learn at widely different speeds
- Because of vanishing gradients problem
  - Deep Neural Network were abandoned for a long time
  - Training the early layer correctly was the basis of network
  - But it proved too difficult that time because of
  - Available activation functions and hardware

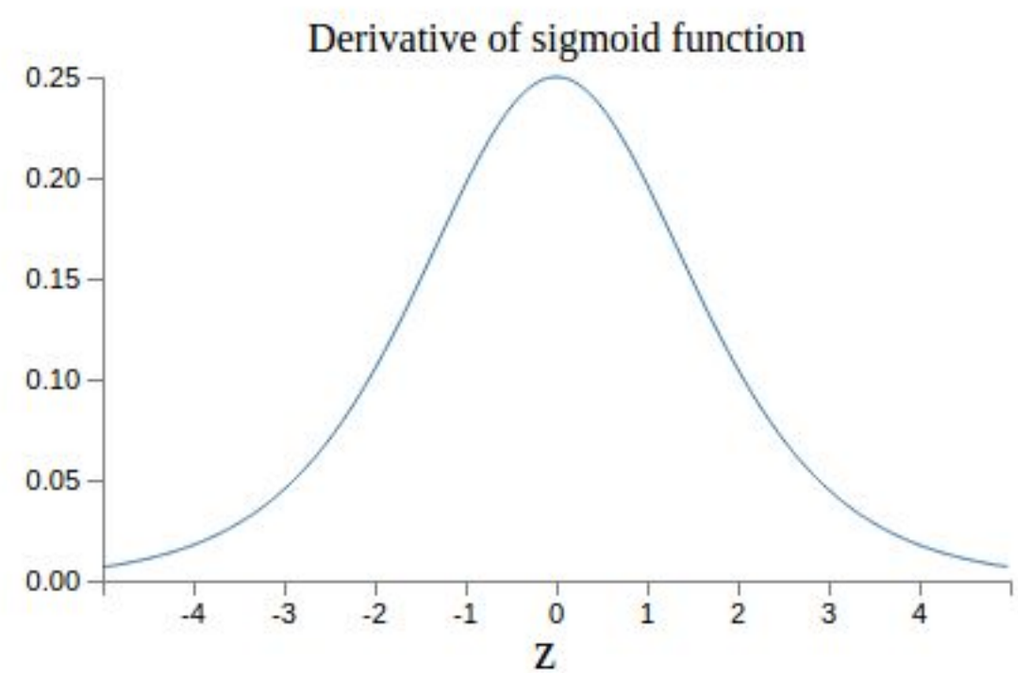
Vanishing / Exploding Gradients Problem

- In 2010, Xavier Glorot and Yoshua Bengio published a paper titled
  - **"Understanding the Difficulty of Training Deep Feedforward Neural Networks"**
- Authors of this paper suggested that root cause of vanishing gradient problem is
  - Nature of the sigmoid activation function derivative

Vanishing / Exploding Gradients Problem

- If input is large,
  - Sigmoid function saturates at 0 or 1
  - And its derivative becomes extremely close to 0





Question

So what is the solution of vanishing gradients problem?

Answer:

Good strategy for initializing weights

&

Use better activation functions

Vanishing / Exploding Gradients Problem

- Kaiming He suggested strategy for initializing the weights
  - To avoid vanishing gradients problem
  - called ***He initialization***
  - with below parameters for various activation functions

Table 11-1. Initialization parameters for each type of activation function

Activation function	Uniform distribution [–r, r]	Normal distribution
Logistic	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
Hyperbolic tangent	$r = 4\sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = 4\sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
ReLU (and its variants)	$r = \sqrt{2}\sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$

Vanishing / Exploding Gradients Problem

HE Initialization

By default, Keras uses Glorot initialization with a uniform distribution.

When creating a layer, you can change this to He initialization by setting `kernel_initializer="he_uniform"` or `kernel_initializer="he_normal"` like this:

```
keras.layers.Dense(10, activation="relu", kernel_initializer="he_normal")
```

Vanishing / Exploding Gradients Problem

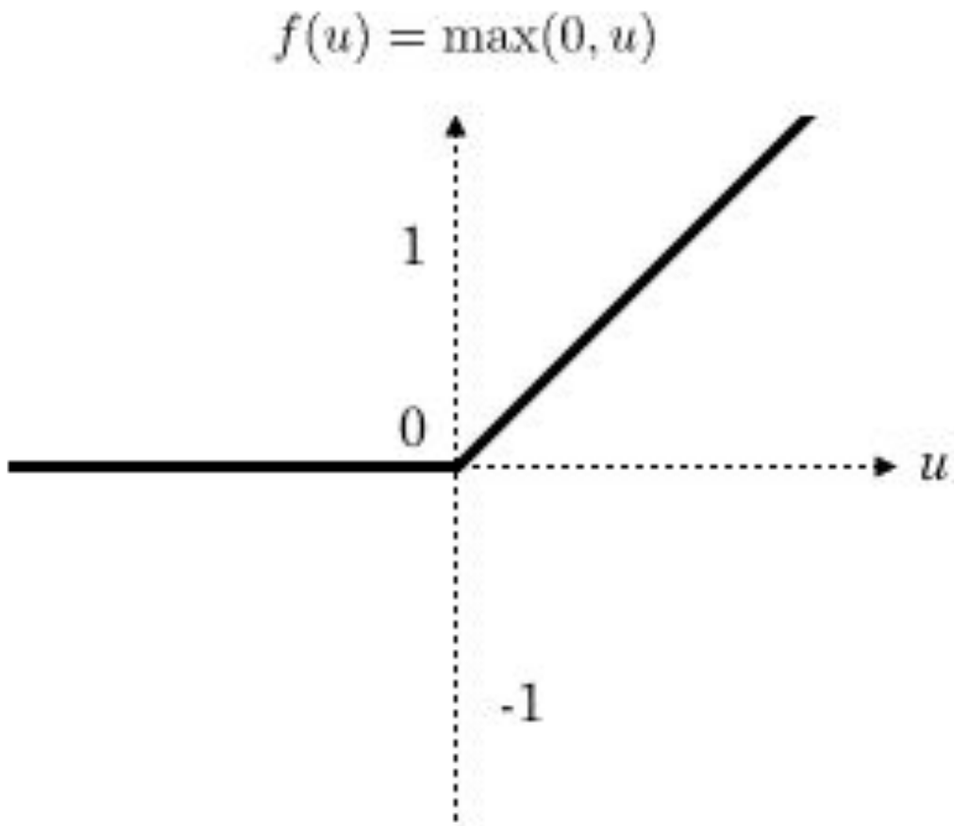
HE Initialization

If you want He initialization with a uniform distribution but based on fan_{avg} rather than fan_{in}, you can use the VarianceScaling initializer like this:

```
>>> he_avg_init =
keras.initializers.VarianceScaling(scale=2.,
mode='fan_avg', distribution='uniform')
>>> keras.layers.Dense(10, activation="sigmoid",
kernel_initializer=he_avg_init)
```

ReLU Activation Function

- It turns out that ReLU activation function works better for Deep Neural Networks
  - Because it does not saturate for positive values
  - And it is quite fast to compute



ReLU (z) = max (0, z)

CLOUD x LAB

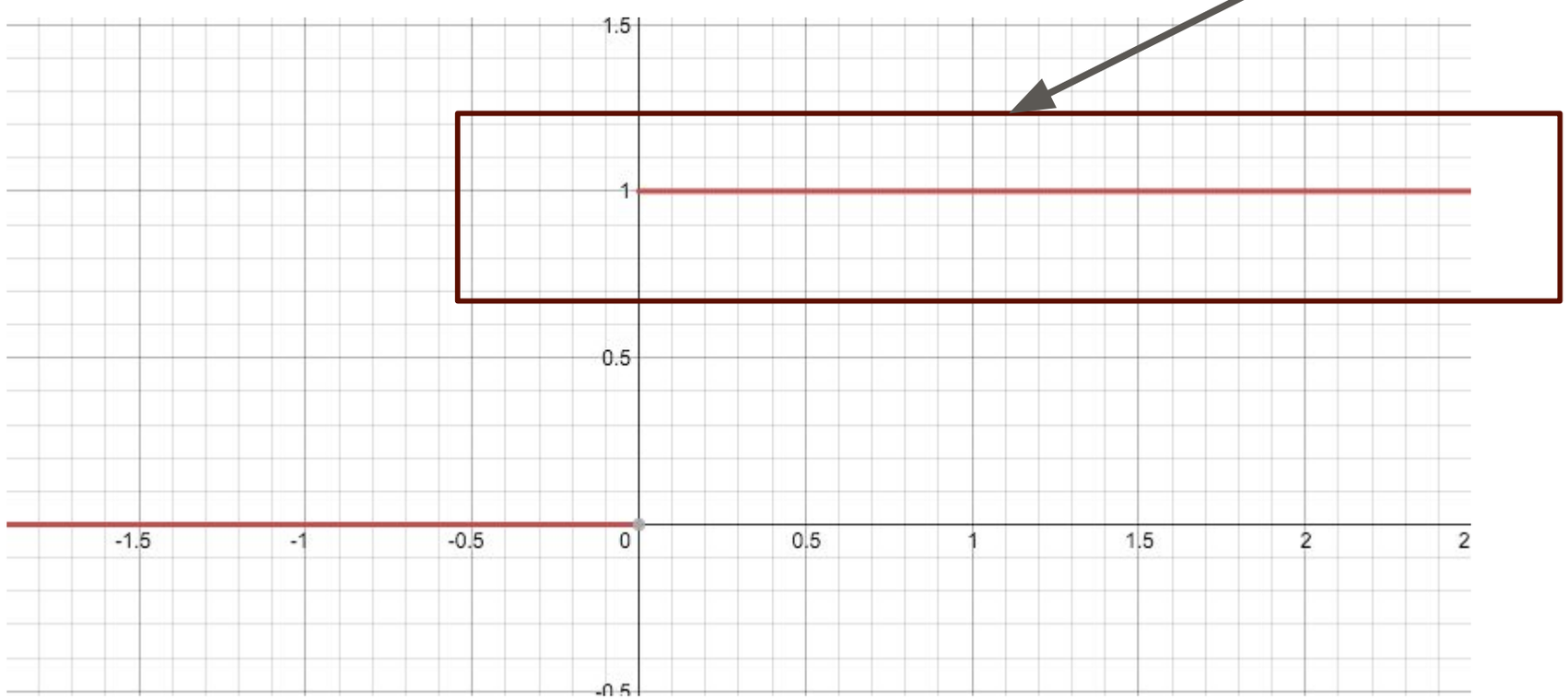
ReLU Activation Function

Derivative of ReLU activation function

- It is not differentiable at x = 0

$$\frac{d}{dx}RELU(x) = \begin{cases} 0 & if \quad x < 0 \\ 1 & if \quad x > 0 \end{cases}$$

For positive inputs , the derivative is always 1



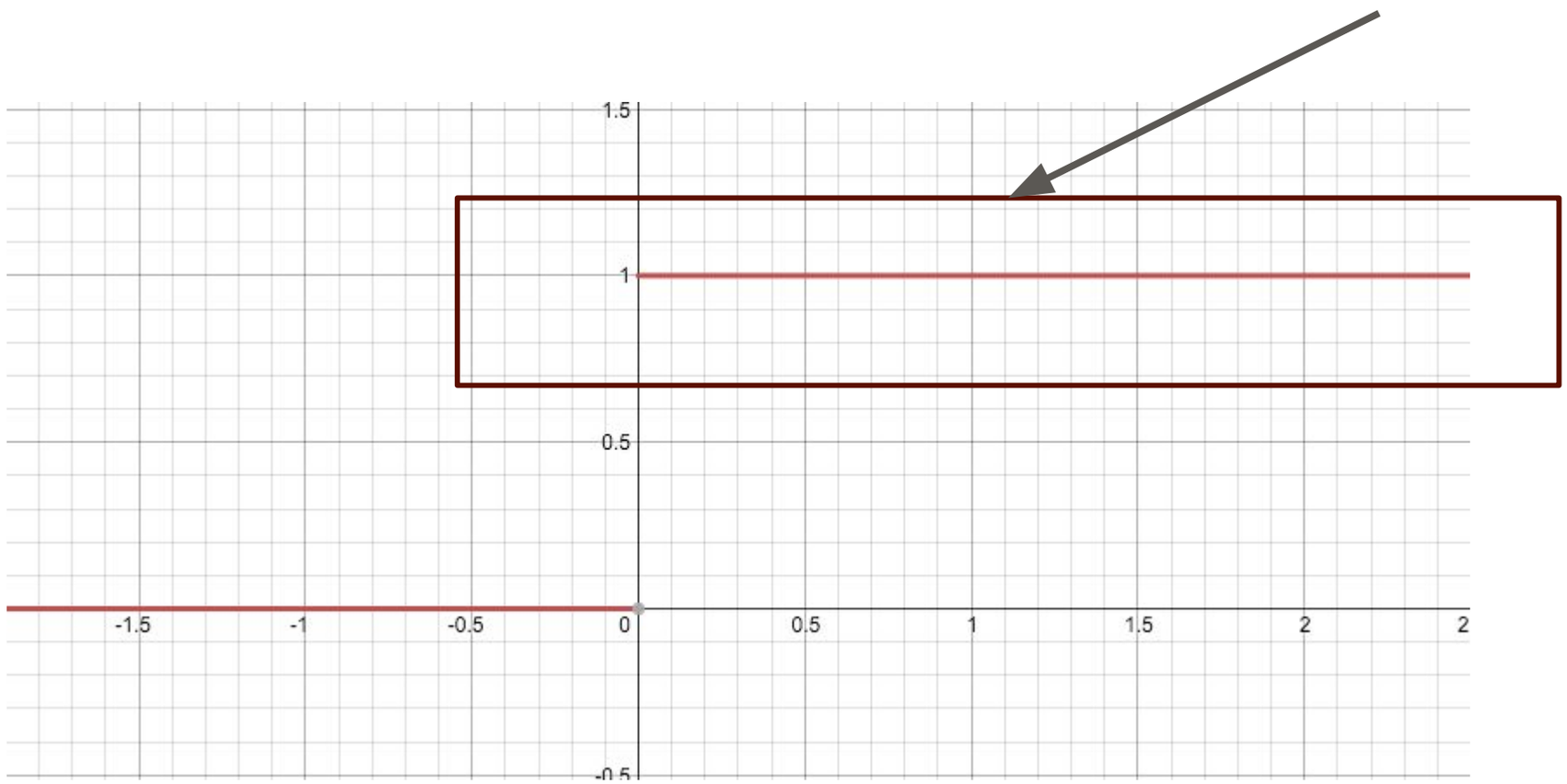
Derivative of ReLU activation function

ReLU Activation Function

Derivative of ReLU activation function

- So with ReLU our gradients will never vanish
- As long as inputs are positive

For positive inputs , the derivative is always 1



Derivative of ReLU activation function

CLOUD x LAB

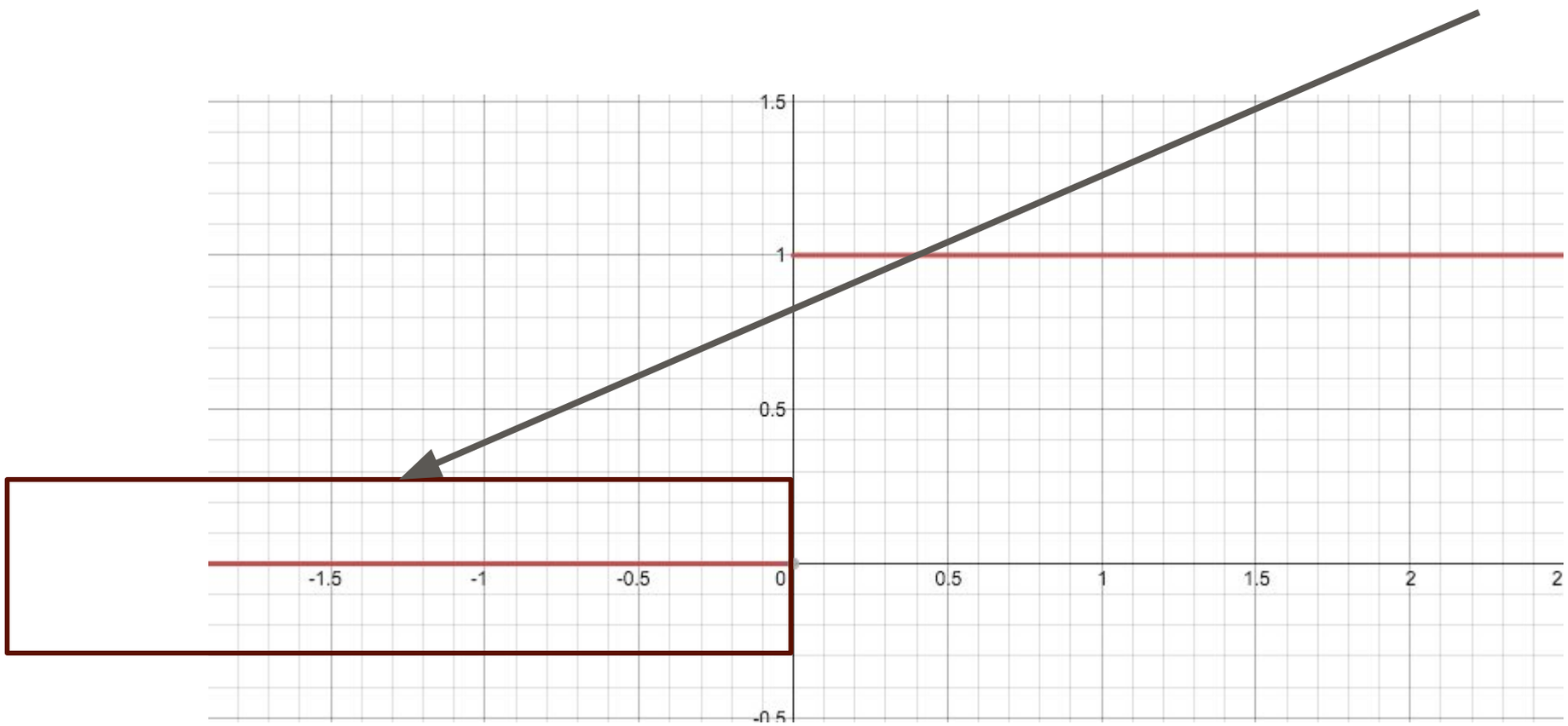
Question

Do you see any problem with the derivative of ReLU activation function?

ReLU Activation Function

- ReLU suffers from problem known as the ***dying ReLUs***
- For negative inputs derivative is zero

For negative inputs , the derivative is always 0



Derivative of ReLU activation function

ReLU Activation Function

Dying ReLUs

- Because of dying ReLUs, during training
  - Some neurons effectively die and
  - They stop outputting anything other than 0
  - It completely blocks the backpropagation

Question

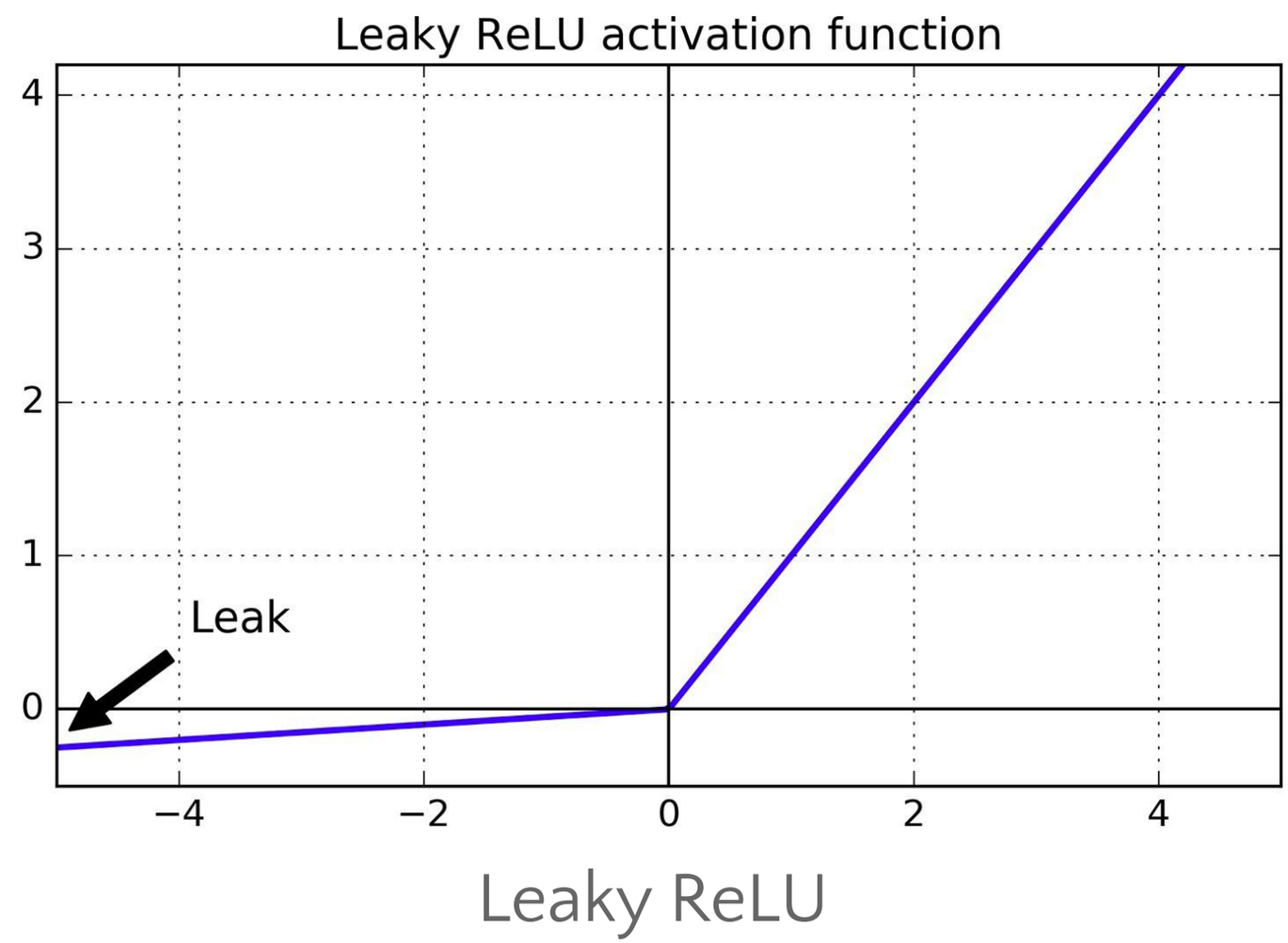
How do we solve dying ReLUs problem?

Leaky ReLU

- To solve dying ReLUs problem we use
  - Variant of ReLUs known as ***leaky ReLU***
- Leaky ReLU output a very small gradient when the input is negative

***α** = 0.01*
*RELU(x) = max(**α**x, x)*

α is the hyperparameter which defines how much the function "leaks" and is typically set to "0.01"



Leaky ReLU

- This small gradient ensures that the
  - Leaky ReLUs never die
- In the recent researches it has been shown that
  - Setting α = 0.2 (huge leak) results in better performance

Leaky ReLU

 # Leaky ReLU plot

```
>>> def leaky_relu(z, alpha=0.01):
>>>     return np.maximum(alpha*z, z)

>>> plt.plot(z, leaky_relu(z, 0.05), "b-", linewidth=2)
>>> plt.plot([-5, 5], [0, 0], 'k-')
>>> plt.plot([0, 0], [-0.5, 4.2], 'k-')
>>> plt.grid(True)
```

$\alpha = 0.01$

$RELU(x) = max(\alpha x, x)$

Leaky ReLU

 # Leaky ReLU plot

```
>>> props = dict(facecolor='black', shrink=0.1)
>>> plt.annotate('Leak', xytext=(-3.5, 0.5), xy=(-5,
-0.2), arrowprops=props, fontsize=14, ha="center")
>>> plt.title("Leaky ReLU activation function",
fontsize=14)
>>> plt.axis([-5, 5, -0.5, 4.2])
>>> plt.show()
```

a *= 0.01*
*RELU(x) = max(**a**x, x)*

Leaky ReLU

 # Implementing Leaky ReLU in Keras

```
>>> model = keras.models.Sequential([
        [...]
>>>         keras.layers.Dense(10,
kernel_initializer="he_normal"),
        keras.layers.LeakyReLU(alpha=0.2),
        [...]
])
```

Leaky ReLU

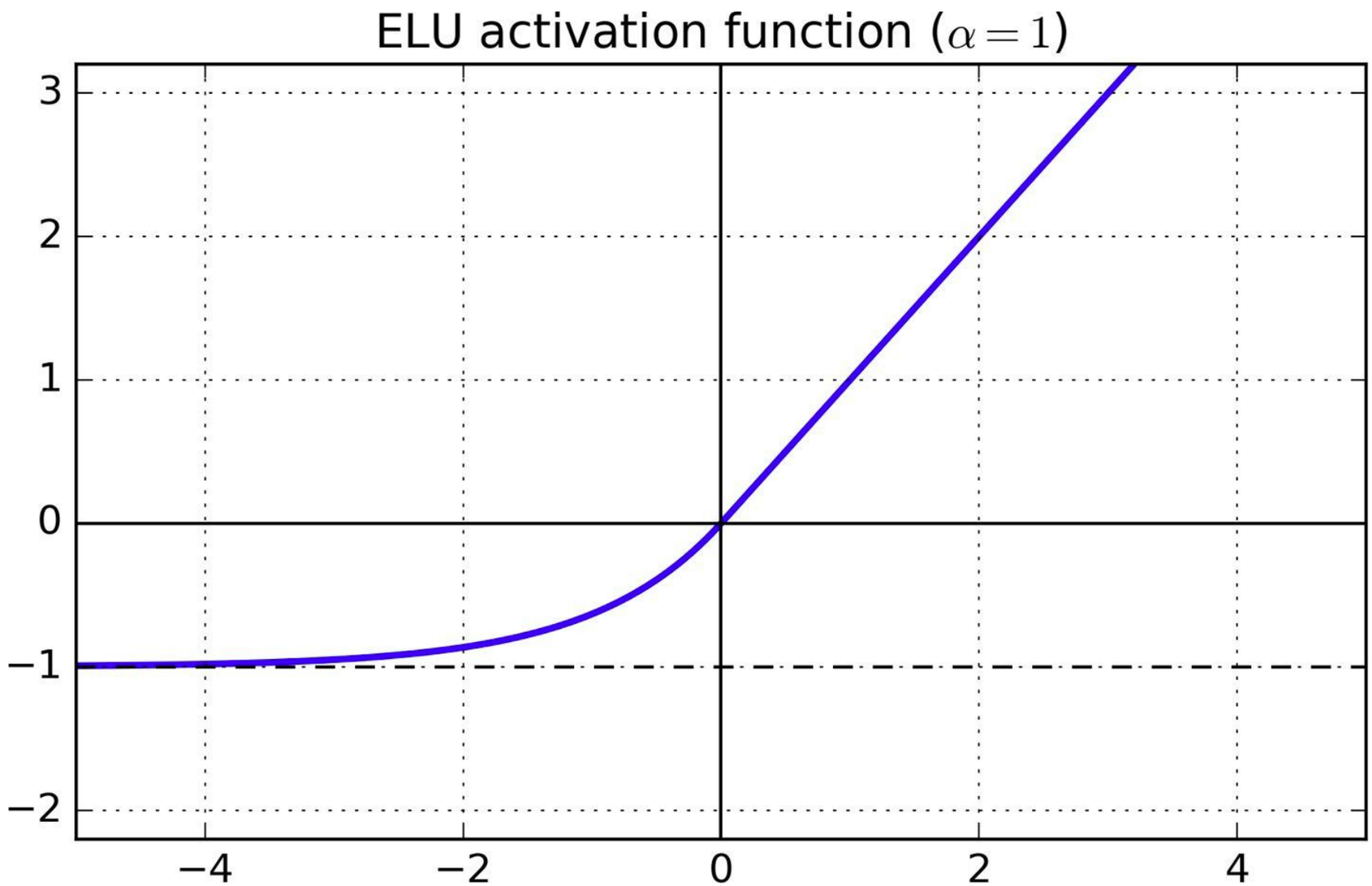
Follow the code in the notebook to train a neural network on MNIST using the Leaky ReLU

ELU Activation Function

- In 2015, Djork-Arné Clevert et al proposed a new activation function
  - ***ELU - Exponential Linear Unit***
- It outperformed all the ReLU variants in their experiments
  - Training time was reduced and
  - Neural network performed better on the test set

ELU Activation Function

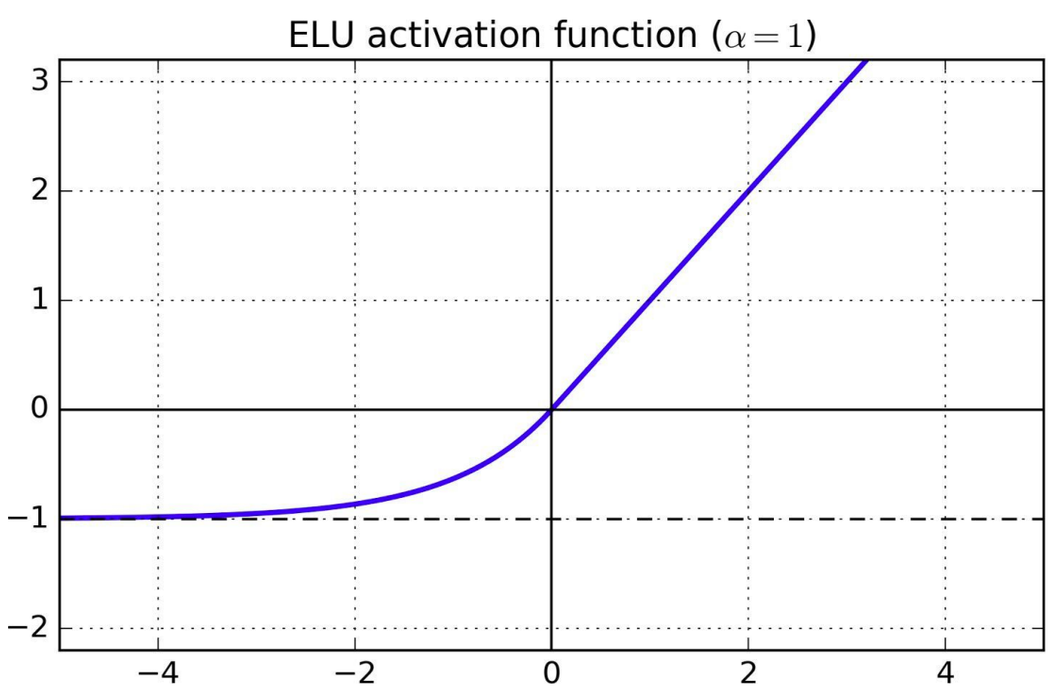
$$\mathrm{ELU}_\alpha(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & if z \geq 0 \end{cases}$$



CLOUD x LAB

ELU Activation Function

- In ELU equation, the hyperparameter α defines the value
  - That ELU function approaches to when z is a large negative number
  - α is usually set to 1
  - But we can tweak it like any other hyperparameter



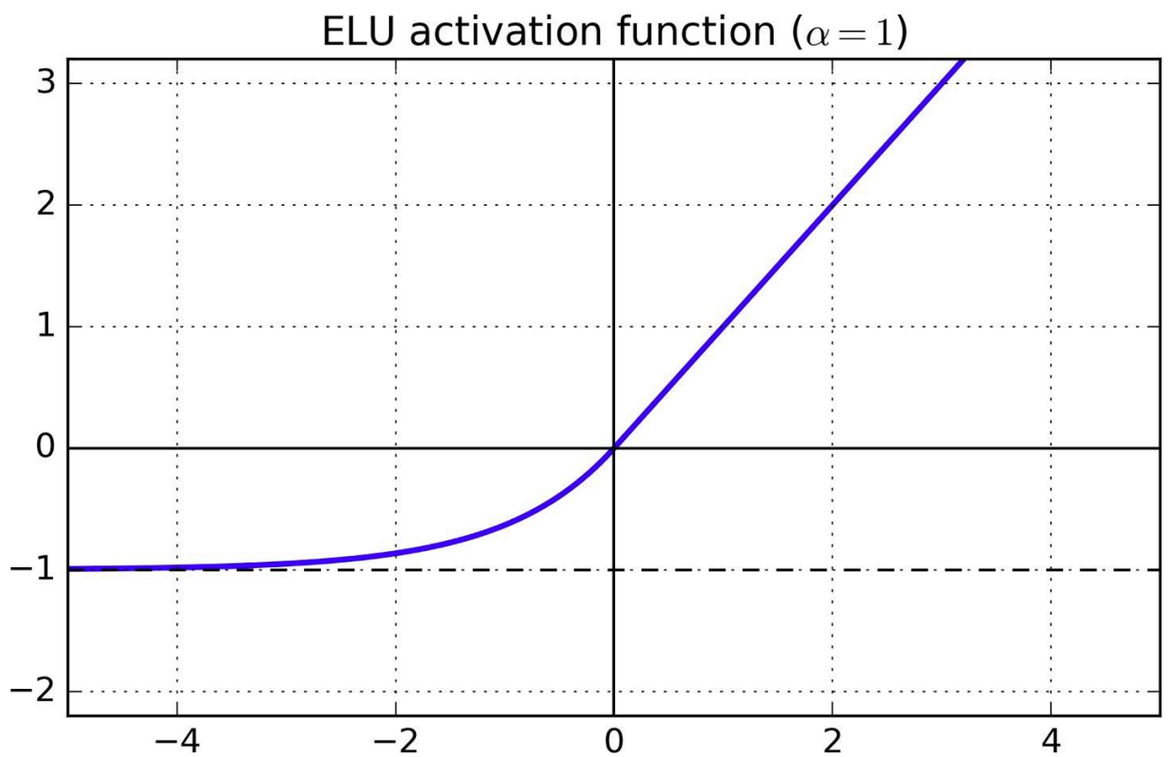
$$\mathrm{ELU}_\alpha(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & if z \geq 0 \end{cases}$$

CLOUD x LAB

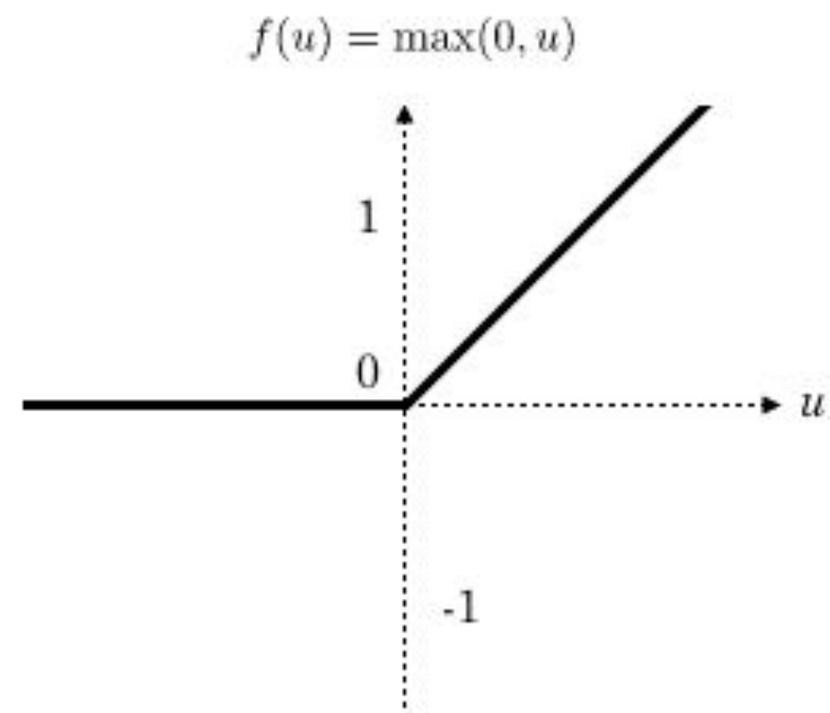
ELU Activation Function

Advantage over ReLU

- It has a nonzero gradient for z < 0
  - Which avoids the dying units issue



ELU



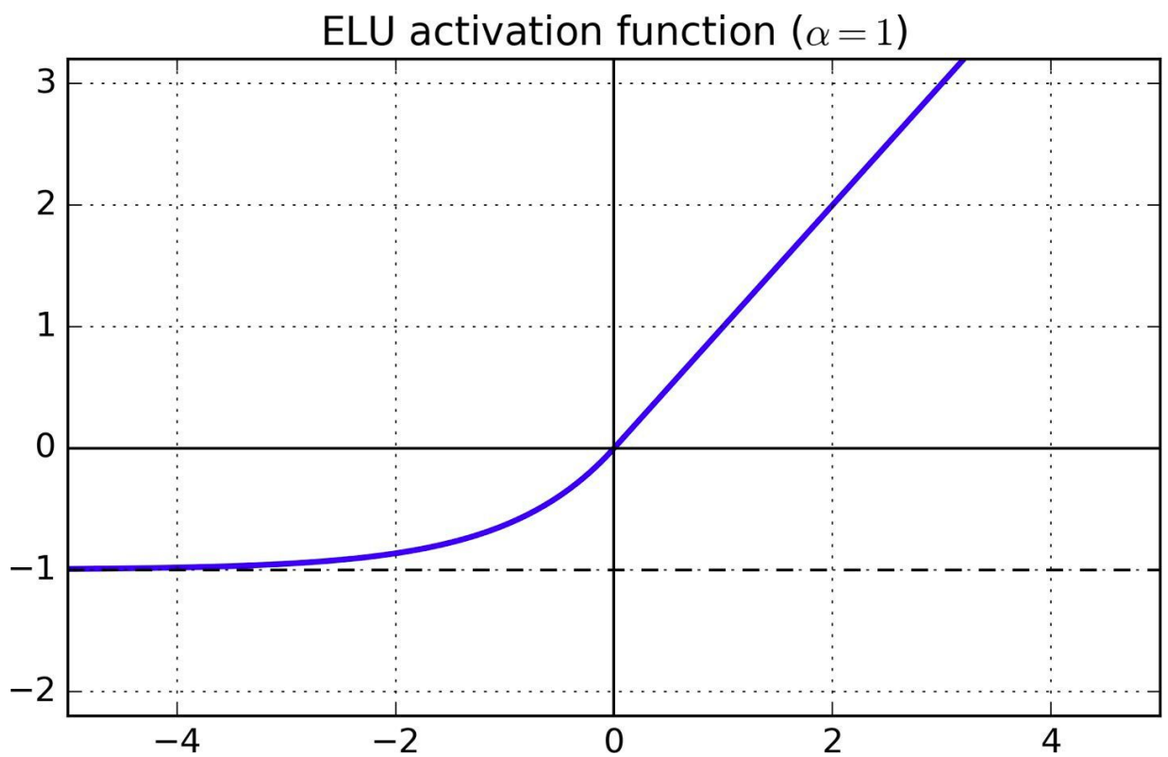
ReLU

CLOUD x LAB

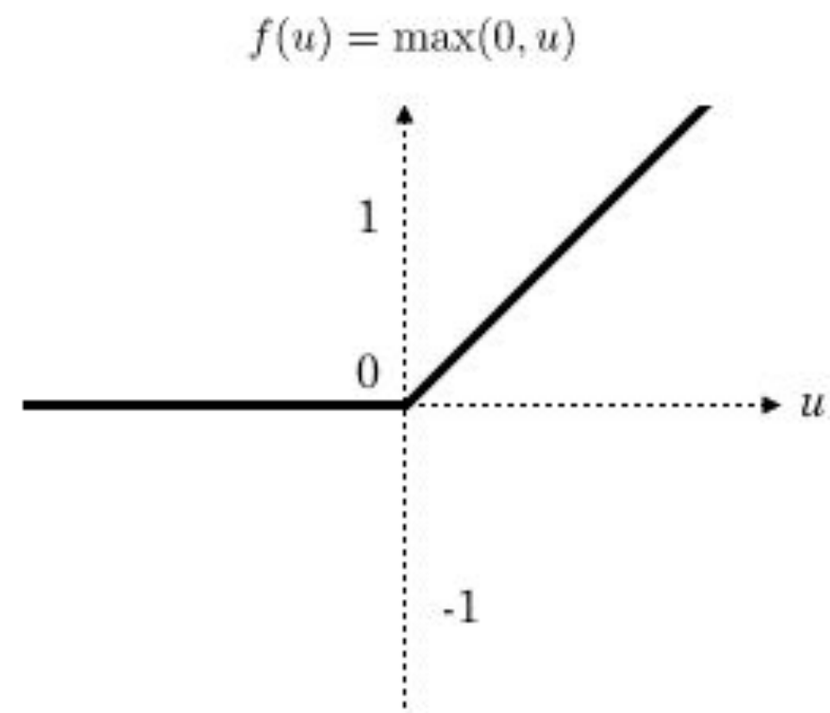
ELU Activation Function

Advantage over ReLU

- It is smooth everywhere including around z = 0
  - It helps speedup Gradient Descent



ELU



ReLU

CLOUD x LAB

ELU Activation Function

Drawbacks over ReLU

- Because of the use of exponential function
  - It is slower to compute than the ReLU
- But during training this slowness gets compensated by
  - The faster convergence rate
- However during testing
  - ELU networks are slower than the ReLU networks

$$\mathrm{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & if z \geq 0 \end{cases}$$

ELU Activation Function

 # ELU plot

```
>>> def elu(z, alpha=1):
>>>     return np.where(z < 0, alpha * (np.exp(z) - 1),
z)
```

ELU Activation Function

 # ELU plot

```
>>> plt.plot(z, elu(z), "b-", linewidth=2)
>>> plt.plot([-5, 5], [0, 0], 'k-')
>>> plt.plot([-5, 5], [-1, -1], 'k--')
>>> plt.plot([0, 0], [-2.2, 3.2], 'k-')
>>> plt.grid(True)
>>> plt.title(r"ELU activation function ($\alpha=1$)",
fontsize=14)
>>> plt.axis([-5, 5, -2.2, 3.2])
>>> plt.show()
```

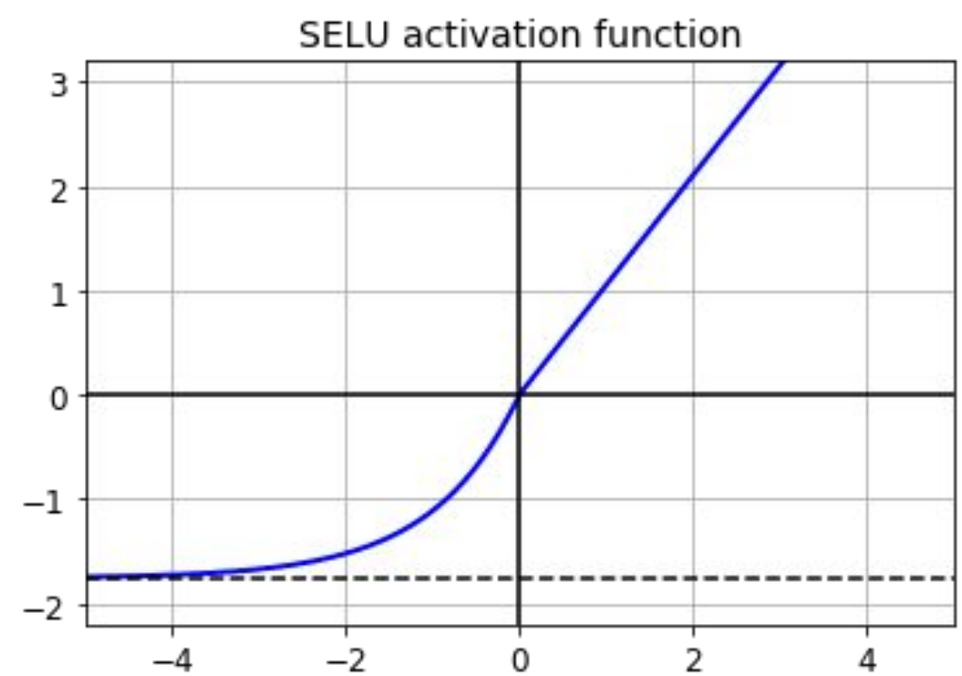
ELU Activation Function

 # Implementing ELU in Keras

```
>>> keras.layers.Dense(10, activation="elu")
```

SELU Activation Function

- In June 2017, Günter Klambauer, Thomas Unterthiner and Andreas Mayr
  - Proposed **SELU** activation function
  - It outperforms the other activation functions
  - Very significantly for deep neural networks
  - Even for 100 layer deep neural network



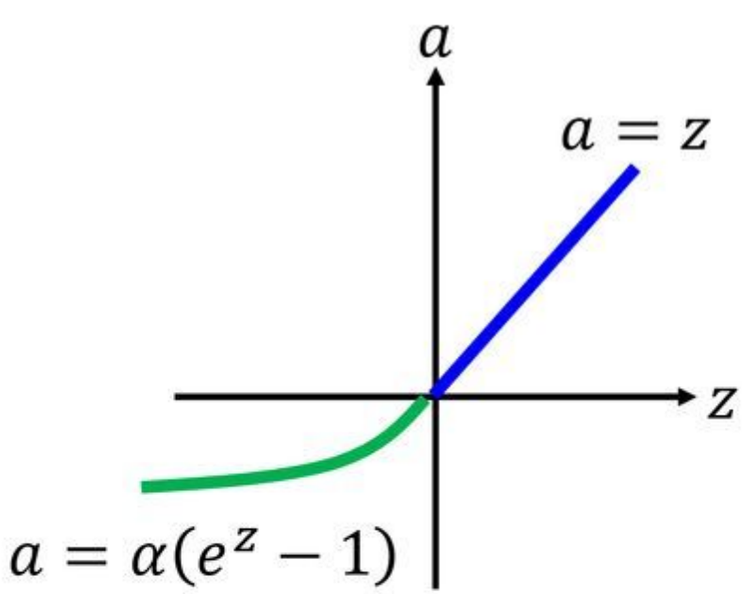
SELU Activation Function

(1) Definition of scaled exponential linear units (SELUs)

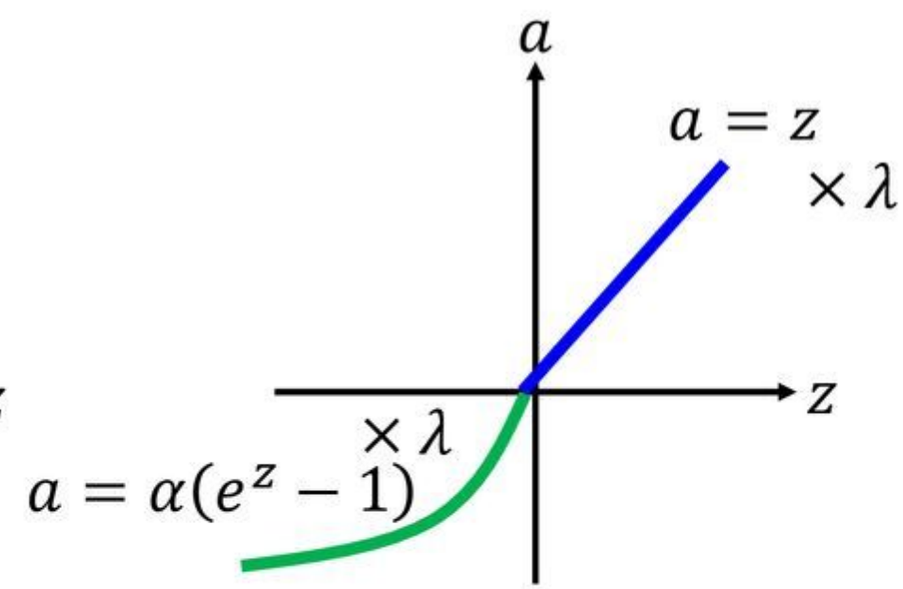
```
In [3]: def selu(x):
            with ops.name_scope('elu') as scope:
                alpha = 1.6732632423543772848170429916717
                scale = 1.0507009873554804934193349852946
                return scale*tf.where(x>=0.0, x, alpha*tf.nn.elu(x))
```

https://github.com/bioinf-jku/SNNs

Exponential Linear Unit (ELU)



Scaled ELU (SELU)



$$\alpha = 1.6732632423543772848170429916717$$
$$\lambda = 1.0507009873554804934193349852946$$

SELU Activation Function

SELU Function in Python

```
>>> from scipy.special import erfc
>>> alpha_0_1 = -np.sqrt(2 / np.pi) /
(erfc(1/np.sqrt(2)) * np.exp(1/2) - 1)
>>> scale_0_1 = (1 - erfc(1 / np.sqrt(2)) *
np.sqrt(np.e)) * np.sqrt(2 * np.pi) * (2 *
erfc(np.sqrt(2))*np.e**2 +
np.pi*erfc(1/np.sqrt(2))**2*np.e -
2*(2+np.pi)*erfc(1/np.sqrt(2))*np.sqrt(np.e)+np.pi+2)**(
-1/2)
```

SELU Activation Function

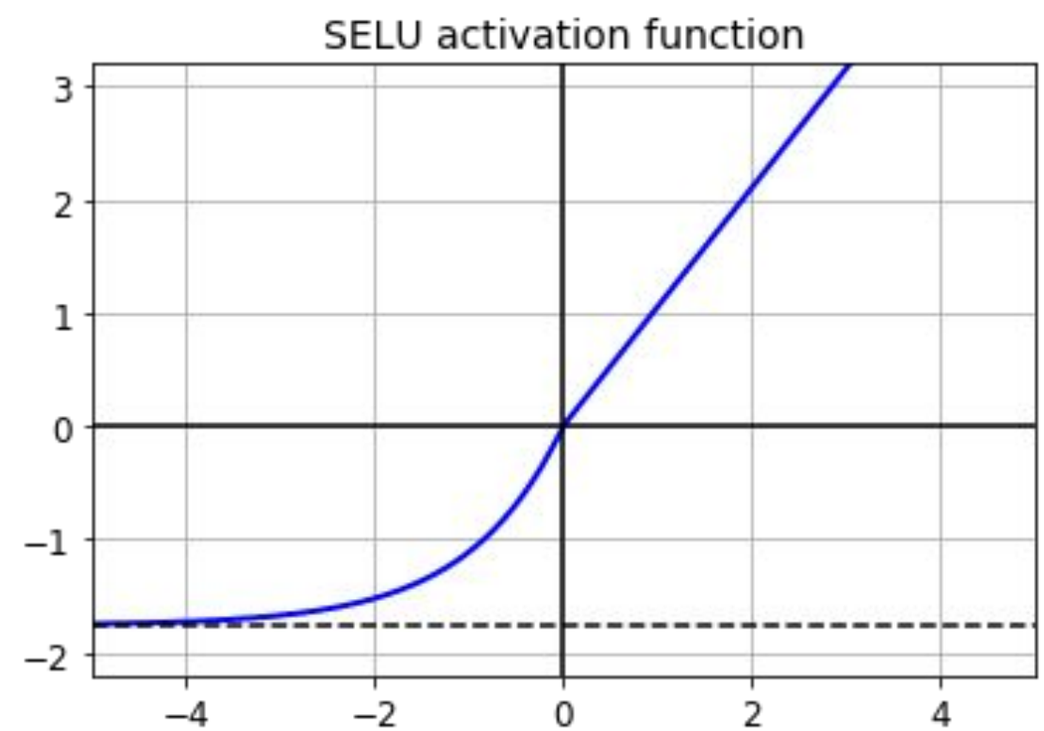
SELU Function in Python

```
>>> def selu(z, scale=scale_0_1, alpha=alpha_0_1):
>>>     return scale * elu(z, alpha)
```

SELU Activation Function

Plot SELU Function

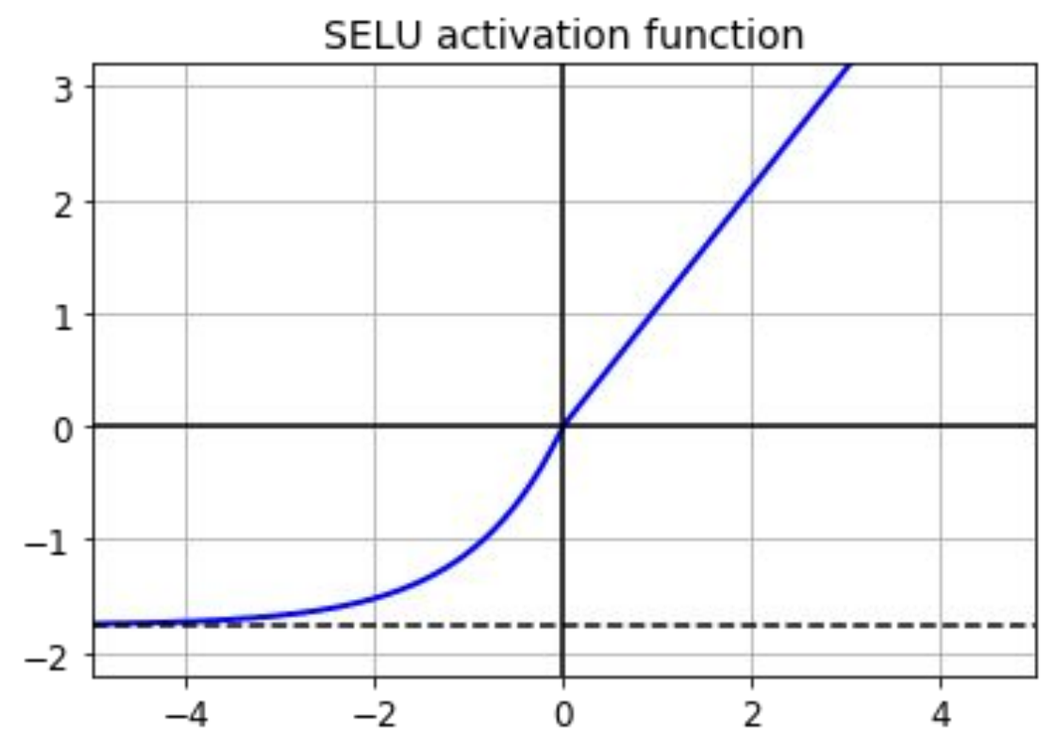
```
>>> plt.plot(z, selu(z), "b-", linewidth=2)
>>> plt.plot([-5, 5], [0, 0], 'k-')
>>> plt.plot([-5, 5], [-1.758, -1.758], 'k--')
>>> plt.plot([0, 0], [-2.2, 3.2], 'k-')
```



SELU Activation Function

Plot SELU Function

```
>>> plt.grid(True)
>>> plt.title("SELU activation function", fontsize=14)
>>> plt.axis([-5, 5, -2.2, 3.2])
>>> plt.show()
```



SELU Activation Function

- With this activation function
  - Even a 100 layer deep neural network
  - Preserves roughly mean 0 and standard deviation 1 across all layers
  - Avoiding the exploding/vanishing gradients problem

SELU Activation Function

Check the mean and standard deviation in the deep layers

```
>>> np.random.seed(42)
>>> Z = np.random.normal(size=(500, 100))
>>> for layer in range(1000):
>>>     W = np.random.normal(size=(100, 100),
scale=np.sqrt(1 / 100)) # LeCun initialization
>>>     Z = selu(np.dot(Z, W))
>>>     means = np.mean(Z, axis=0).mean()
>>>     stds = np.std(Z, axis=0).mean()
```

SELU Activation Function

Check the mean and standard deviation in the deep layers

```
>>>     if layer % 100 == 0:
>>>         print("Layer {}: mean {:.2f}, std deviation
{:.2f}".format(layer, means, stds))
```

SELU Activation Function

Follow the code in the notebook to create a neural net for MNIST using the SELU activation function

CLOUD x LAB

PRELU Activation Function

- In ELU equation, the hyperparameter α defines the value
  - if a_i=0, f becomes ReLU
  - if a_i>0, f becomes leaky ReLU
  - if a_i is a learnable parameter, f becomes PReLU
  - See https://arxiv.org/abs/1502.01852

$$f(y_i) = \begin{cases} y_i, & \text{if } y_i > 0 \\ a_i y_i, & \text{if } y_i \leq 0 \end{cases}$$

Question

So which activation function should we use?

Which Activation Function to Use?

Answer

In general,

SELU > ELU > Leaky ReLU > ReLU > tanh > $\boxed{\text{logistic}}$

Vanishing gradient

CLOUD x LAB

Which Activation Function to Use?

- If runtime performance is important then
  - Prefer Leaky ReLUs over ELUs
- Also instead of tweaking hyperparameter α
  - We may use default suggested values
    - 0.2 for the leaky ReLUs and
    - 1 for ELU
- If we have spare time and computing power
  - Use cross-validation to evaluate the other activation functions

Batch Normalization

Batch Normalization

- Using He initialization and proper activation functions
  - Like ELU or any variant of ReLU
  - Vanishing / exploding gradient problem significantly reduces
  - But there is no guarantee that
  - This problem will not come back during training
- In 2015, Sergey Ioffe and Christian Szegedy
  - Proposed a technique called **Batch Normalization (BN)**
  - To address the vanishing/exploding gradients problems

Batch Normalization

- Batch Normalization helps in
  - Vanishing gradient problem and
  - It also helps the neural network to learn faster
- Let’s understand Batch Normalization

Batch Normalization - Feature Scaling

- As discussed earlier in machine learning projects
  - Gradient Descent does not work well
  - If the input features are on different scales
  - Like say if we have number of miles individual has driven in last 5 years
    - This data can have a large varying scale
    - As someone might have driven 100, 000 miles
    - While other person might have driven 100 miles
    - So here the range is 100 - 100, 000

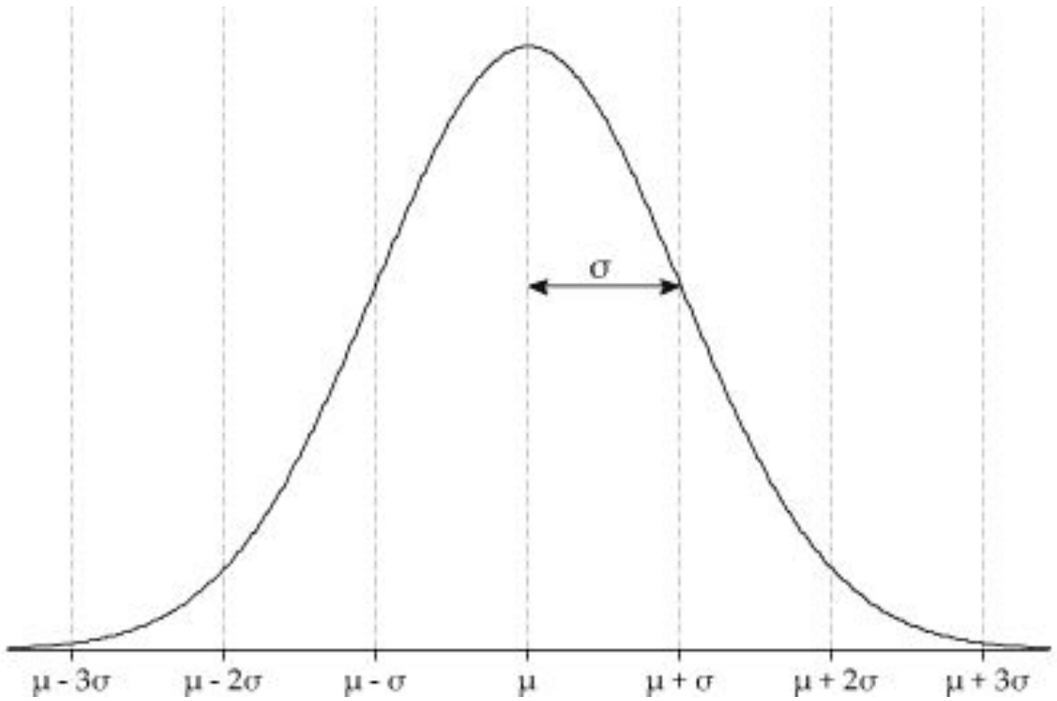
Batch Normalization – Feature Scaling

- One of the techniques of feature scaling is
  - Standardization
- In Standardization, features are rescaled
  - So that output will have the properties of
  - Standard normal distribution with
    - Zero mean and
    - Unit variance

$$\bar{x} = 0 \; and \; \sigma = 1$$

Mean

Standard Deviation



Batch Normalization – Feature Scaling

Standardization

- The general method of calculation
  - Calculate distribution mean and standard deviation for each feature
  - Subtract the mean from each feature
  - Divide the result from previous step of each feature by its standard deviation

$$x' = \frac{x - \bar{x}}{\sigma}$$

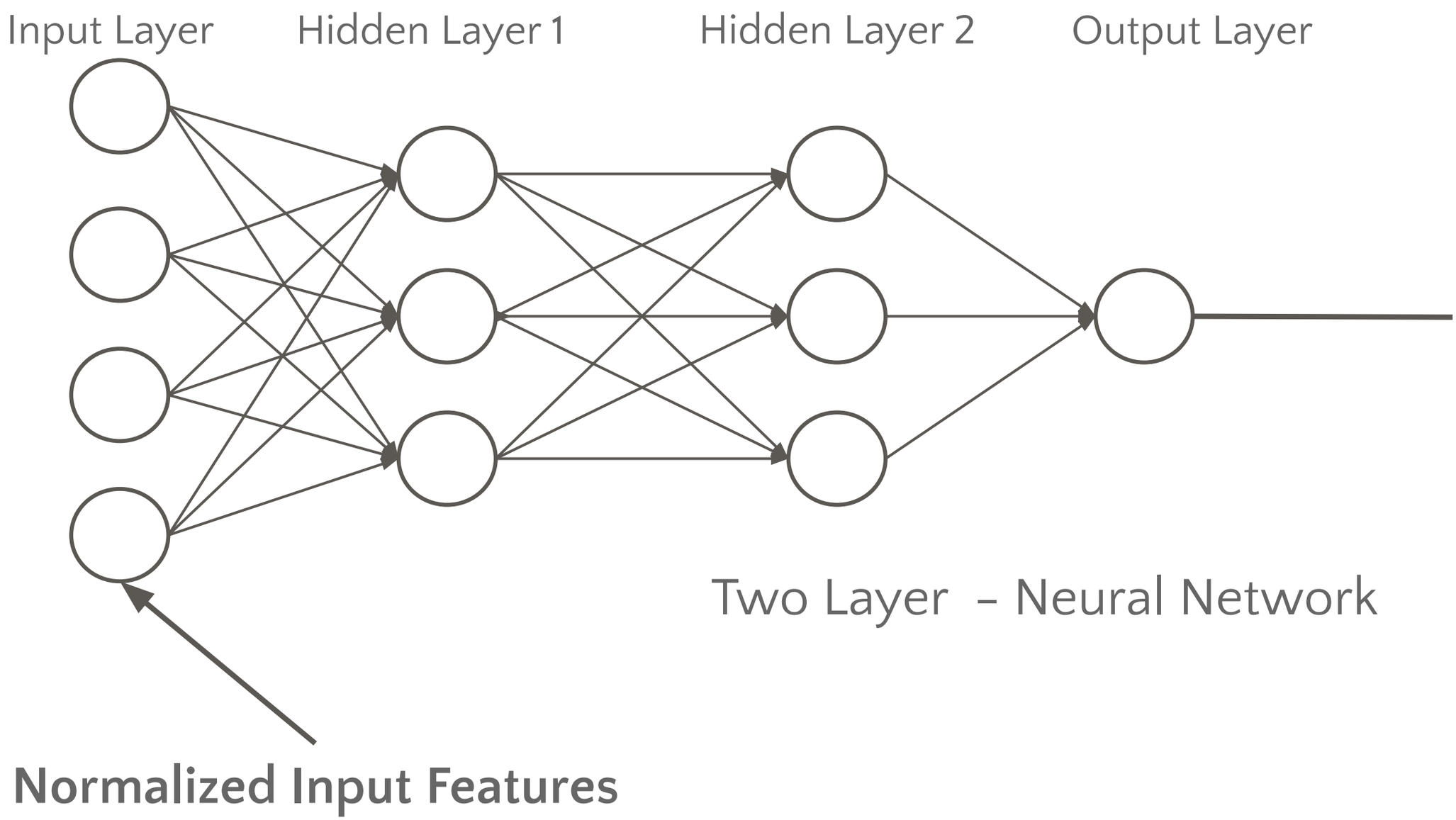
Standardized Value

Batch Normalization – Feature Scaling

Standardization

- As a preprocessing step
  - We apply standardization to the input dataset
  - So that all the features will have same scale
    - With 0 mean
    - And unit standard deviation
  - And Gradient Descent converges faster

Batch Normalization – Feature Scaling



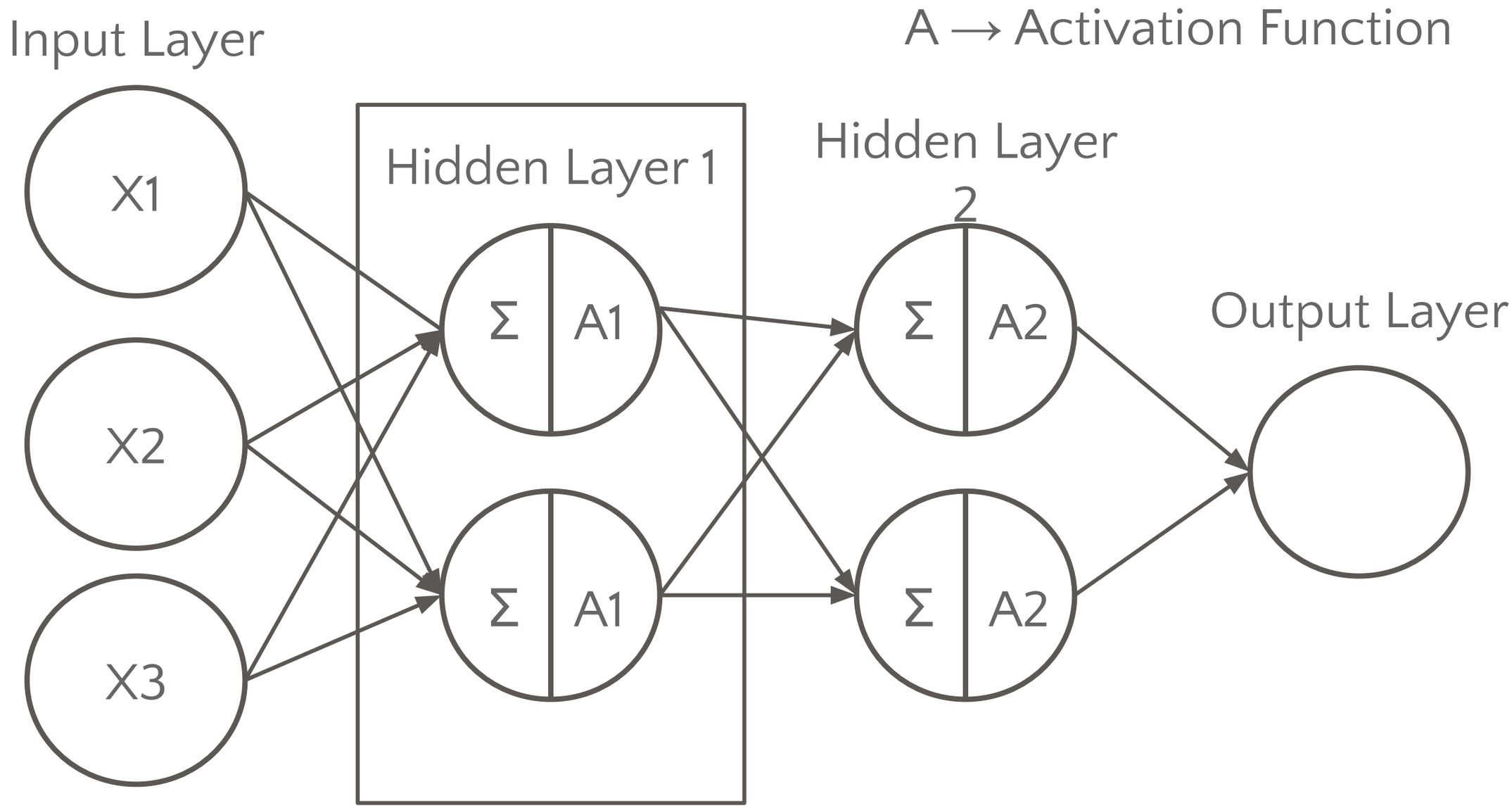
Batch Normalization - Feature Scaling

- As we have discussed, if we normalize input features
  - It helps in converging faster
- If we normalize hidden layers also in deep neural network
  - Then it will speed up the learning
  - This is what we do in Batch normalization
    - **We normalize hidden layers**
  - Now let's understand how do we do batch normalization in deep neural networks

CLOUD x LAB

Batch Normalization – Feature Scaling

A → Activation Function

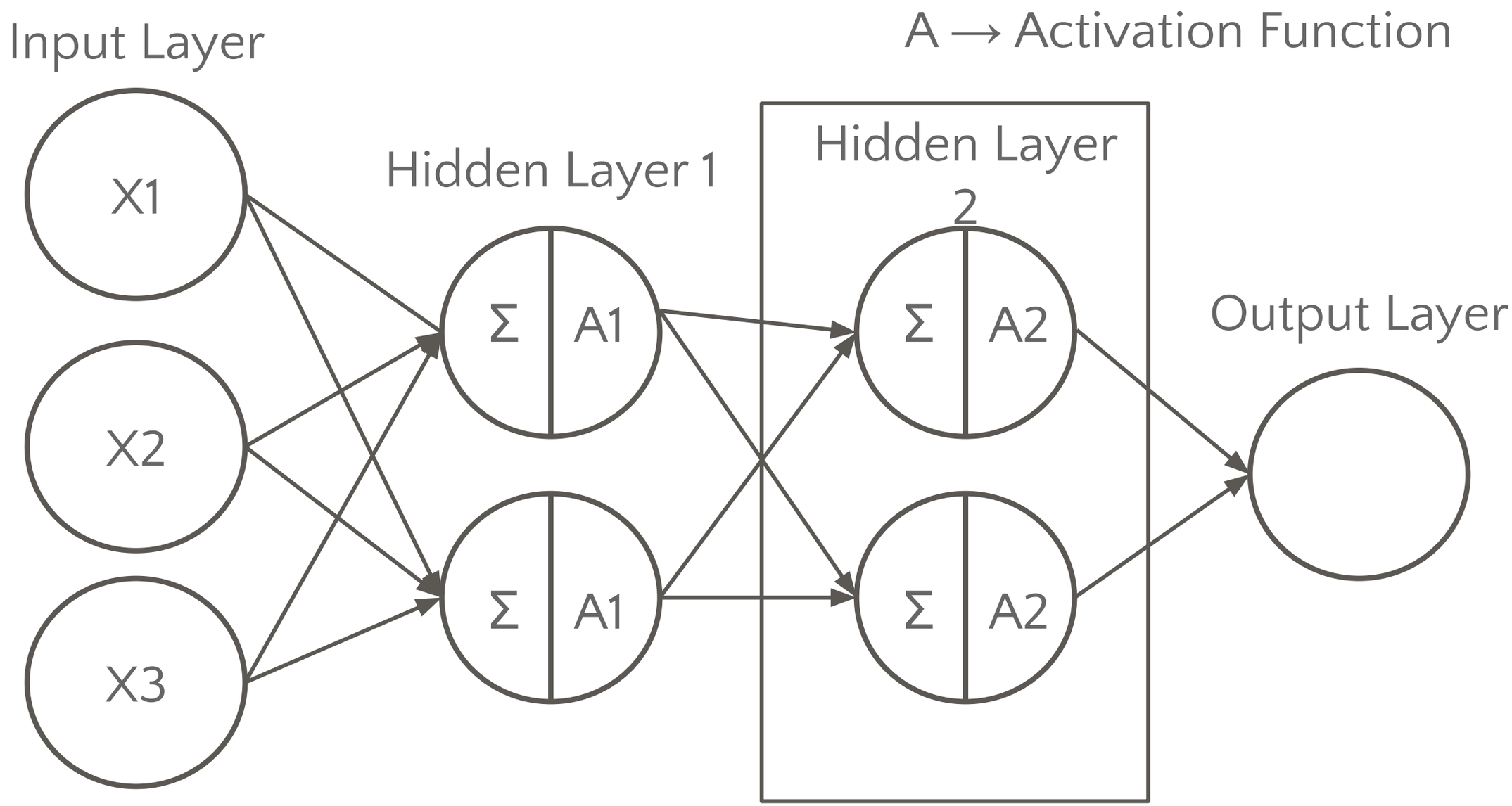


Hidden Layer 1

X → Σ → Batch Norm → A1 → Z1

CLOUD x LAB

Batch Normalization – Feature Scaling



Hidden Layer 2

X → Σ → Batch Norm → A1 → Z1 → Σ → Batch Norm → A2 → Z2

CLOUD x LAB

Batch Normalization - Algorithm

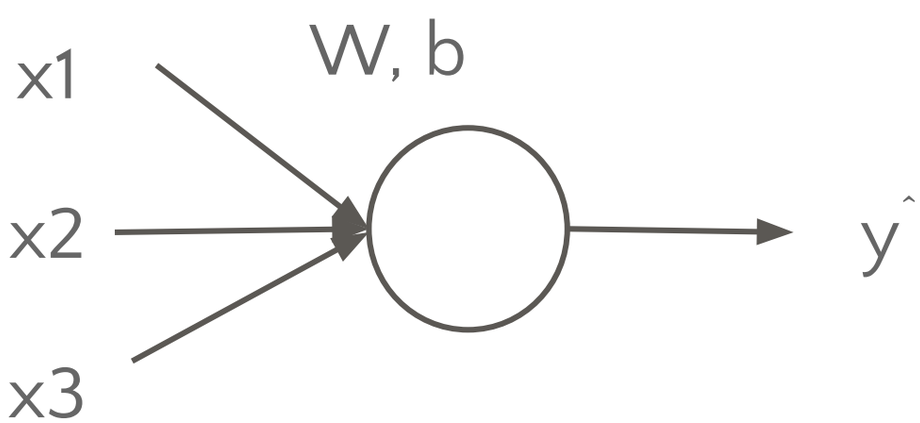
Algorithm

for T in 1 ……. number of mini batches:

- Compute forward propagation for mini-batch $X^{(T)}$
- In each hidden layer normalize inputs
- Use back propagation and update parameters

Batch Normalization - Feature Scaling

- Let's say we have a simple network and
- Here normalizing input features helps in Calculate W and b more efficiently



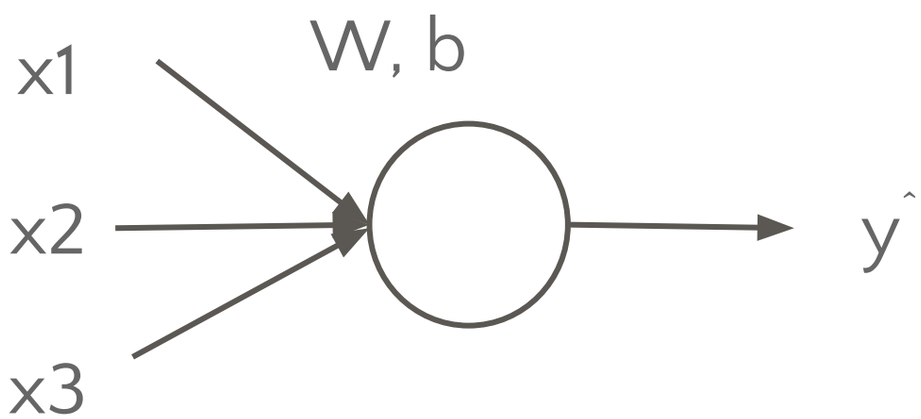
Step 1 - Calculate mean

$$\mu = \frac{1}{m}\sum_i X^{(i)}$$

CLOUD x LAB

Batch Normalization – Feature Scaling

- Normalize the input features



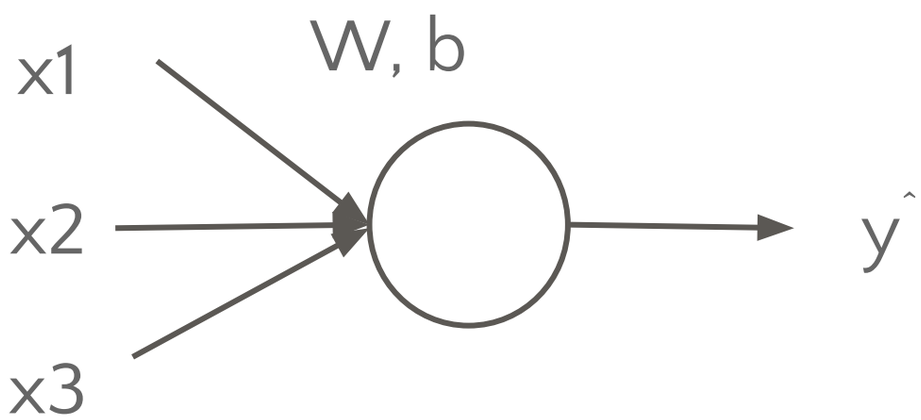
Step 1 – Calculate mean

$$\mu = \frac{1}{m}\sum_i X^{(i)}$$

CLOUD x LAB

Batch Normalization – Feature Scaling

- Normalize the input features



Step 1 – Calculate mean

$$\mu = \frac{1}{m}\sum_i X^{(i)}$$

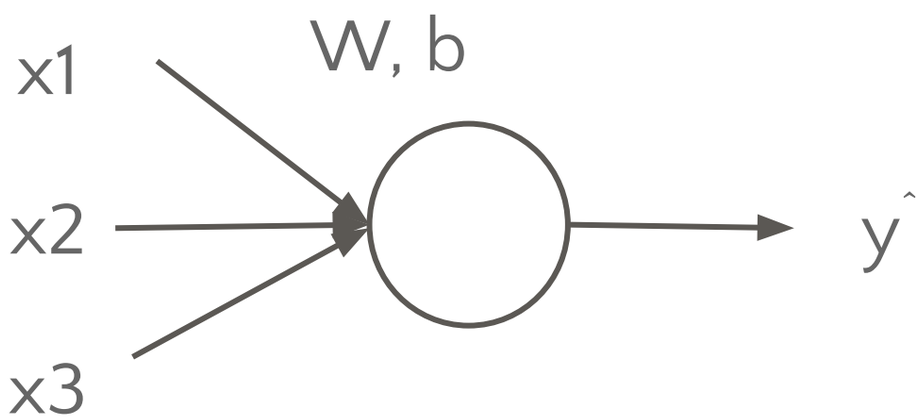
Step 2 – Calculate SD

$$\sigma^2 = \frac{1}{m}\sum_i (X^{(i)} - \mu)^2$$

CLOUD x LAB

Batch Normalization – Feature Scaling

- Normalize the input features



Step 1 – Calculate mean

$$\mu = \frac{1}{m}\sum_i X^{(i)}$$

Step 2 – Calculate SD

$$\sigma^2 = \frac{1}{m}\sum_i (X^{(i)} - \mu)^2$$

Step 3 – Normalize

$$X^{(i)} = \frac{X^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

μ_B is the mean, evaluated over the whole mini-batch B

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B^{\,2} = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^{\,2} + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \hat{\mathbf{x}}^{(i)} + \beta$$

$\boldsymbol{\sigma}_{\mathbf{B}}$ is the standard deviation, evaluated over the whole mini-batch B

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

$\mathbf{m_B}$ is number of instances in the mini-batch B

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

$x^{(i)}$ is the normalized output

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

ε is a tiny small number to avoid division by zero

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

γ and **β** are parameters which are learnt during training

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

$\mathbf{z}^{(i)}$ is the output of the BN operations.

CLOUD x LAB

Batch Normalization – Feature Scaling

- Generalize previous steps for the deep neural network

1. $$\mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

2. $$\sigma_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$$

3. $$\mathbf{x}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$$

4. $$\mathbf{z}^{(i)} = \gamma \mathbf{x}^{(i)} + \beta$$

- In general four parameters are trained for each batch-normalized layer
  - μ (mean) – moving
  - σ (SD) – moving
  - γ and
  - β

CLOUD x LAB

Question

At the test time how do we test the deep neural network trained with batch normalization as there will not be any mini batch to compute the mean and standard deviation?

Answer

By computing the moving average of whole training set's mean and standard deviation during training

Follow code in the notebook to implement

Batch Normalization with Keras

Batch Normalization

Drawbacks

- In batch normalization,
  - The neural network makes slower predictions
  - Due to the extra computations required at each layer
- If we need fast predictions
  - We should first check
    - How Plain ELU + He initialization performs
    - Before playing with batch normalization

Gradient Clipping

Gradient Clipping

- We can reduce the exploding gradients problem
  - By clipping the gradients during backpropagation
  - So that they never exceed some threshold
  - This is called **Gradient Clipping**

Gradient Clipping

- This technique is most often used in
  - Recurrent Neural Networks
  - as Batch Normalization is tricky to use in RNNs

Gradient Clipping

How to apply Gradient Clipping?

In Keras, implementing Gradient Clipping is just a matter of setting the clipvalue or clipnorm argument when creating an optimizer, like this:

```
>>> optimizer = keras.optimizers.SGD(clipvalue=1.0)
>>> model.compile(loss="mse", optimizer=optimizer)
```


CLOUD x LAB

Gradient Clipping

- This optimizer will
  - clip every component of the **gradient vector**
  - to a value between **–1.0 and 1.0**
- This means that all the partial derivatives of the loss (with regard to each and every trainable parameter) will be clipped between –1.0 and 1.0.
- The threshold is a hyperparameter you can tune

Note: This may change the orientation of the gradient vector.

Gradient Clipping

How does the gradient vector change orientation?

- If the original gradient vector is **[0.9, 100.0]** it points mostly in the direction of the second axis
- If you clip it by value, you get **[0.9, 1.0]**, which points roughly in the diagonal between the two axes.

Gradient Clipping

How to avoid the gradient vector change orientation?

- If you want to ensure that Gradient Clipping does not change the direction of the gradient vector, you should
  - Clip by norm by setting clipnorm instead of clipvalue
  - This will clip the whole gradient if its $\ell 2$ norm is greater than the threshold you picked.

Gradient Clipping

How to avoid the gradient vector change it's orientation?

- For example:
  - If you set clipnorm=1.0
  - then the vector **[0.9, 100.0]** will be clipped to **[0.00899964, 0.9999595]**, preserving its orientation but almost eliminating the first component.

Reusing Pretrained Layers

Reusing Pretrained Layers

Transfer Learning

- It is not a good idea to train a very large DNN from scratch
- We should find an existing neural network if possible
  - Which accomplishes a similar task we are trying to tackle
- If we can find such network
  - Then just reuse the lower layers (early layers) of this network
  - This is called **Transfer Learning**

Reusing Pretrained Layers

Transfer Learning

- There are two major advantages of Transfer Learning
  - It speeds up training considerably
  - It requires much less training data

Reusing Pretrained Layers

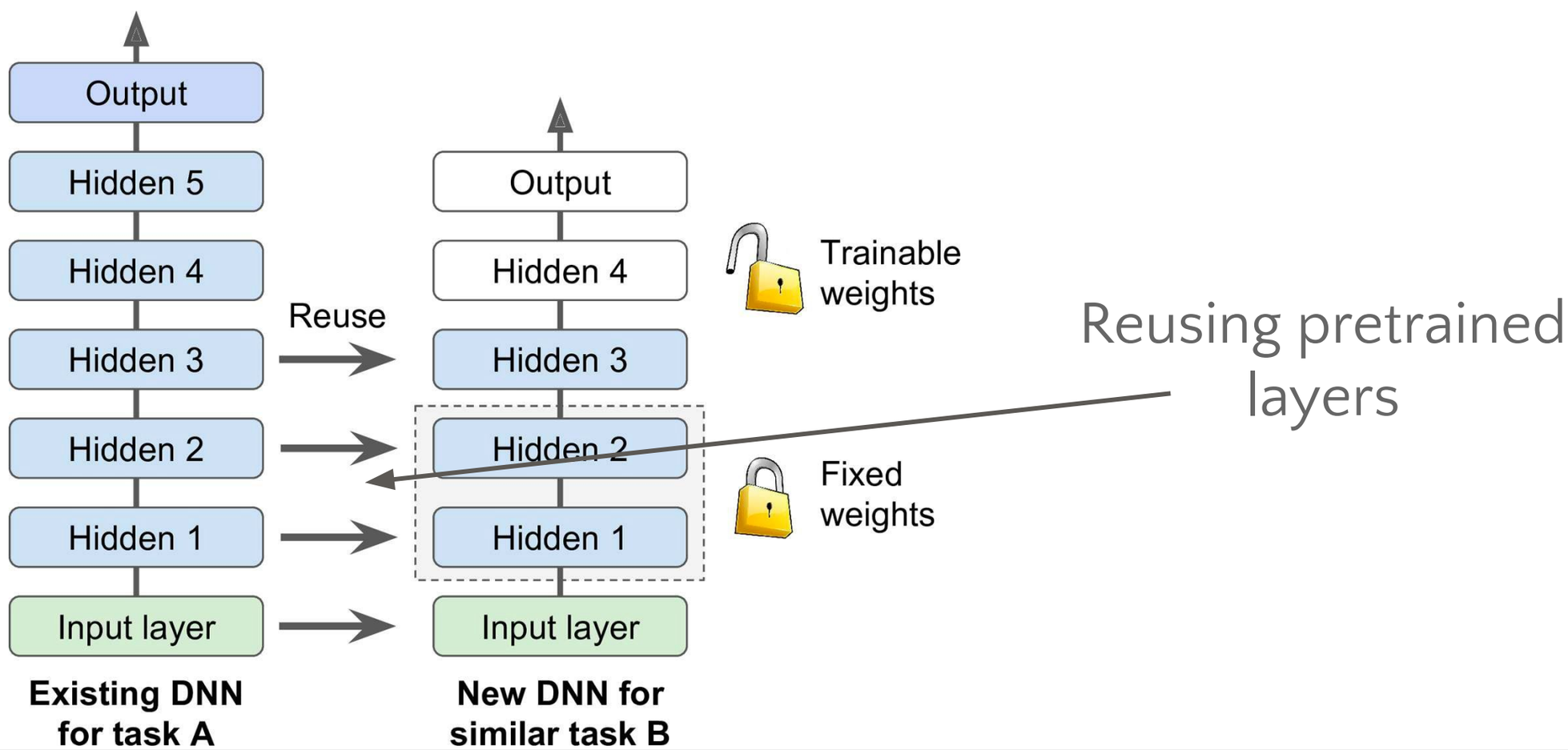
Transfer Learning – Examples

- Let's say we have found an existing DNN
  - That was trained to classify pictures
  - Into 100 different categories like
    - Animals,
    - Plants,
    - Vehicles and
    - Everyday objects

Reusing Pretrained Layers

Transfer Learning – Examples

- Now we want to train a DNN to classify specific types of vehicles
- These tasks are similar to existing DNN and
- We should try to reuse the pretrained layers of the existing network



CLOUD x LAB

Reusing Pretrained Layers

Transfer Learning

- If input pictures in our task do not have the same size as the one in the existing network
- Then we have to add a preprocessing step to resize them to the size
  - As expected by the existing model
- Also transfer learning works only when inputs in our task
  - Have similar low-level features as in the existing model

Reusing Pretrained Layers

Reusing a Keras Model

- First, you need to load model A and create a new model based on that model's layers. Let's reuse all the layers except for the output layer:

```
>>> model_A = keras.models.load_model("my_model_A.h5")
>>> model_B_on_A =
keras.models.Sequential(model_A.layers[:-1])
>>> model_B_on_A.add(keras.layers.Dense(1,
activation="sigmoid"))
```

Reusing Pretrained Layers

- Note that `model_A` and `model_B_on_A` now share some layers. When you train `model_B_on_A`, it will also affect `model_A`.
- If you want to avoid that, you need to clone `model_A` before you reuse its layers.

Reusing Pretrained Layers

- To do this, you clone model A's architecture with `clone_model()`, then copy its weights (since `clone_model()` does not clone the weights):

```
>>> model_A_clone = keras.models.clone_model(model_A)
>>> model_A_clone.set_weights(model_A.get_weights())
```

Reusing Pretrained Layers

Let's see example of how to reuse a Keras model

Reusing Pretrained Layers

Reusing a Keras Model – Step 1

- To reuse the model
  - First we need to load model A
  - and create a new model based on that model's layers.

```
>>> model_A = keras.models.load_model("my_model_A.h5")
>>> model_B_on_A =
keras.models.Sequential(model_A.layers[:-1])
>>> model_B_on_A.add(keras.layers.Dense(1,
activation="sigmoid"))
```

Reusing Pretrained Layers

Reusing a Keras Model – Step 2

- Next, freeze the reused layers during the first few epochs
  - giving the new layer some time to learn reasonable weights since
  - the new output layer was initialized randomly it will make large errors
  - so there will be large error gradients

```
>>> for layer in model_B_on_A.layers[:-1]:
>>> layer.trainable = False
>>> model_B_on_A.compile(loss="binary_crossentropy",
optimizer="sgd", metrics=["accuracy"])
```

Reusing Pretrained Layers

Reusing a Keras Model – Step 3

- Now you can train the model for a few epochs
  - then unfreeze the reused layers
  - and continue training to fine-tune the reused layers for task B

```
>>> history = model_B_on_A.fit(X_train_B, y_train_B,
epochs=4, validation_data=(X_valid_B, y_valid_B))
>>> for layer in model_B_on_A.layers[:-1]:
>>> layer.trainable = True
```

Reusing Pretrained Layers

Reusing a Keras Model – Step 3

```
>>> optimizer = keras.optimizers.SGD(lr=1e-4) # the
default lr is 1e-2
>>> model_B_on_A.compile(loss="binary_crossentropy",
optimizer=optimizer, metrics=["accuracy"])
>>> history = model_B_on_A.fit(X_train_B, y_train_B,
epochs=16, validation_data=(X_valid_B, y_valid_B))
```

Reusing Pretrained Layers

Reusing a Keras Model – Step 4

- Now you can evaluate the model

```
>>> model_B_on_A.evaluate(X_test_B, y_test_B)
[0.06887910133600235, 0.9925]
```

Reusing Pretrained Layers

Model Zoos

CLOUD x LAB

Model Zoos

- As we discussed we can reuse the existing pretrained neural network for our new tasks
- But where can we find a trained neural network for the task similar to ours?

Model Zoos

- The first place is to look our own catalog of models
  - This is why we should save all our models and
  - Organize them properly so that
  - We can retrieve them later
- Another option is to search in a **model zoo**
  - Many people after training their models
  - Release the trained models to the public

Model Zoos

- **Keras Applications** are deep learning models that are made available alongside pre-trained weights.
- These models can be used for
  - **prediction**
  - **feature extraction**
  - **Fine-tuning**
- Below is the link to the Keras Applications page:

  https://keras.io/api/applications/

Model Zoos

- TensorFlow has its own model zoo available at
  - https://github.com/tensorflow/models
- It contains most of the image classification nets such as
  - VCG, Inception and ResNet
    - Including the code
    - The pretrained models and
    - Tools to download popular image datasets

Model Zoos

- Another popular model zoo is Caffe's Model Zoo
  - https://github.com/BVLC/caffe/wiki/Model-Zoo
- It contains many computer vision models trained on various datasets
- We can also use below converter
  - To convert Caffe models to TensorFlow models
  - https://github.com/ethereon/caffe-tensorflow

Reusing Pretrained Layers

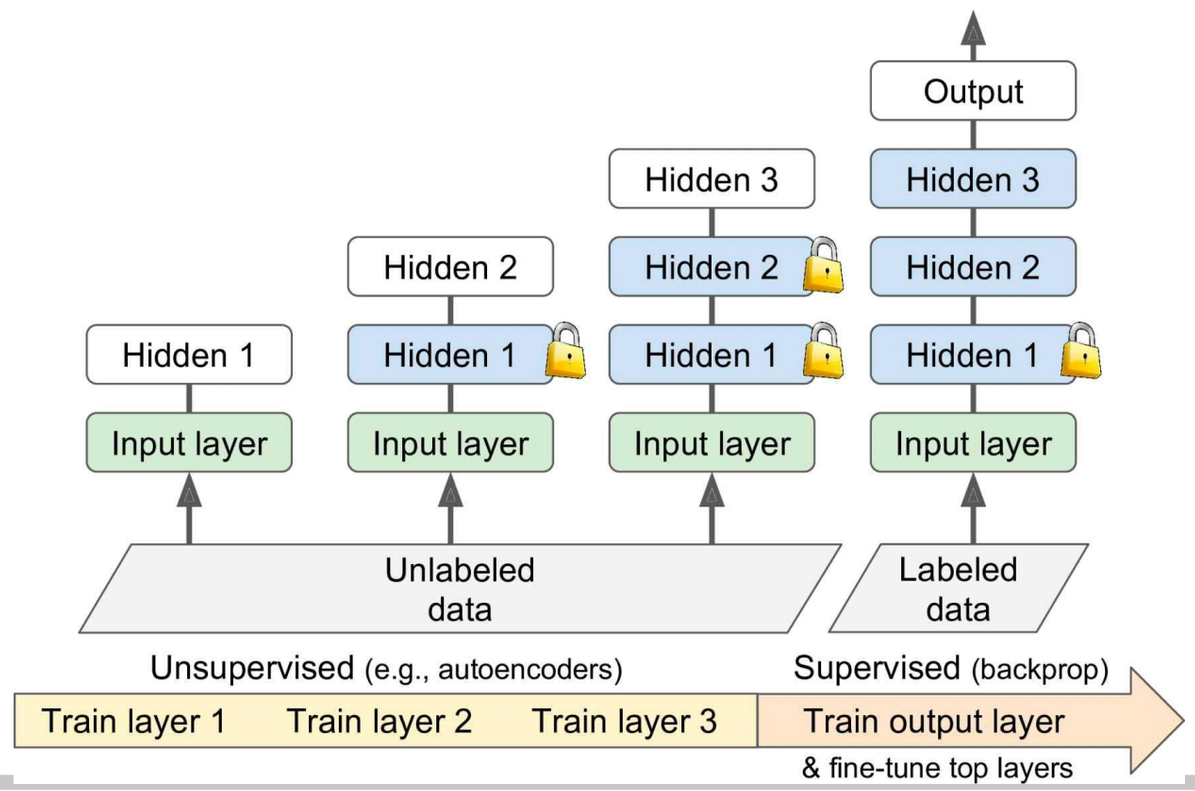
Unsupervised Pretraining

Unsupervised Pretraining

- If we want to train a model for complex task
  - And we do not have much labeled training data
  - Also we could not find a pretrained model on similar task
- Then in this case how should we tackle the task?

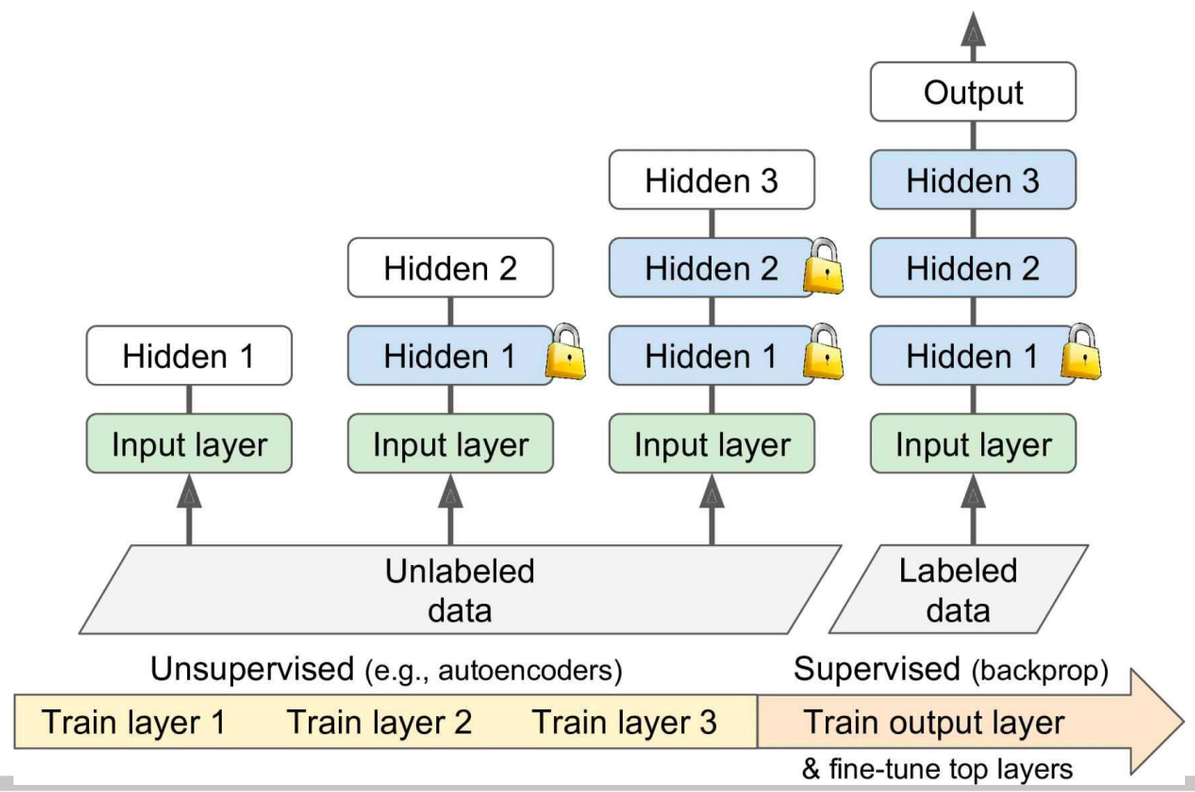
Unsupervised Pretraining

- Try to gather more labeled training data
  - But if it is too hard or too expensive to get the training data
  - Then try to perform **unsupervised pretraining**



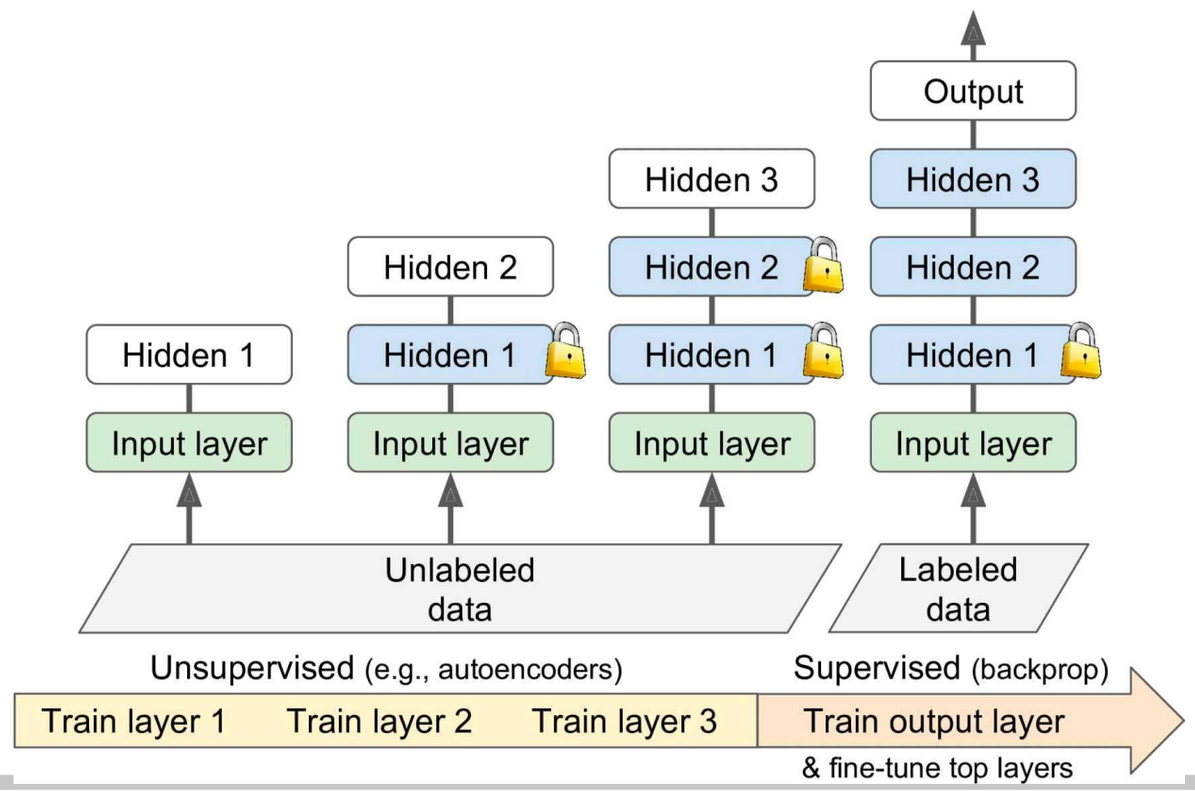
Unsupervised Pretraining

- If we have plenty of unlabelled training data then
  - Try to train the layers one by one
  - Starting with the lowest layer and then going up
  - Using an unsupervised feature detector algorithm such as
    - Restricted Boltzmann Machines (RBMs) or autoencoders



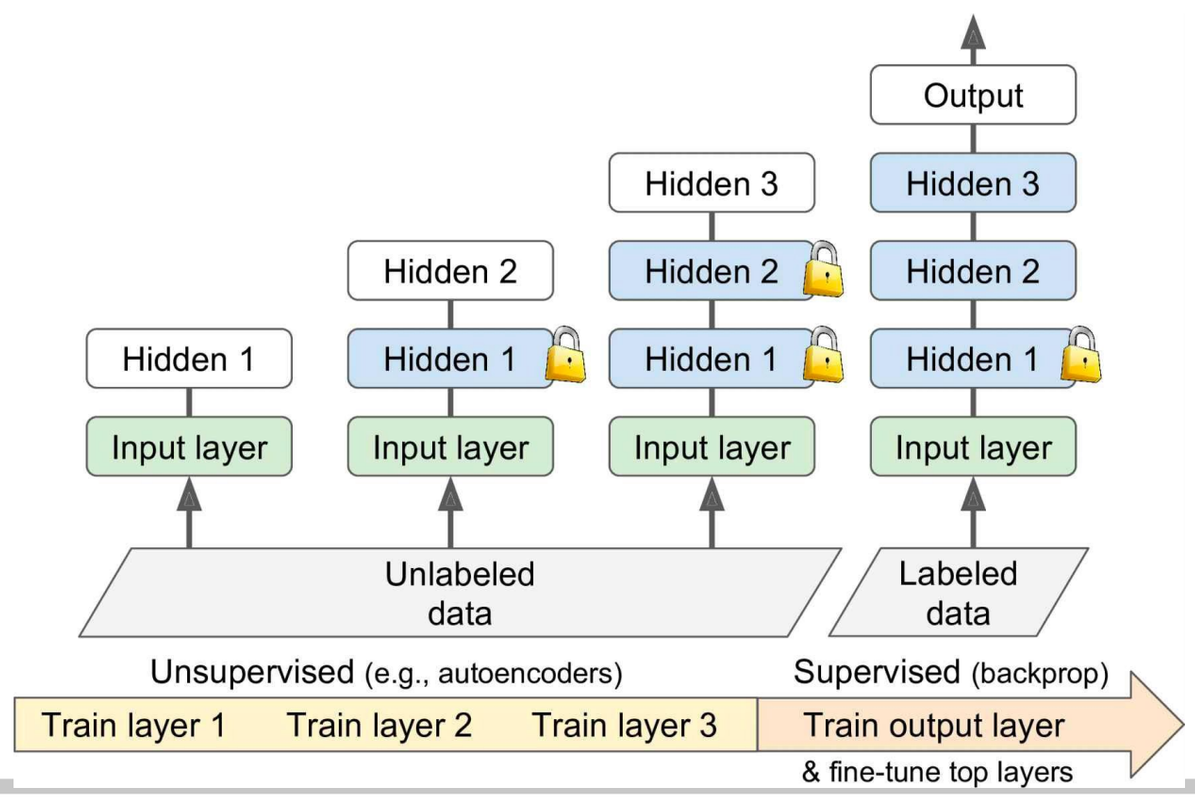
Unsupervised Pretraining

- Each layer is trained on the output of the
  - Previously trained layers
  - All layers except the one being trained are frozen



Unsupervised Pretraining

- Once all layers have been trained
  - We can fine-tune the network
  - Using supervised learning (with backpropagation)
- This is the long and tedious process
  - But often works well



Unsupervised Pretraining

- This technique was used by Geoffrey Hinton and his team in 2006
- It led to the revival of neural networks and the success of Deep Learning
- Until 2010, unsupervised pretraining (typically using RBMs)
  - Was the norm for deep nets
- Only after the vanishing gradients problem was alleviated
  - It became much more common to train
  - DNNs purely using backpropagation

Unsupervised Pretraining

- Unsupervised pretraining
  - Using autoencoders than RBM (Restricted Boltzmann Machines) is still a good option when we have complex task to solve
    - And no similar pretrained model is available
    - And there is a little labeled training data but lot of unlabeled training data is available

Reusing Pretrained Layers

Pretraining on an Auxiliary Task

Pretraining on an Auxiliary Task

- Let's say we want to build a system to recognize faces
- And as a training set
  - We may only have few pictures of each individual
  - Clearly not enough training set to train a good classifier
  - And gathering hundred of pictures of each person will not be practical

Solution??

Pretraining on an Auxiliary Task

Solution -

- We can download a lot of pictures of random people from internet
- And train a first neural network to detect
  - If two different pictures are of the same person
- Such a network would learn good feature detectors for faces
- So reusing its lower layers would allow us to train
  - A good face classifier
  - Using little training data which we had

Pretraining on an Auxiliary Task

- It is cheap to gather unlabeled training data
  - Like in previous example
  - We could download images from internet for almost free
  - But it is quite expensive to label them
- A common technique is to
  - Label all the training examples as “good”
  - And then generate many new labeled training instances
  - By corrupting the good ones and
  - Label these corrupted instances as bad

Pretraining on an Auxiliary Task

- And then we can train neural network
  - To classify these instances good or bad
- For example
  - Download millions of sentences
  - Then label all of them as “good”
  - Then randomly change a world in each sentence
  - And label the resulting sentence as “bad”

Pretraining on an Auxiliary Task

- Now if neural network can tell that
  - “The dog sleeps” is a good sentence and
  - “The dog they” is a bad sentence
  - Then it probably knows a lot about language
- Reusing its lower layers will help in many language processing tasks

Faster Optimizers

Faster Optimizers

What are the improvements do you suggest to gradient descent?

New Weight = Old weight – Eta * Rate of change in error wrt weight

Faster Optimizers

1. Training a deep neural network can be painfully slow
2. So far we have seen four ways to speedup training
   2.1. Applying a good initialization strategy for the connection weights
   2.2. Using a good activation function
   2.3. Using Batch Normalization
   2.4. Reusing parts of a pretrained network

Faster Optimizers

- Speed boost also comes from using a faster optimizer
  - Than the Gradient Descent optimizer
- Popular optimizers are
  - Momentum optimization
  - Nesterov Accelerated Gradient
  - AdaGrad
  - RMSProp and
  - Adam optimization

Increasing order of performance

Faster Optimizers – Momentum optimization

Analogy

- Imagine a bowling ball rolling down a gentle slope on a smooth surface
- It will start out slowly, but it will quickly pick up momentum until it eventually reaches terminal velocity.
- This is the very simple idea behind Momentum optimization, proposed by Boris Polyak in 1964

Faster Optimizers – Momentum optimization

How is Momentum optimization different from Gradient Descent

- Regular Gradient Descent will simply take **small regular steps** down the slope, so it will take much more time to reach the bottom.
- Gradient Descent simply updates the weights θ by directly subtracting the gradient of the cost function $J(\theta)$ with regards to the weights ($\nabla_\theta J(\theta)$) multiplied by the learning rate η.

Faster Optimizers – Momentum optimization

How is Momentum optimization different from Gradient Descent

- The equation of Gradient descent is: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$.
- It **does not care** about what the earlier gradients were. If the local gradient is tiny, it goes very slowly
- Momentum optimization **cares a great deal** about what previous gradients were

CLOUD x LAB

Faster Optimizers – Momentum optimization

How does Momentum optimization work ?

- At each iteration, it adds the local gradient to the **momentum vector m**, multiplied by the learning rate **η**,
- And it updates the weights by simply subtracting this momentum vector.

1. $$\mathbf{m} \leftarrow \beta \mathbf{m} + \eta \nabla_{\theta} J(\theta)$$
2. $$\theta \leftarrow \theta - \mathbf{m}$$

CLOUD x LAB

Faster Optimizers – Momentum optimization

How does Momentum optimization work ?

- In other words, the gradient is used as an acceleration, not as a speed.
- To simulate some sort of friction mechanism and prevent the momentum from growing too large, the algorithm introduces a new **hyperparameter β**, simply called the **momentum**, which must be set between 0 (high friction) and 1 (no friction).
- A typical momentum value is 0.9.

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} + \eta \nabla_{\theta} J(\theta)$$

$$2. \quad \theta \leftarrow \theta - \mathbf{m}$$

CLOUD x LAB

Faster Optimizers – Momentum optimization

Advantages of Momentum optimization

- Gradient Descent goes down the steep slope quite fast, but then it takes a very long time to go down the valley.
- Whereas Momentum optimization will roll down the bottom of the valley faster and faster until it reaches the bottom (the optimum)

CLOUD x LAB

Faster Optimizers – Momentum optimization

Advantages of Momentum optimization

- In deep neural networks that don't use Batch Normalization, the upper layers will often end up having inputs with very different scales, so using Momentum optimization helps a lot.
- It can also help roll past local optima.

CLOUD x LAB

Faster Optimizers – Momentum optimization

Disadvantage of Momentum optimization

- The one drawback of Momentum optimization is that it adds yet another hyperparameter to tune.
- However, the momentum value of 0.9 usually works well in practice and almost always goes faster than Gradient Descent.

Faster Optimizers – Momentum optimization

Implementing Momentum optimization

Implementing Momentum optimization in Keras is easy: just use the SGD optimizer and set its momentum hyperparameter

```
>>> optimizer = keras.optimizers.SGD(lr=0.001,
momentum=0.9)
```

Faster Optimizers - Nesterov Accelerated Gradient

- It is a small variant to Momentum optimization, proposed by Yurii Nesterov in 1983, is almost always faster than vanilla Momentum optimization.

Faster Optimizers – Nesterov Accelerated Gradient

- The idea of Nesterov Momentum optimization, or Nesterov Accelerated Gradient (NAG), is to
  - Measure the gradient of the cost function not at the local position but slightly ahead in the direction of the momentum.
  - The only difference from vanilla Momentum optimization is that the gradient is measured at **θ + βm** rather than at **θ**

1. $\mathbf{m} \leftarrow \beta \mathbf{m} + \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$
2. $\theta \leftarrow \theta - \mathbf{m}$

CLOUD x LAB

Faster Optimizers – Nesterov Accelerated Gradient

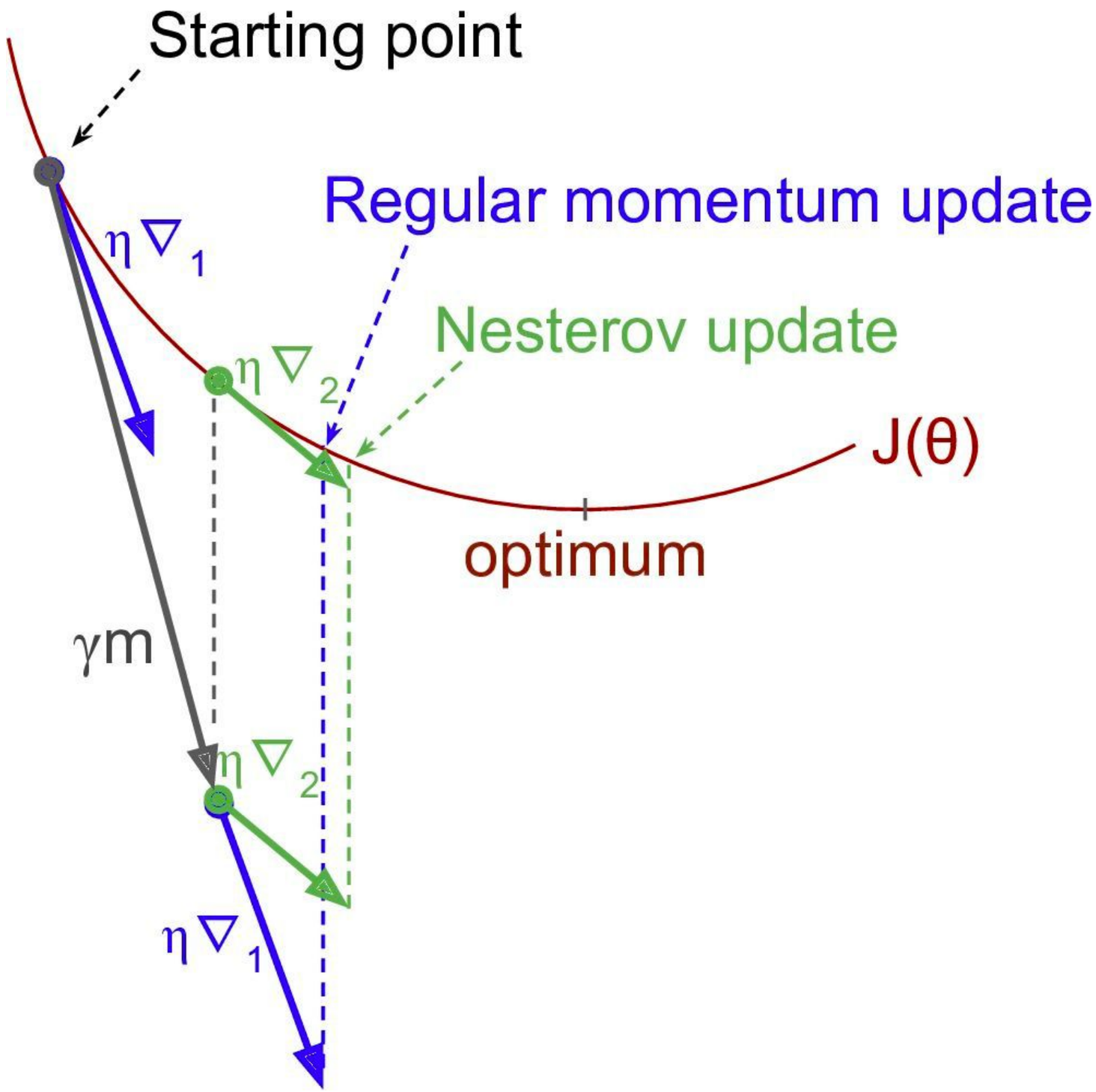
- This small tweak works because in general the momentum vector will be pointing in the right direction (i.e., toward the optimum),
- So it will be slightly more accurate to use the gradient measured a bit farther in that direction rather than using the gradient at the original position

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} + \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$$

$$2. \quad \theta \leftarrow \theta - \mathbf{m}$$

CLOUD x LAB

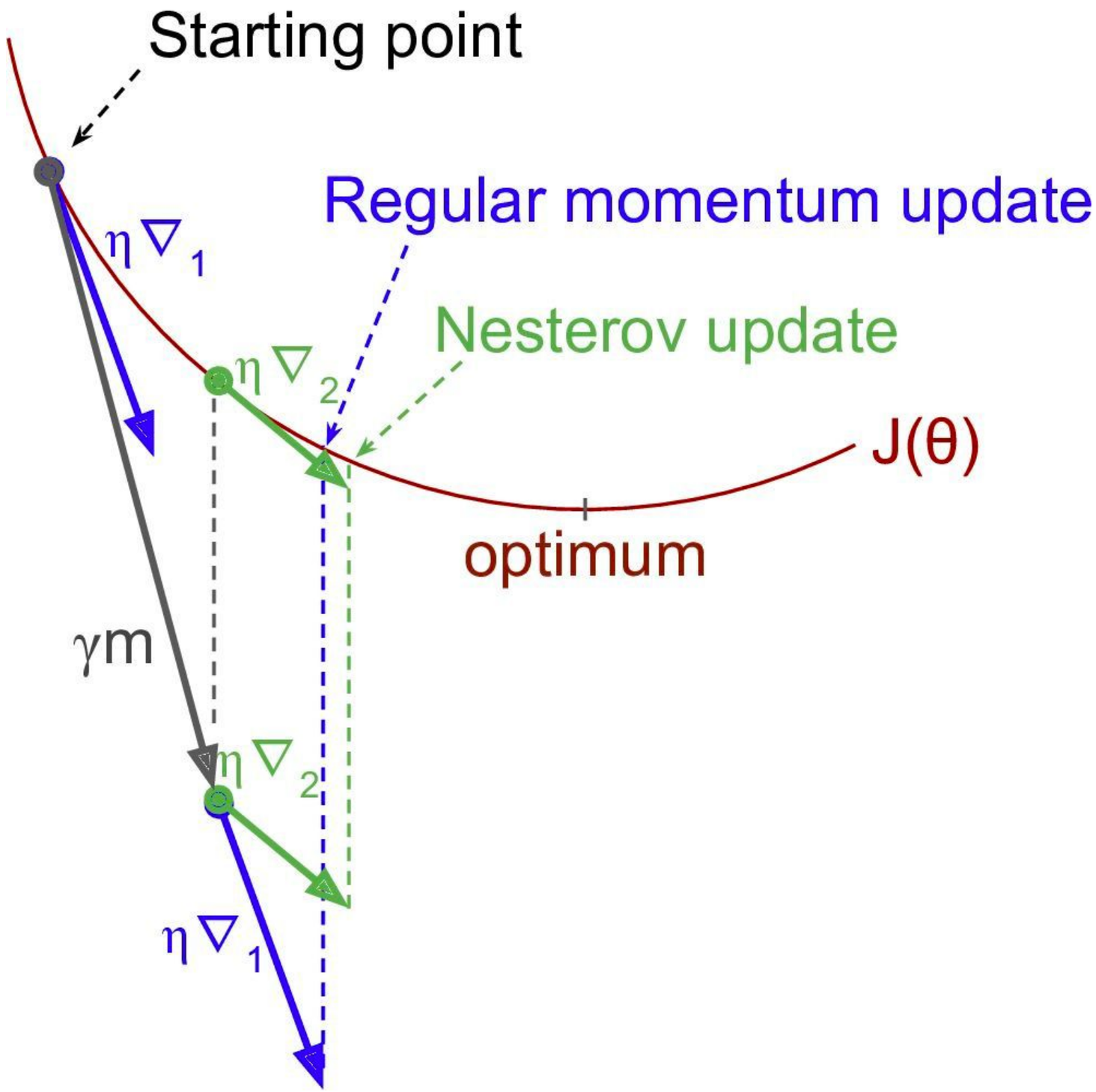
Faster Optimizers – Nesterov Accelerated Gradient



- ∇1 represents the gradient of the cost function measured at the starting point θ
- ∇2 represents the gradient at the point located at θ + βm

CLOUD x LAB

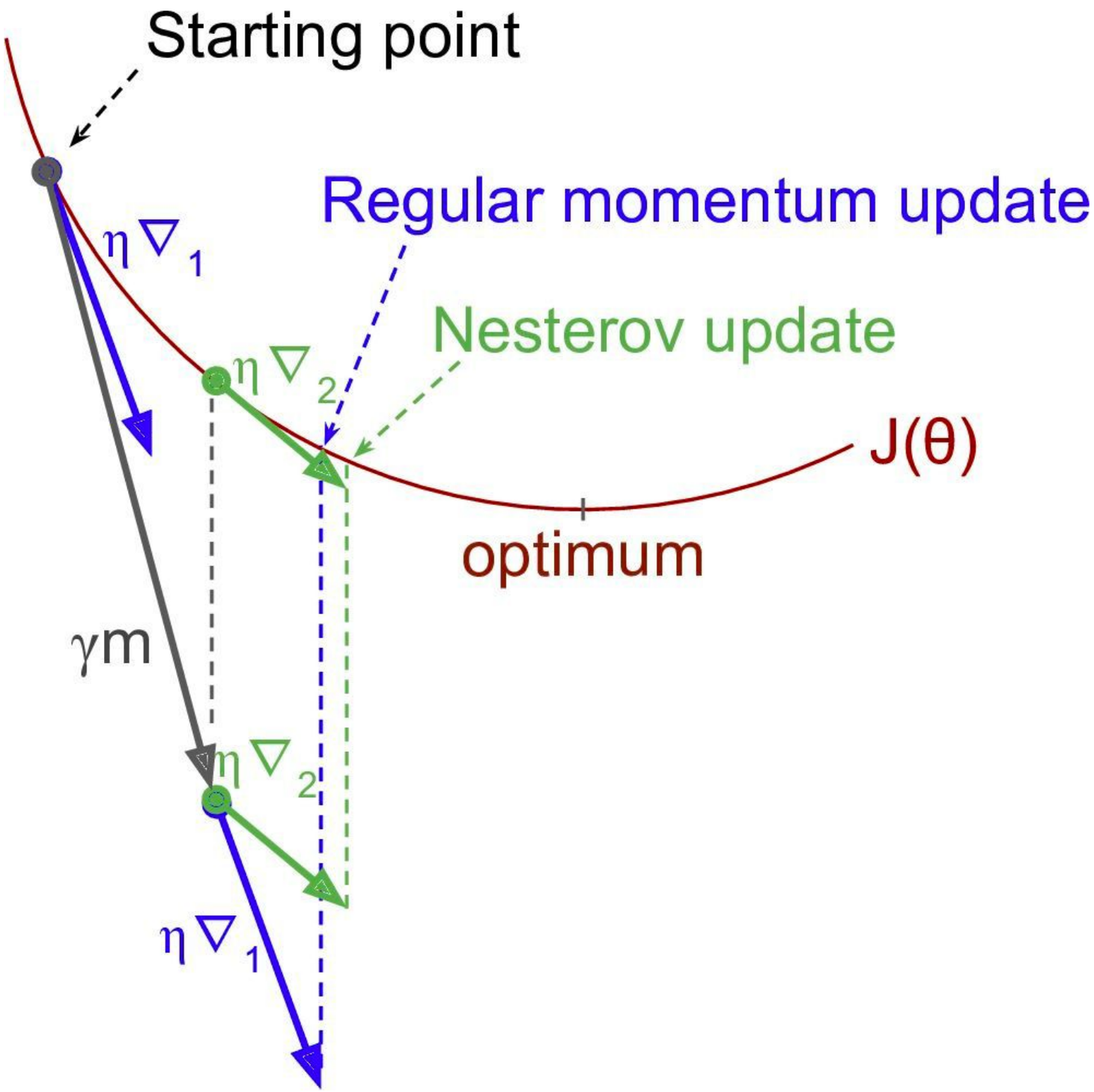
Faster Optimizers – Nesterov Accelerated Gradient



- The Nesterov update ends up slightly closer to the optimum.
- After a while, these small improvements add up and NAG ends up being significantly faster than regular Momentum optimization

CLOUD x LAB

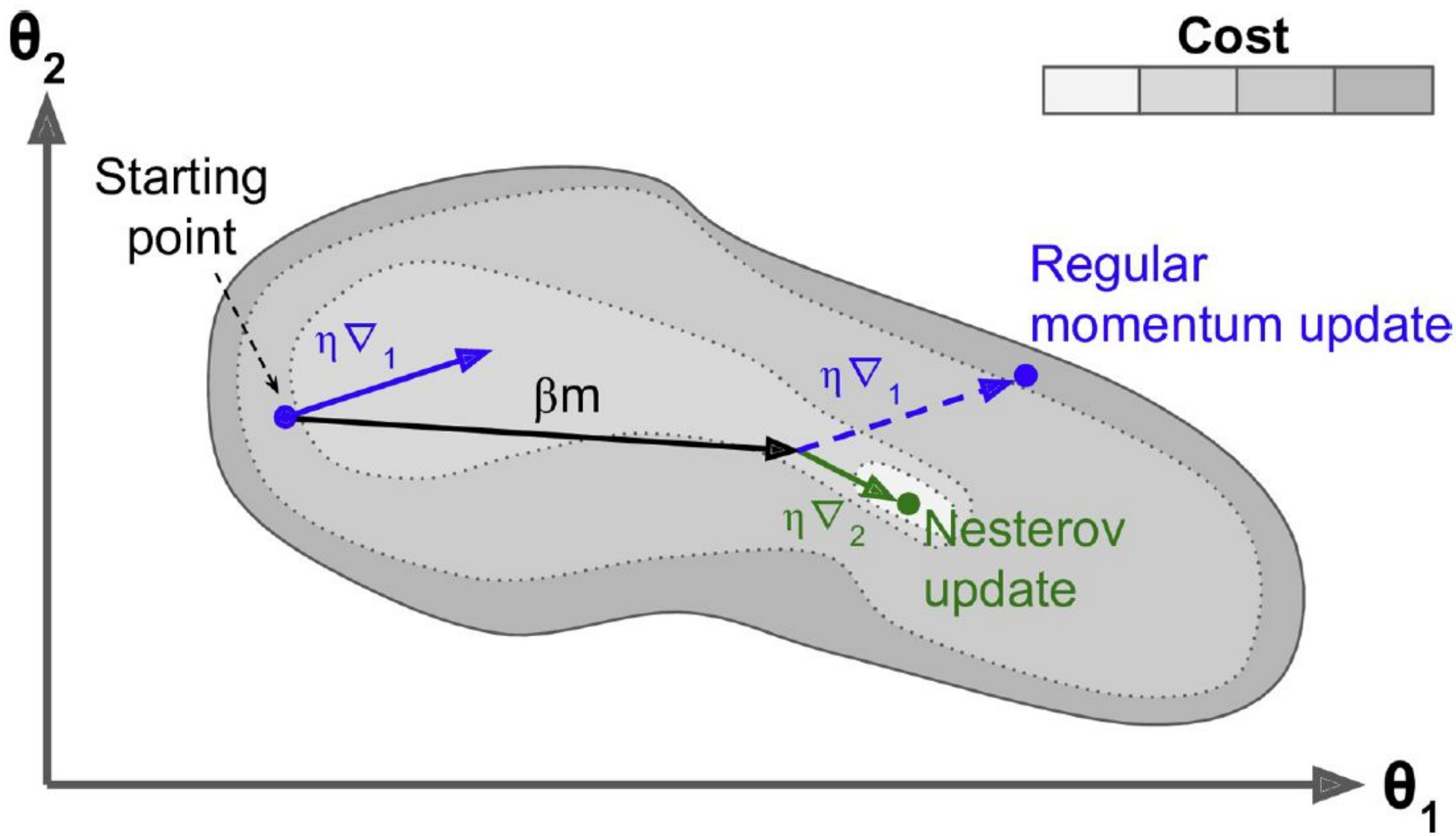
Faster Optimizers - Nesterov Accelerated Gradient



- Note that when the momentum pushes the weights across a valley, ∇1 continues to push further across the valley, while ∇2 pushes back toward the bottom of the Valley.
- This helps reduce oscillations and thus converges faster.

CLOUD x LAB

Faster Optimizers – Nesterov Accelerated Gradient



CLOUD x LAB

Faster Optimizers - Nesterov Accelerated Gradient

Implementing Nesterov Accelerated Gradient

NAG will almost always speed up training compared to regular Momentum optimization. To use it, simply set **use_nesterov=True** when creating the MomentumOptimizer:

```
>>> optimizer = keras.optimizers.SGD(lr=0.001,
momentum=0.9, nesterov=True)
```


CLOUD x LAB

Faster Optimizers – AdaGrad

- Gradient Descent starts by quickly going down the steepest slope, then slowly goes down the bottom of the valley
- It would be nice if the algorithm could detect this early on and correct its direction to point a bit more toward the global optimum
- The AdaGrad algorithm achieves this by scaling down the gradient vector along the steepest dimensions

Faster Optimizers – AdaGrad

How does AdaGrad work ?

- **The first step** accumulates the square of the gradients into the vector s
- The $\otimes$ symbol represents the element-wise multiplication
- This vectorized form is equivalent to computing $\mathbf{s}_i \leftarrow \mathbf{s}_i + (\partial / \partial\, \boldsymbol{\theta}_i\, J(\boldsymbol{\theta}))^2$ for each element s_i of the vector s

$$1. \qquad \boldsymbol{s} \leftarrow \boldsymbol{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$$

CLOUD x LAB

Faster Optimizers – AdaGrad

How does AdaGrad work ?

- In other words, each s_i accumulates the squares of the partial derivative of the cost function with regards to parameter θ_i
- If the cost function is steep along the ith dimension, then s_i will get larger and larger at each iteration.

$$1. \qquad \boldsymbol{s} \leftarrow \boldsymbol{s} + \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$$

Faster Optimizers – AdaGrad

How does AdaGrad work ?

- **The second step** is almost identical to Gradient Descent, but with one big difference:
  - The gradient vector is scaled down by a factor $\sqrt{\boldsymbol{s} + \epsilon}$
  - The $\oslash$ symbol represents the element-wise division, and ϵ is a smoothing term to avoid division by zero, typically set to 10^{-10}

$$1. \quad \boldsymbol{s} \leftarrow \boldsymbol{s} + \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$$

$$2. \quad \theta \leftarrow \theta - \eta \, \nabla_\theta J(\theta) \oslash \sqrt{\boldsymbol{s} + \epsilon}$$

Faster Optimizers – AdaGrad

How does AdaGrad work ?

- This vectorized form is equivalent to computing for all parameters θ_i

$$\theta_i \leftarrow \theta_i - \eta \, \partial / \partial \theta_i \, J(\theta) / \sqrt{s_i + \epsilon}$$

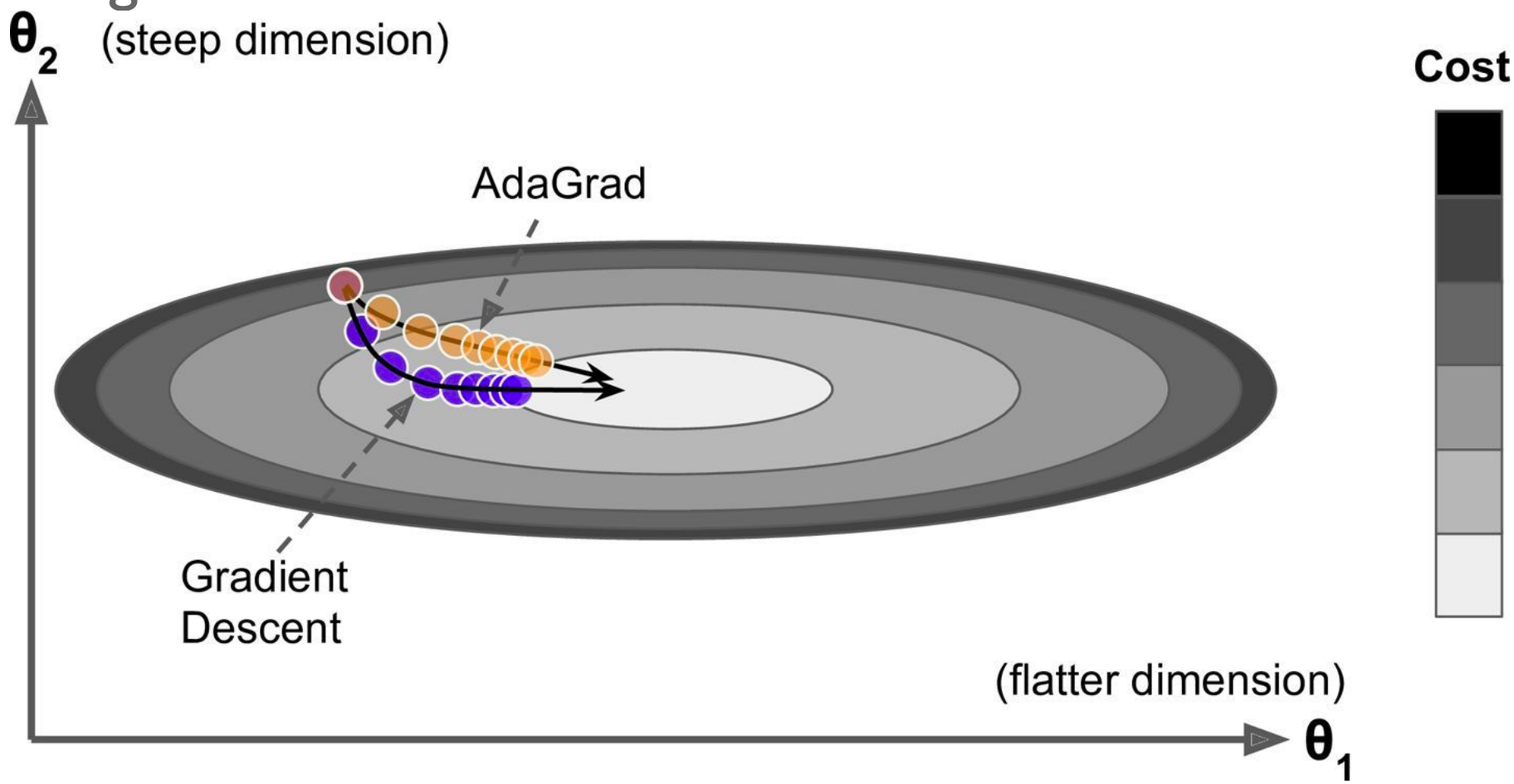
- This algorithm decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes.
- This is called an adaptive learning rate.

$$1. \quad \boldsymbol{s} \leftarrow \boldsymbol{s} + \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$$

$$2. \quad \theta \leftarrow \theta - \eta \, \nabla_\theta J(\theta) \oslash \sqrt{\boldsymbol{s} + \epsilon}$$

Faster Optimizers – AdaGrad

Advantages of AdaGrad



- It helps point the resulting updates more directly toward the global optimum. One additional benefit is that it requires much less tuning of the learning rate hyperparameter η

Faster Optimizers – AdaGrad

Disadvantages of AdaGrad

- AdaGrad often performs well for simple quadratic problems, but unfortunately it often stops too early when training neural networks
- The learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum.
- So even though Keras has an Adagrad optimizer, you **should not use** it to train deep neural networks
- It may be efficient for simpler tasks such as Linear Regression

Faster Optimizers – RMSProp

- AdaGrad slows down a bit too fast and ends up never converging to the global optimum
- The **RMSProp** algorithm fixes this by accumulating only the gradients from the most recent iterations, as opposed to all the gradients since the beginning of training
- It does so by using exponential decay in the first step

Faster Optimizers – RMSProp

- The decay rate **β** is typically set to 0.9
- It is once again a new hyperparameter, but this default value often works well, so you may not need to tune it at all

$$
\begin{aligned}
&1. \quad \boldsymbol{s} \leftarrow \beta \boldsymbol{s} + (1-\beta)\, \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta) \\
&2. \quad \theta \leftarrow \theta - \eta\, \nabla_\theta J(\theta) \oslash \sqrt{\boldsymbol{s} + \epsilon}
\end{aligned}
$$

Faster Optimizers – RMSProp

Implementing RMSProp

```
>>> optimizer = keras.optimizers.RMSprop(lr=0.001,
rho=0.9)
```

Faster Optimizers – RMSProp

Implementing RMSProp

- Except on very simple problems, this optimizer almost always performs much better than AdaGrad
- It also generally performs better than Momentum optimization and Nesterov Accelerated Gradients
- In fact, it was the preferred optimization algorithm of many researchers until Adam optimization came around

Faster Optimizers – Adam Optimization

- **Adam** which stands for adaptive moment estimation, combines the ideas of
  - Momentum optimization
  - And RMSProp
- Just like Momentum optimization it keeps track of an exponentially decaying average of past gradients
- And just like RMSProp it keeps track of an exponentially decaying average of past squared gradients

Faster Optimizers – Adam Optimization

1. $\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_\theta J(\theta)$

2. $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$

3. $\mathbf{m} \leftarrow \dfrac{\mathbf{m}}{1 - \beta_1}$

4. $\mathbf{s} \leftarrow \dfrac{\mathbf{s}}{1 - \beta_2}$

5. $\theta \leftarrow \theta - \eta \mathbf{m} \oslash \sqrt{\mathbf{s} + \epsilon}$

- If you just look at steps 1, 2, and 5, you will notice Adam's close similarity to both Momentum optimization and RMSProp.

Faster Optimizers – Adam Optimization

1. $$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_\theta J(\theta)$$

- The only difference is that step 1 computes an exponentially decaying average rather than an exponentially decaying sum
- But these are actually equivalent except for a constant factor, the decaying average is just 1 – β1 times the decaying sum

CLOUD x LAB

Faster Optimizers – Adam Optimization

3. $$\boldsymbol{m} \leftarrow \frac{\boldsymbol{m}}{1-\beta_1}$$

4. $$\boldsymbol{s} \leftarrow \frac{\boldsymbol{s}}{1-\beta_2}$$

- Steps 3 and 4 are somewhat of a technical detail
  - Since m and s are initialized at 0, they will be biased toward 0 at the beginning of training
- So these two steps will help boost m and s at the beginning of training.

Faster Optimizers – Adam Optimization

1. $\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1 - \beta_1) \nabla_\theta J(\theta)$
2. $\boldsymbol{s} \leftarrow \beta_2 \boldsymbol{s} + (1 - \beta_2) \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$
3. $\boldsymbol{m} \leftarrow \dfrac{\boldsymbol{m}}{1 - \beta_1}$
4. $\boldsymbol{s} \leftarrow \dfrac{\boldsymbol{s}}{1 - \beta_2}$

- The momentum decay hyperparameter **β1** is typically initialized to 0.9, while the scaling decay hyperparameter **β2** is often initialized to 0.999.
- As earlier, the smoothing term **ϵ** is usually initialized to a tiny number such as 10^{-8}

Faster Optimizers – Adam Optimization

- Since Adam is an adaptive learning rate algorithm, like AdaGrad and RMSProp, it requires less tuning of the learning rate hyperparameter **η**
- We can often use the default value η = 0.001, making Adam even easier to use than Gradient Descent

CLOUD x LAB

Faster Optimizers – Adam Optimization

Implementing Adam Optimization in Keras

```
>>> optimizer = keras.optimizers.Adam(lr=0.001,
beta_1=0.9, beta_2=0.999)
```

Faster Optimizers – AdaMax Optimization

1. $\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1 - \beta_1) \nabla_\theta J(\theta)$
2. $\boldsymbol{s} \leftarrow \beta_2 \boldsymbol{s} + (1 - \beta_2) \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta)$

$$\boxed{\mathbf{s} \leftarrow \max\left(\beta_2 \mathbf{s}, \nabla_\theta J(\theta)\right)}$$

Adam plus this term

3. $\boldsymbol{m} \leftarrow \dfrac{\boldsymbol{m}}{1 - \beta_1}$
4. $\boldsymbol{s} \leftarrow \dfrac{\boldsymbol{s}}{1 - \beta_2}$

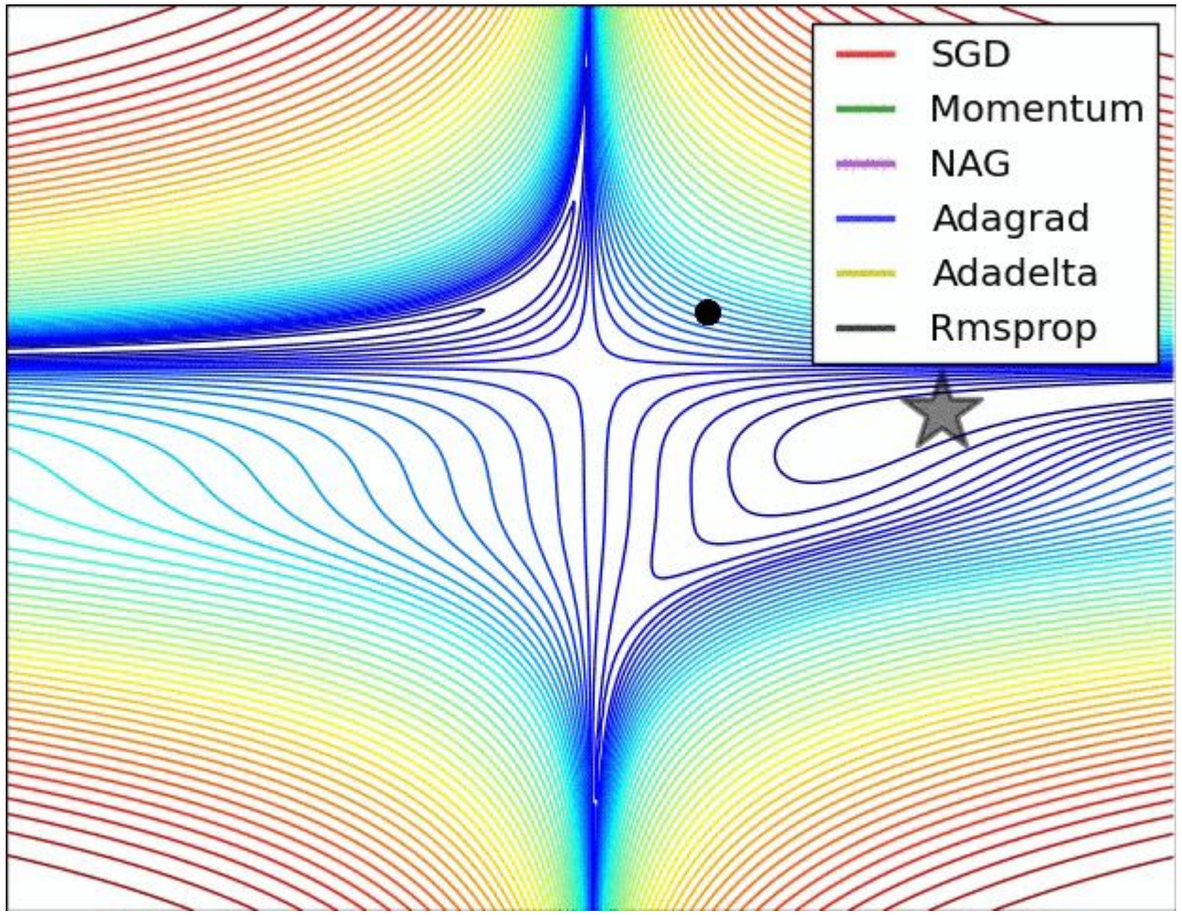
Faster Optimizers – Nadam Optimization

- Adam optimization plus the Nesterov trick
- Converge slightly faster than Adam
- Generally outperforms Adam
- Sometimes outperformed by RMSProp
  - RMSProp > Nadam > Adam

CLOUD x LAB

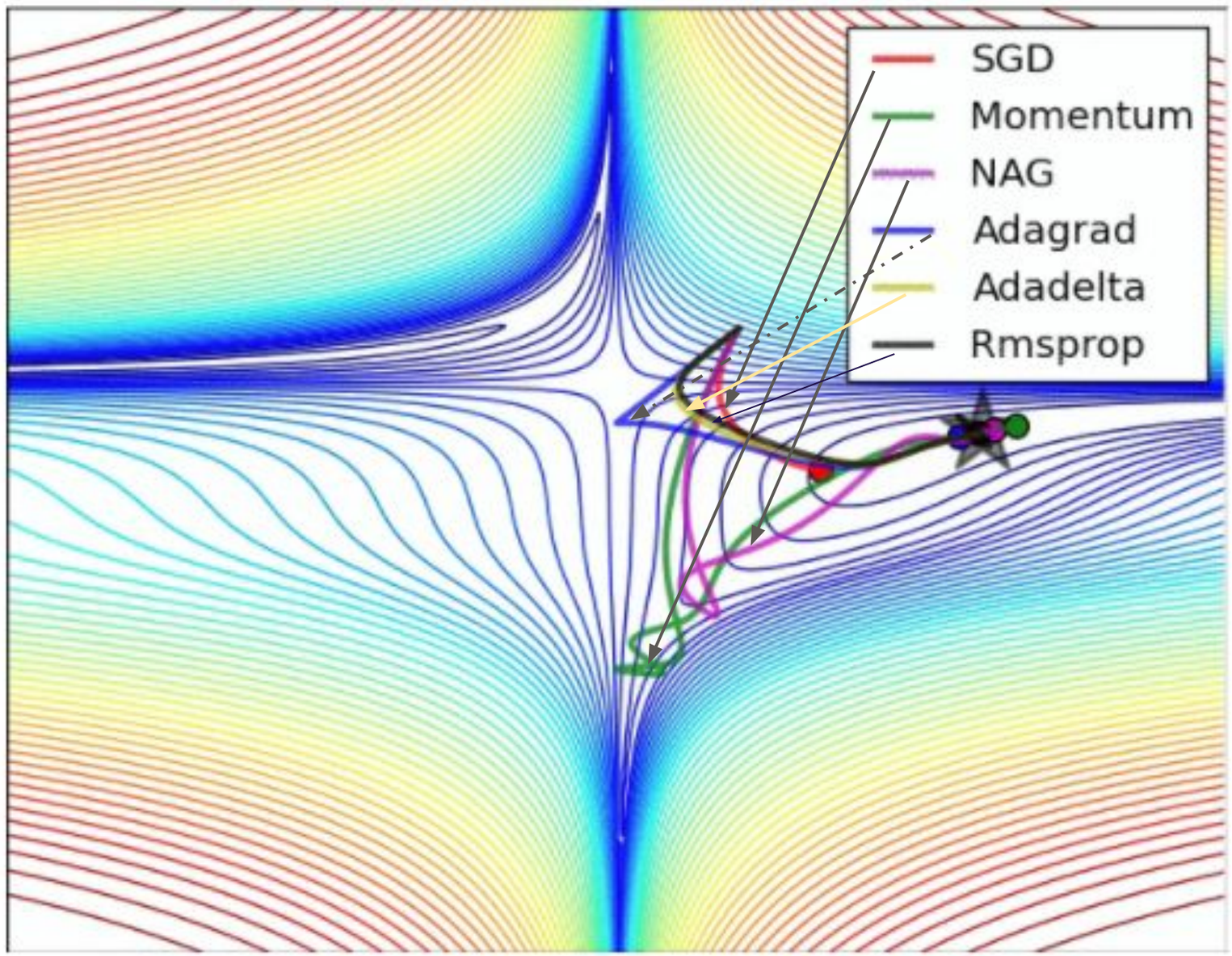
Faster Optimizers – Compare

See http://ruder.io/optimizing-gradient-descent/



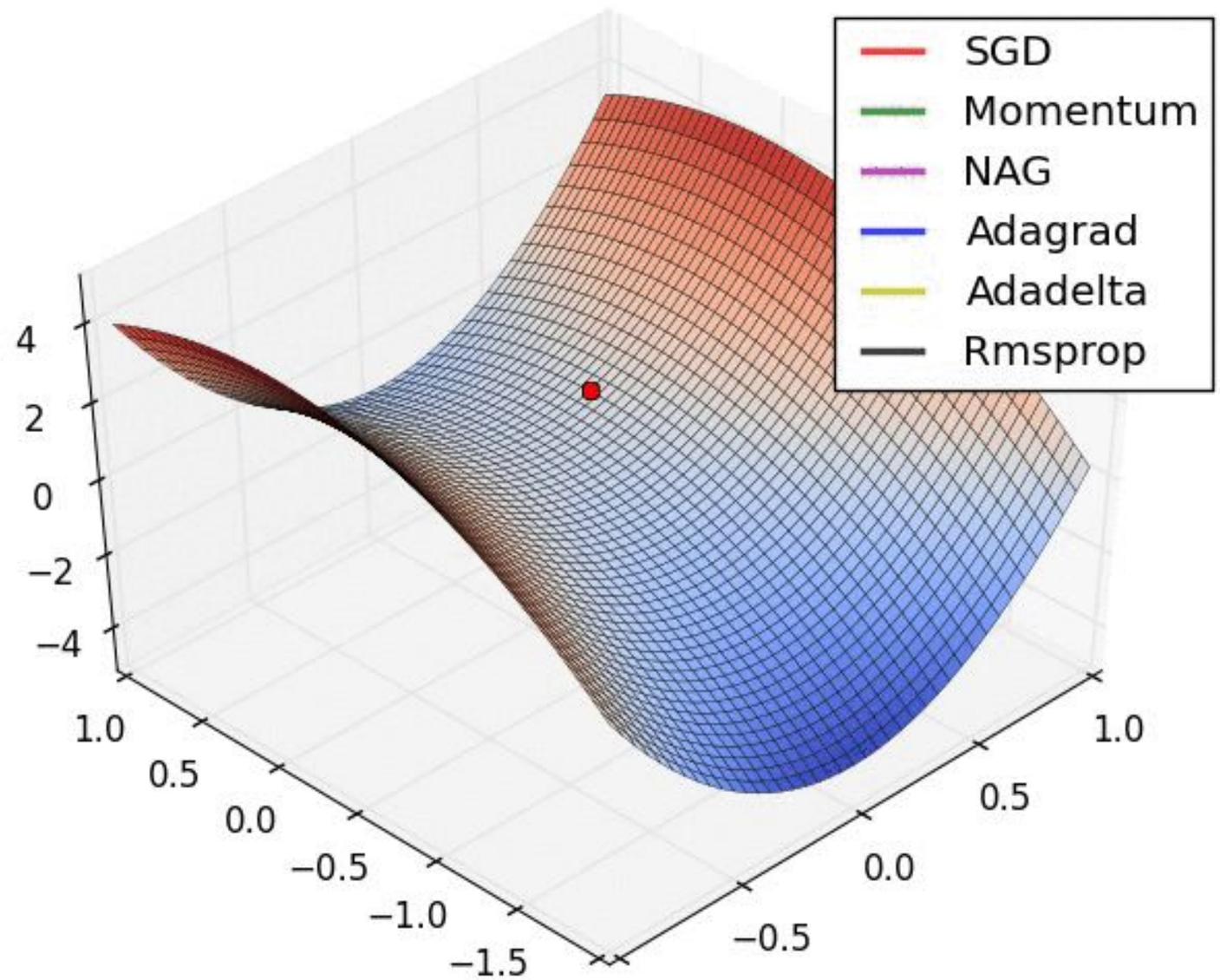
Faster Optimizers – Compare

See http://ruder.io/optimizing-gradient-descent/

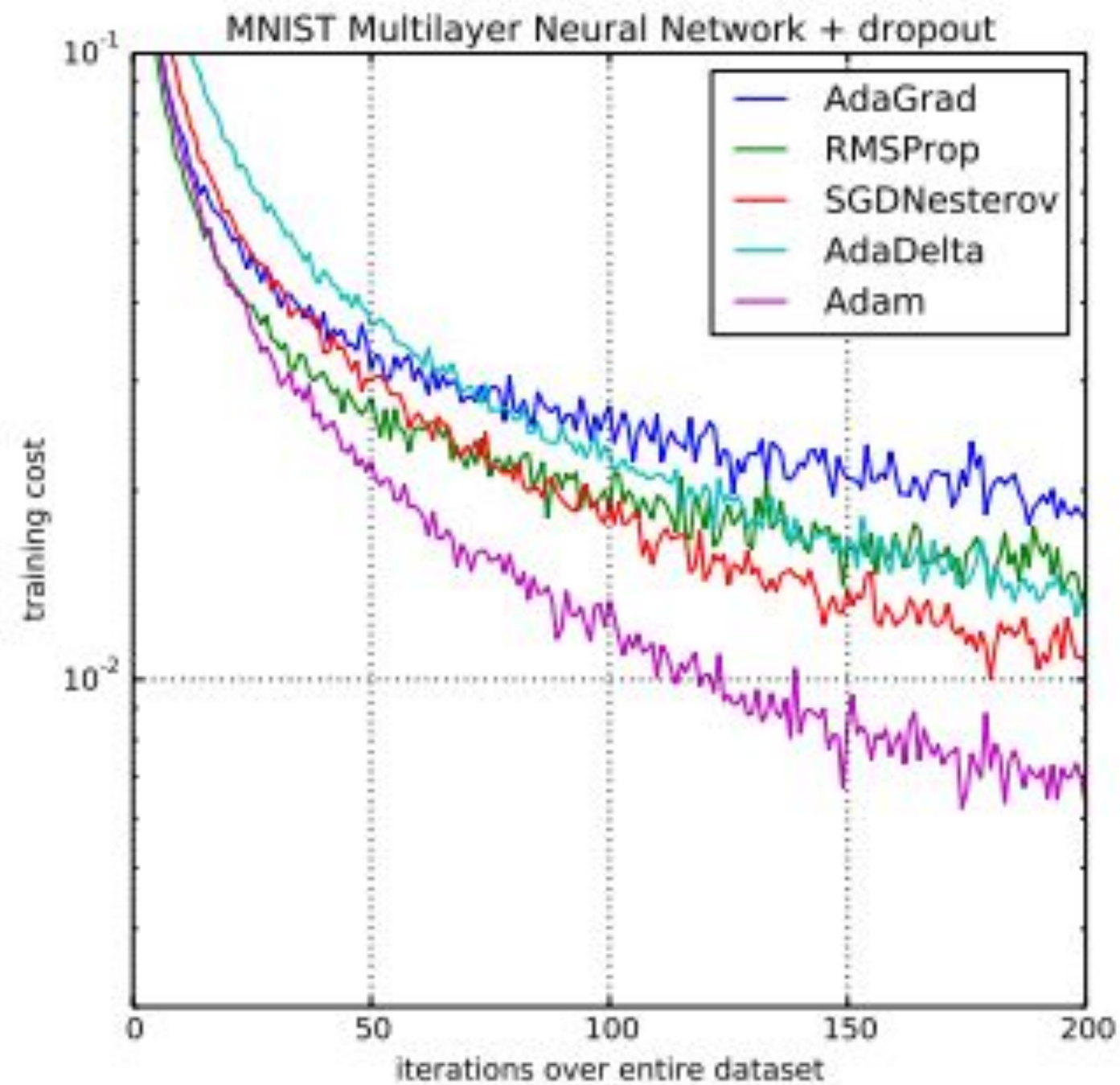


Faster Optimizers – Compare

See http://ruder.io/optimizing-gradient-descent/



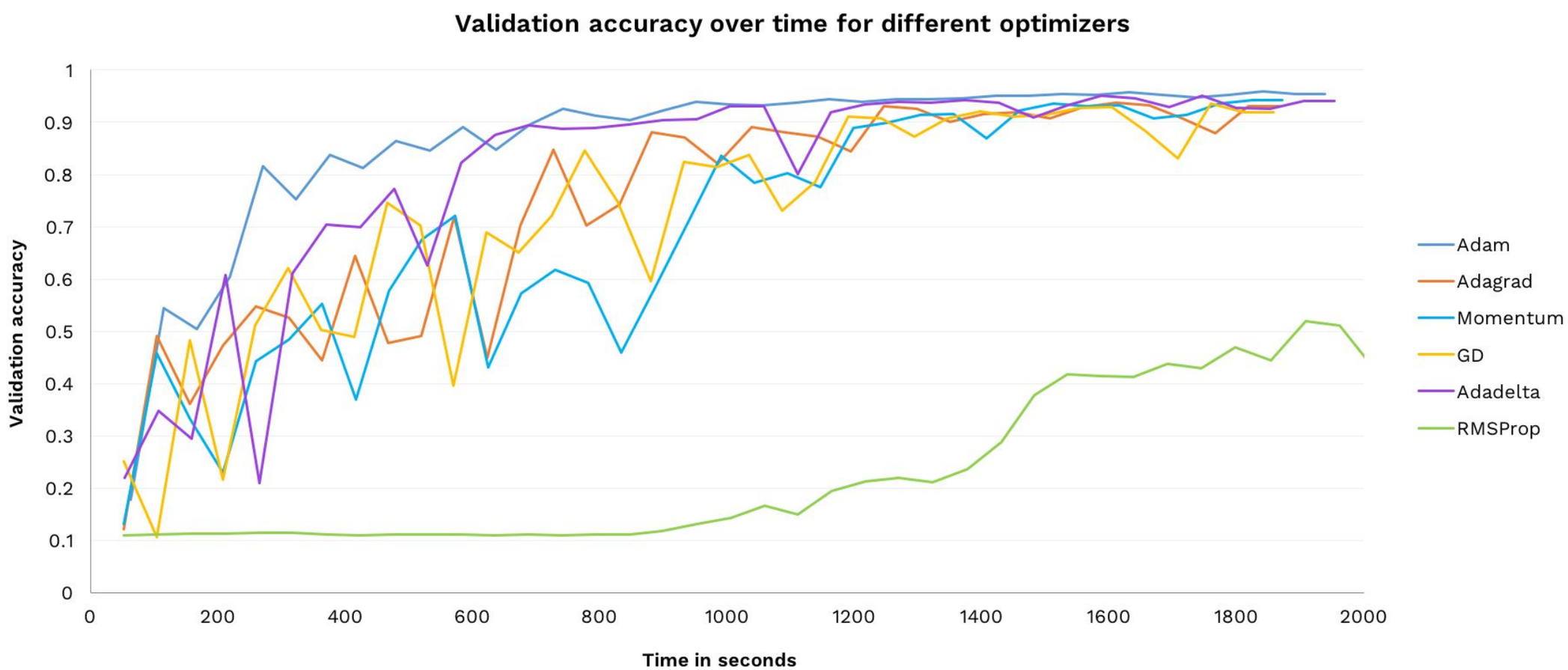
Faster Optimizers – Compare



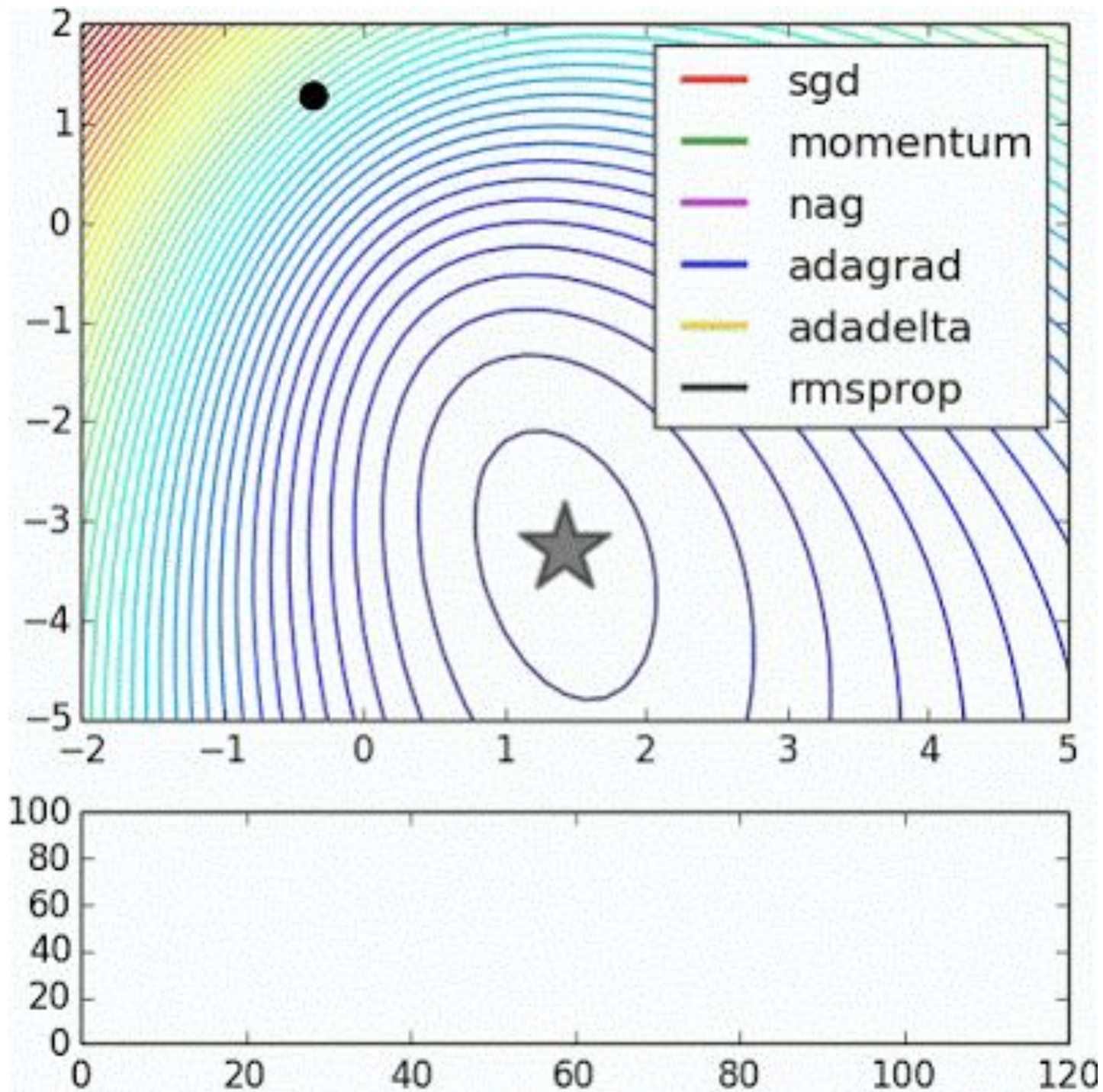
Faster Optimizers - Compare

See

https://medium.com/octavian-ai/which-optimizer-and-learning-rate-should-i-use-for-deep-learning-5acb418f9b2



Faster Optimizers - Comparisions



TRAINING SPARSE MODELS

1. Most of the models discussed produce dense models
   a. (mostly non-zero weights)
2. For faster runtime models or low memory models (edge computing)
   a. You may want the sparse models
3. Apply L1 regularization during training that will reduce some weights to zero
4. Other techniques are **Quantization and Binarization** and **Network Pruning**. Ref https://arxiv.org/pdf/1710.09282.pdf
5. Checkout https://www.tensorflow.org/model_optimization/

Faster Optimizers – Learning Rate Scheduling

How do we find a good learning rate ??

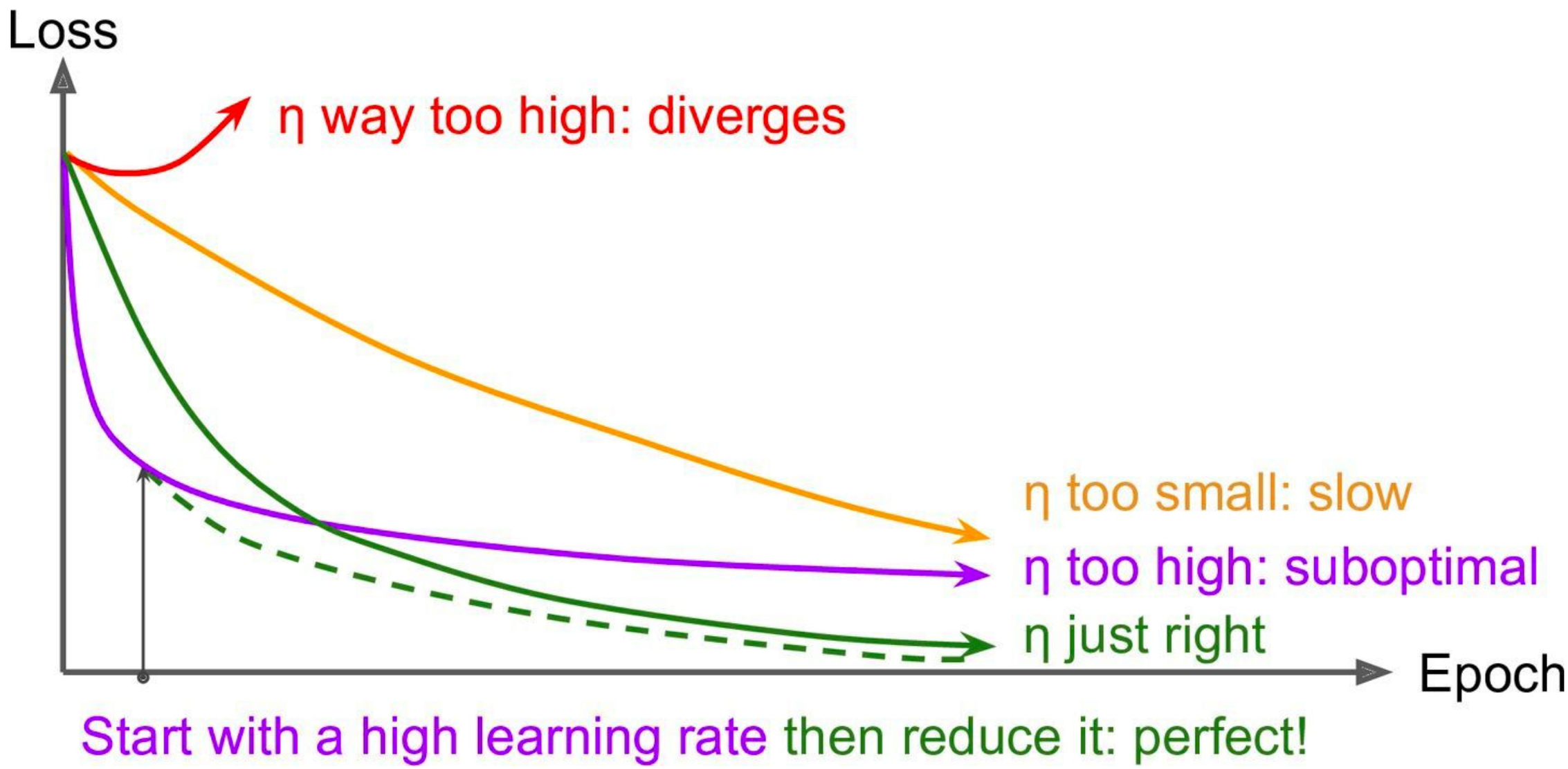
Faster Optimizers – Learning Rate Scheduling

- Finding a good learning rate can be tricky.
- If we set it way **too high**,
  - Training may actually diverge
- If you set it **too low**,
  - Training will eventually converge to the optimum, but it will take a very long time.

Faster Optimizers – Learning Rate Scheduling

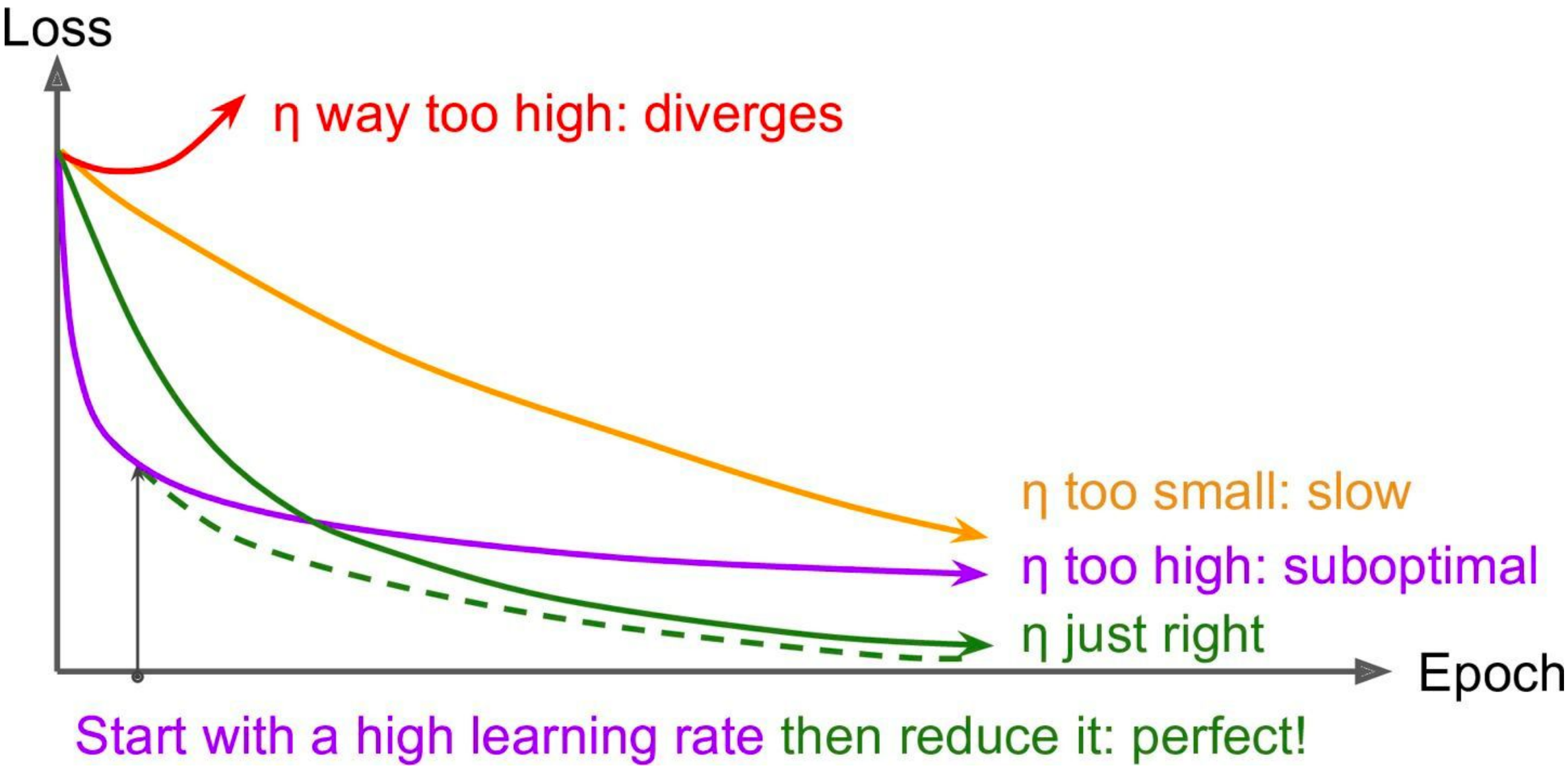
- If you set it **slightly too high**,
  - It will make progress very quickly at first,
  - But it will end up dancing around the optimum, never settling down
- We have to use an adaptive learning rate optimization algorithm such as AdaGrad, RMSProp, or Adam,
  - But even then it may take time to settle
- If you have a limited computing budget, you may have to interrupt training before it has converged properly, yielding a suboptimal solution

Faster Optimizers – Learning Rate Scheduling



- We may be able to find a fairly good learning rate by training your network several times during just a few epochs using various learning rates and comparing the learning curves

Faster Optimizers – Learning Rate Scheduling



The ideal learning rate will learn quickly and converge to good solution

Faster Optimizers – Learning Rate Scheduling

- We can do better than a constant learning rate:
- If we start with a high learning rate and then reduce it once it stops making fast progress
- We can reach a good solution faster than with the optimal constant learning rate.

Faster Optimizers – Learning Rate Scheduling

- There are many different strategies to reduce the learning rate during training.
- These strategies are called **learning schedules,** the most common ones are now discussed

Faster Optimizers – Learning Rate Scheduling

Predetermined piecewise constant learning rate

- For example, set the learning rate to $\eta_0 = 0.1$ at first, then to $\eta_1 = 0.001$ after 50 epochs.
- Although this solution can work very well, it often requires fiddling around to figure out the right learning rates and when to use them.

Faster Optimizers – Learning Rate Scheduling

Performance scheduling

- Measure the validation error every N steps, just like for early stopping and reduce the learning rate by a factor of λ when the error stops dropping.

Exponential scheduling

- Set the learning rate to a function of the iteration number t: $\eta(t) = \eta_0 10^{-t/r}$. This works great, but it requires tuning η_0 and r. The learning rate will drop by a factor of 10 every r steps.

Faster Optimizers – Learning Rate Scheduling

Power scheduling

- Set the learning rate to $\eta(t) = \eta_0 (1 + t/r)^{-c}$.
- The hyperparameter c is typically set to 1.
- This is similar to exponential scheduling, but the learning rate drops much more slowly.
- Inverse Time Decay in Keras

```
>>> tf.keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate, decay_steps, decay_rate,
staircase=False, name=None
```


CLOUD x LAB

Faster Optimizers – Learning Rate Scheduling

Implementing a learning schedule with Keras

```
>>> optimizer = keras.optimizers.SGD(lr=0.01,
decay=1e-4)
```

Run it on Notebook

CLOUD x LAB

Faster Optimizers – Learning Rate Scheduling

Implementing a learning schedule with Keras

```
>>> model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="selu",
kernel_initializer="lecun_normal"),
    keras.layers.Dense(100, activation="selu",
kernel_initializer="lecun_normal"),
    keras.layers.Dense(10, activation="softmax")
])
```

Run it on Notebook

Faster Optimizers – Learning Rate Scheduling

Implementing a learning schedule with Keras

```
>>> model.compile
(loss="sparse_categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])
```

Run it on Notebook

Faster Optimizers – Learning Rate Scheduling

Implementing a learning schedule with Keras

```
optimizer = keras.optimizers.SGD(lr=0.01, decay=1e-4)
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="selu",
kernel_initializer="lecun_normal"),
    keras.layers.Dense(100, activation="selu",
kernel_initializer="lecun_normal"),
    keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])
```

Run it on Notebook

Faster Optimizers – Learning Rate Scheduling

Since AdaGrad, RMSProp, and Adam optimization automatically reduce the learning rate during training, it is ***not necessary to add an extra learning schedule****.*

For other optimization algorithms, using exponential decay or performance scheduling can considerably speed up convergence.

Faster Optimizers

- The conclusion is that we should always use Adam optimization
  - We really do not have to know about internals
  - Simply replace GradientDescentOptimizer with AdamOptimizer
  - With this small change training will be several times faster

Avoid Overfitting Through Regularization

Overfitting

"With four parameters I can fit an elephant and with five I can make him wiggle his trunk. "

-- John von Neumann, cited by Enrico Fermi in Nature 427

Avoid Overfitting Through Regularization

- Deep neural networks may have **millions of parameters**
- With so many parameters network
  - has a **huge amount of freedom**
  - And it can fit variety of **complex datasets**
  - Also it becomes **prone to overfitting**

Avoid Overfitting Through Regularization

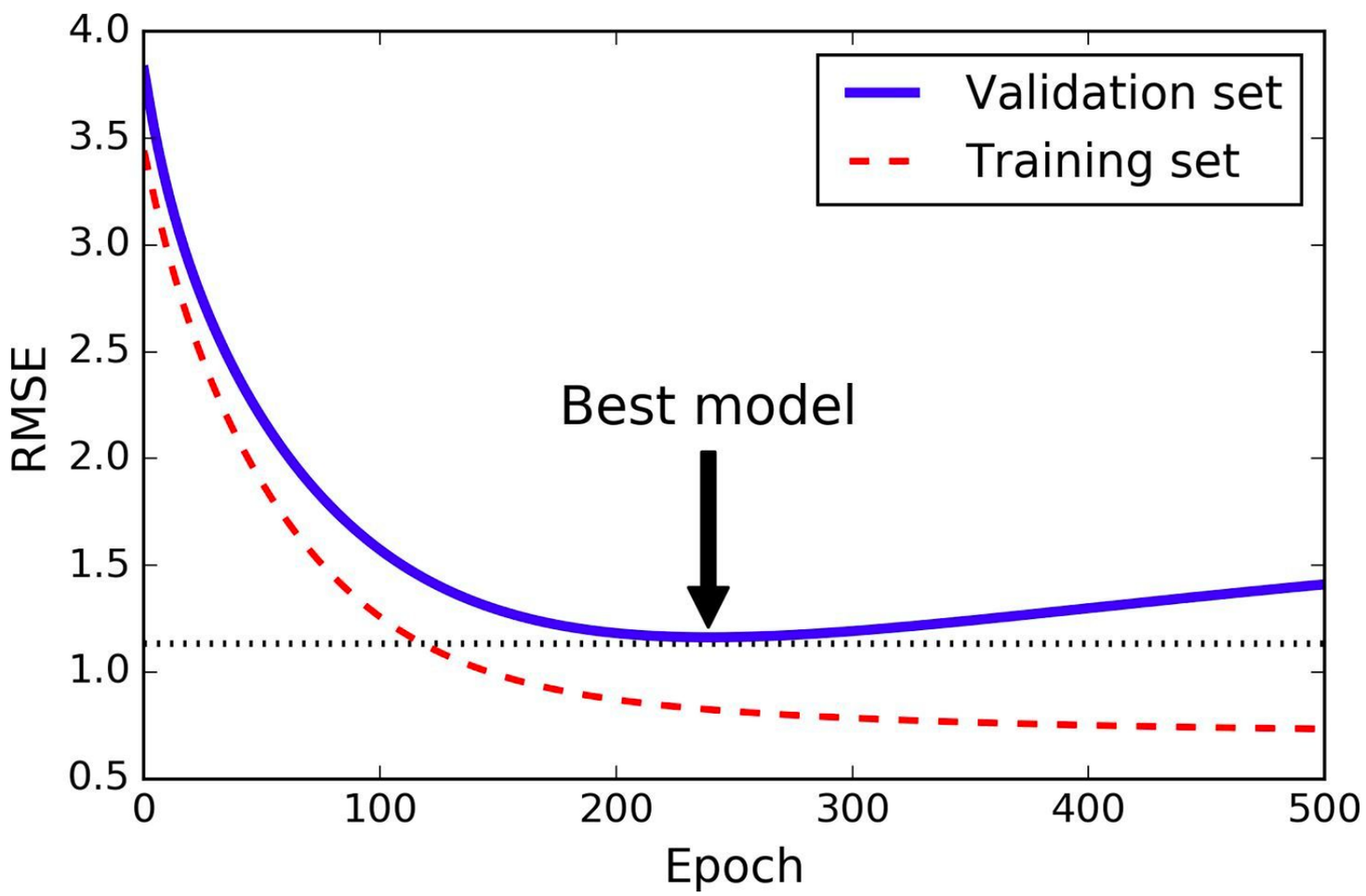
- In this section, we will go through
  - Some of the most popular regularization techniques
  - For neural network and how to implement them with Keras
    - Early stopping
    - $\ell 1$ and $\ell 2$ regularization
    - Dropout
    - Max-Norm Regularization and
    - Data augmentation

Avoid Overfitting Through Regularization

Early Stopping

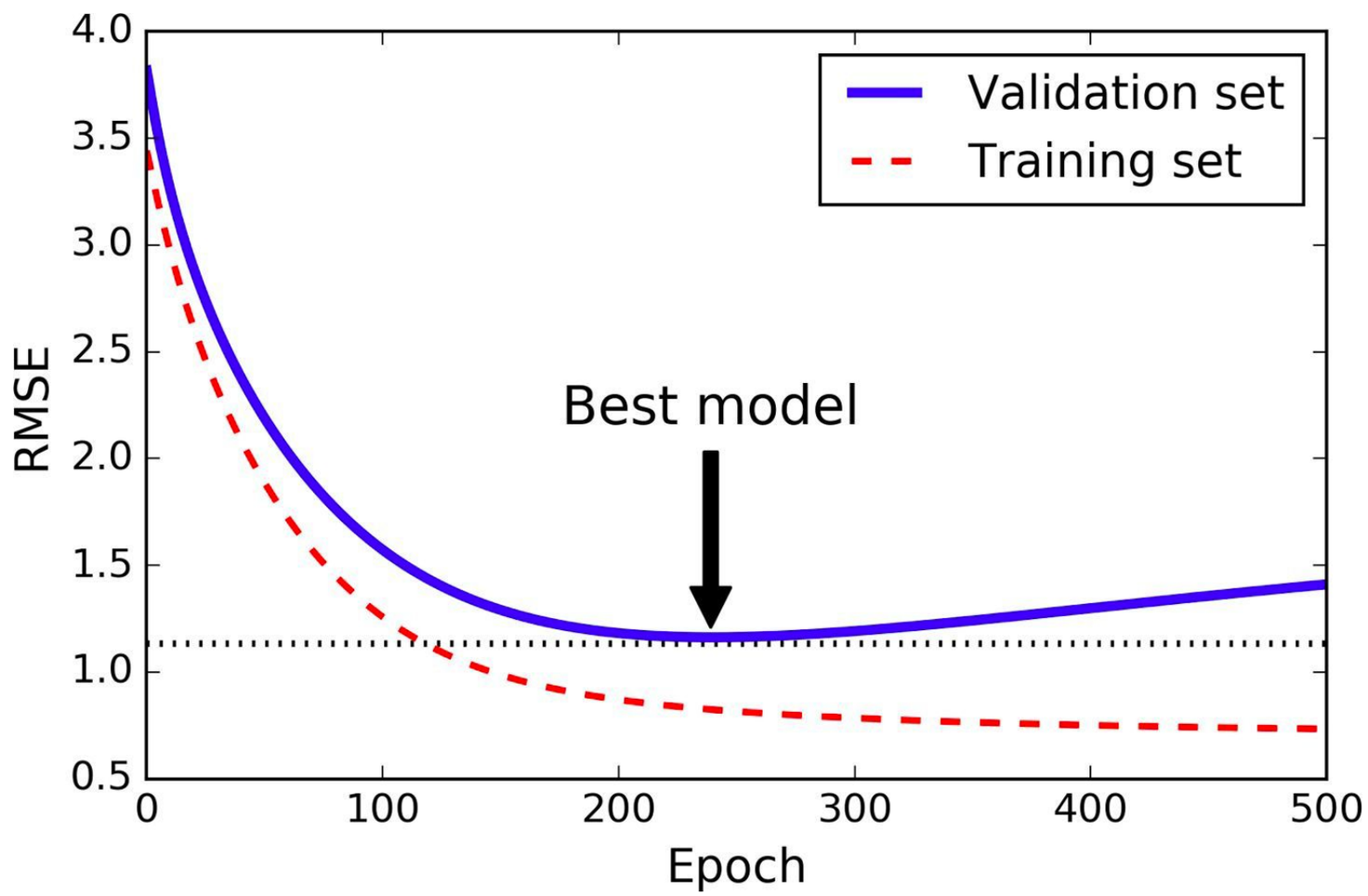
Early Stopping

- As discussed in Machine Learning course
  - To avoid overfitting the training set
  - A great solution is **early stopping**



Early Stopping

- Stop training as soon as the validation error reaches a minimum
- This is called **early stopping**



Avoid Overfitting Through Regularization

$\ell 1$ and $\ell 2$ Regularization

ℓ1 and ℓ2 Regularization

- Just like we apply ℓ1 and ℓ2 regularization for simple linear models
  - We can apply the same regularization to constrain
  - Neural network's connection weights (not biases)

ℓ1 and ℓ2 Regularization

- For example, here is how to apply ℓ2 regularization to a Keras layer's connection weights, using a regularization factor of 0.01:

```
>>> layer = keras.layers.Dense(100, activation="elu",
kernel_initializer="he_normal",
kernel_regularizer=keras.regularizers.l2(0.01))
```


CLOUD x LAB

ℓ1 and ℓ2 Regularization

Follow the code in the notebook to implement ℓ2 regularization

Avoid Overfitting Through Regularization

Dropout

Dropout

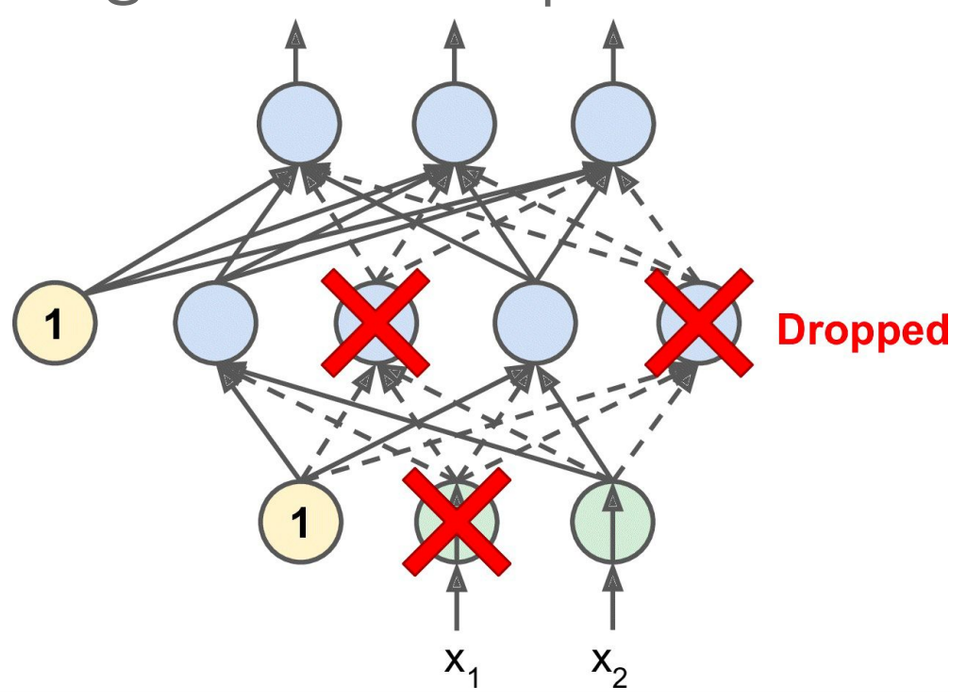
- Dropout is the most popular
  - Regularization technique for deep neural networks
- It was proposed by G. E. Hinton in 2012
- Even the state-of-the-art neural networks
  - Got a 1–2% accuracy boost
  - Simply by adding dropout

Dropout

- 1-2% accuracy boost may not sound like a lot
  - But when a model has 95% accuracy
  - Then 2% accuracy boost means dropping the error rate by 40%
  - (Going from 5% error to roughly 3%)

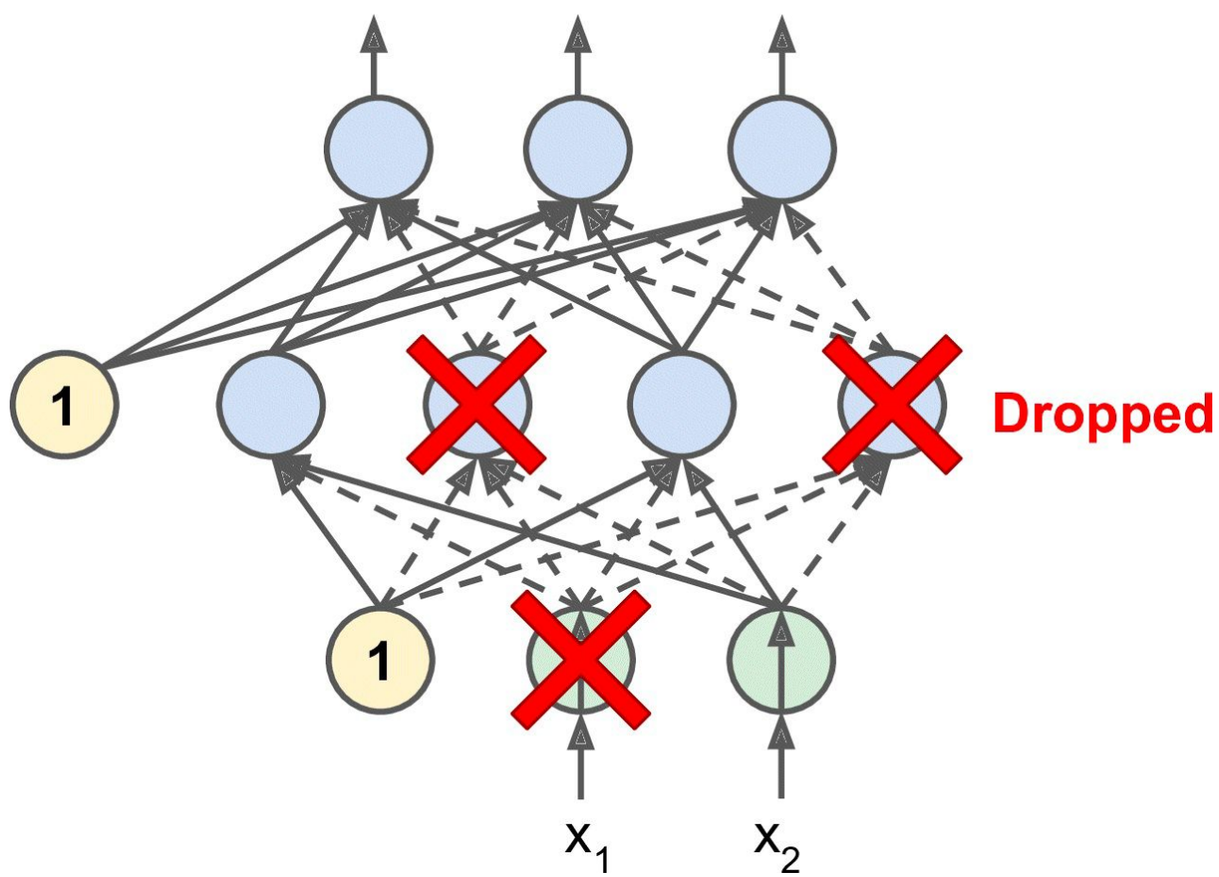
Dropout

- It is a fairly simple algorithm
- At every training step, every neuron
  - Including the input neurons but excluding the output neurons
  - Has a probability **p** of being temporarily **"dropped out"**
  - Meaning it will be entirely ignored during this training step
  - But it may be active during the next step



Dropout

- The hyperparameter **p** is called the **dropout rate**
  - And it is typically set to 50%
- After training, neurons don't get dropped anymore
- Let's understand this technique with an example



CLOUD x LAB

Dropout

Question

Would a company perform better if its employees were told to toss a coin every morning to decide whether or not to go to work?

Dropout

Answer

Perhaps it would. Who knows :)

Dropout

- In that case company would be forced to adapt its organization
  - No single person will be responsible for filling the coffee machine
  - Or cleaning the office
  - Or performing any other critical tasks
- So these expertise would have to be spread across many people
- Employees would have to learn to
  - Cooperate with many of their coworkers

Dropout

Question

What will be the advantages of such a system?

Dropout

- The company would become much more resilient
- If one person quits, it would not make much difference
- Not sure if this idea will work for companies
  - But it definitely works for neural networks

Dropout

- Neurons trained with dropout
  - Can not co-adapt with their neighbouring neurons
  - They have to be as useful as possible on their own
  - They also can not rely excessively on just a few input neurons
  - They also must pay attention to each of their input neurons
  - As a result of this
    - They end up being less sensitive to slight changes in the inputs
- In the end we get a more robust network that generalizes better

Dropout

- To implement dropout using Keras you can use the keras.layers.Dropout layer.
- During training, it randomly drops some inputs (setting them to 0) and divides the remaining inputs by the keep probability
- After training, it does nothing at all; it just passes the inputs to the next layer.

Dropout

Follow the code in the notebook to apply dropout regularization using Keras

Dropout

- If you observe model is overfitting
  - Then increase the dropout rate
- Else if model is underfitting
  - Then decrease the dropout rate
- It can also help to
  - Increase the dropout rate for large layers, and
  - Reduce it for small ones

Dropout

- Please note that dropout does
  - Tend to slow down convergence
  - But it results in a much better model when tuned properly
  - It is worth the extra time

Avoid Overfitting Through Regularization

Data Augmentation

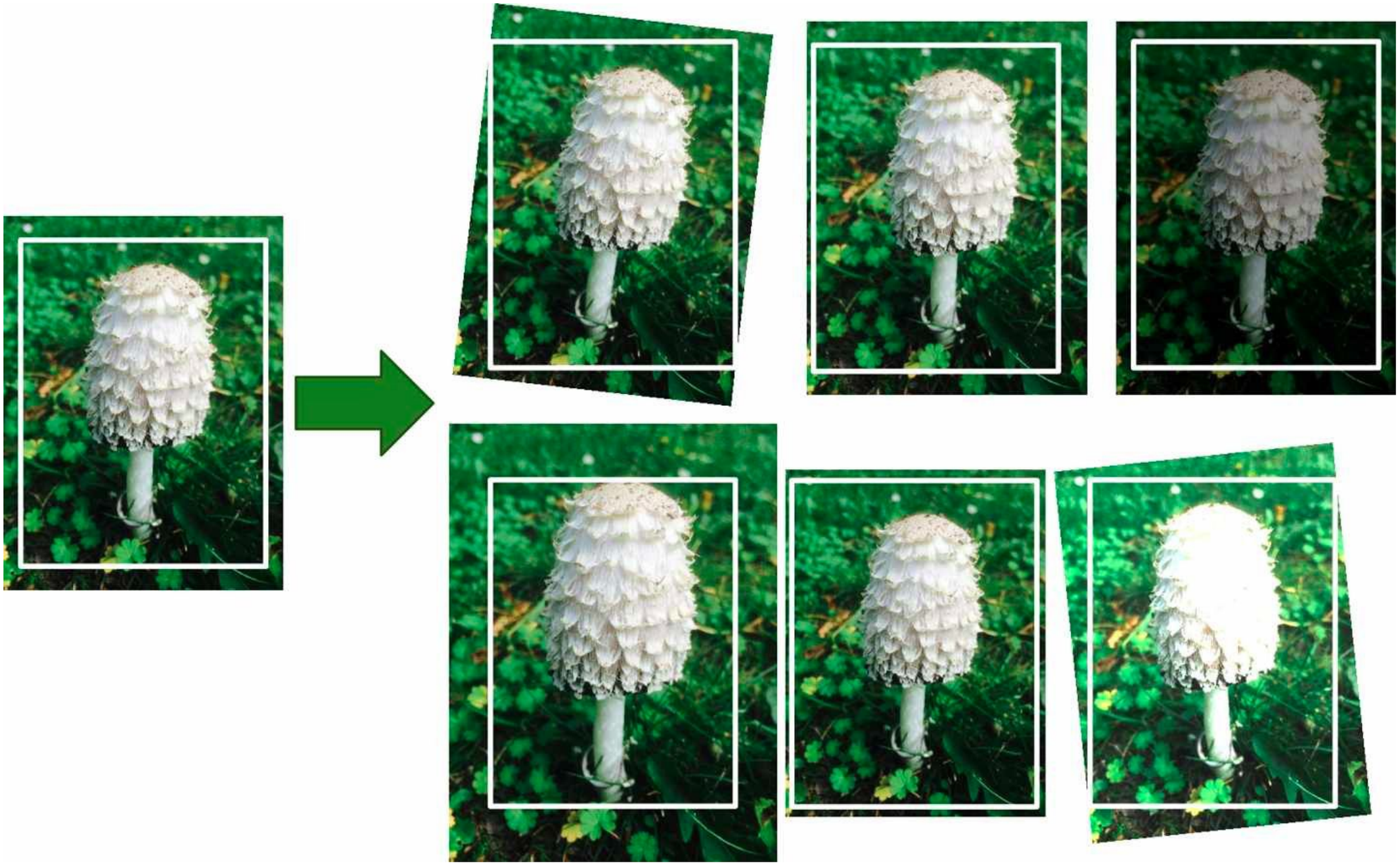
Data Augmentation

- Data augmentation consists of
  - Generating new training instances from existing ones
  - Thereby increasing the size of the training set
- Let's understand this with an example

Data Augmentation

- Let's say we have to train a model to classify pictures of mushrooms
- Then we can slightly shift, rotate and resize
  - Every picture in the training set and
  - Add the resulting pictures to the training set
  - Thereby increasing the size of the training set

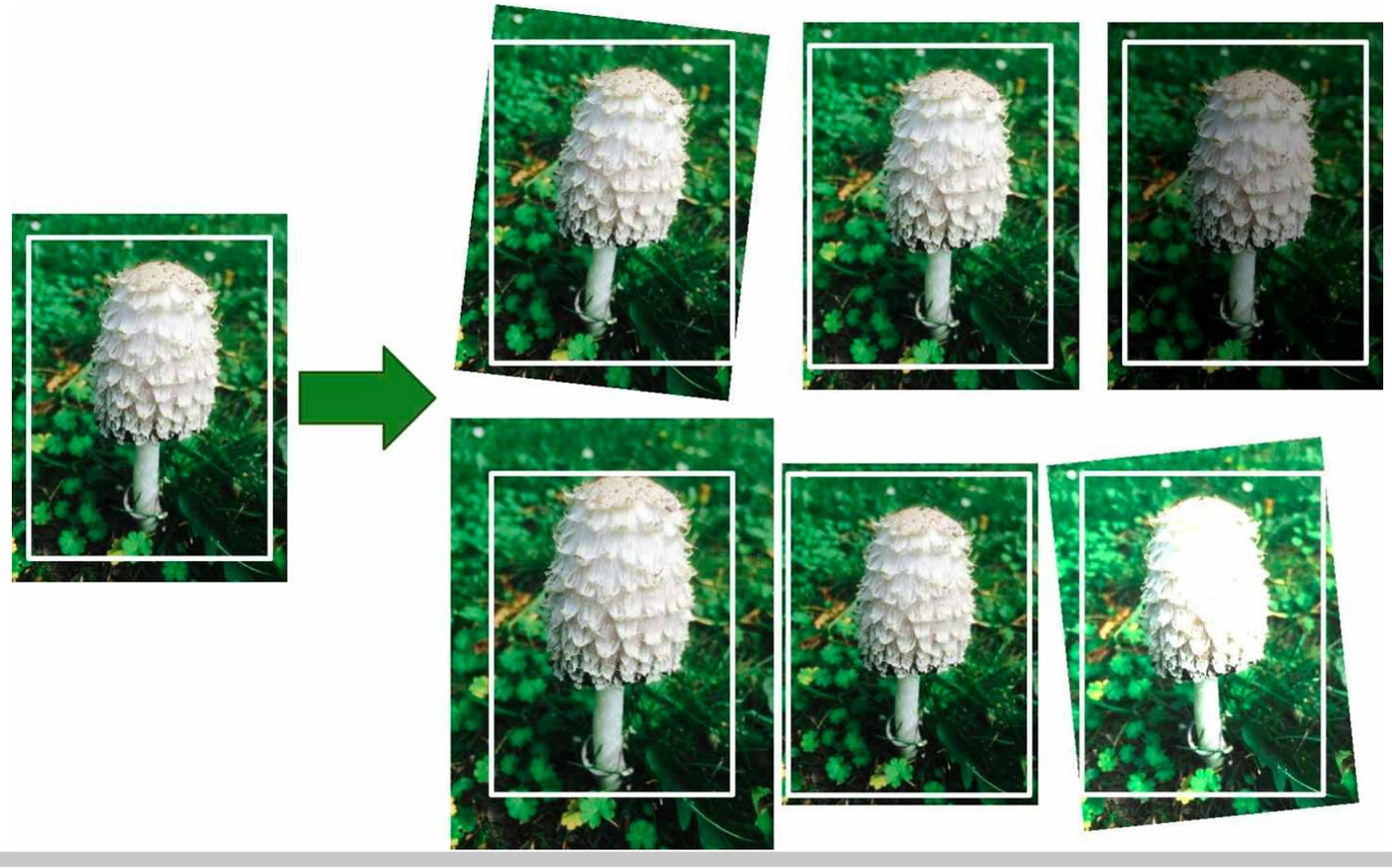
Data Augmentation



Generating new training instances of mushrooms from existing ones

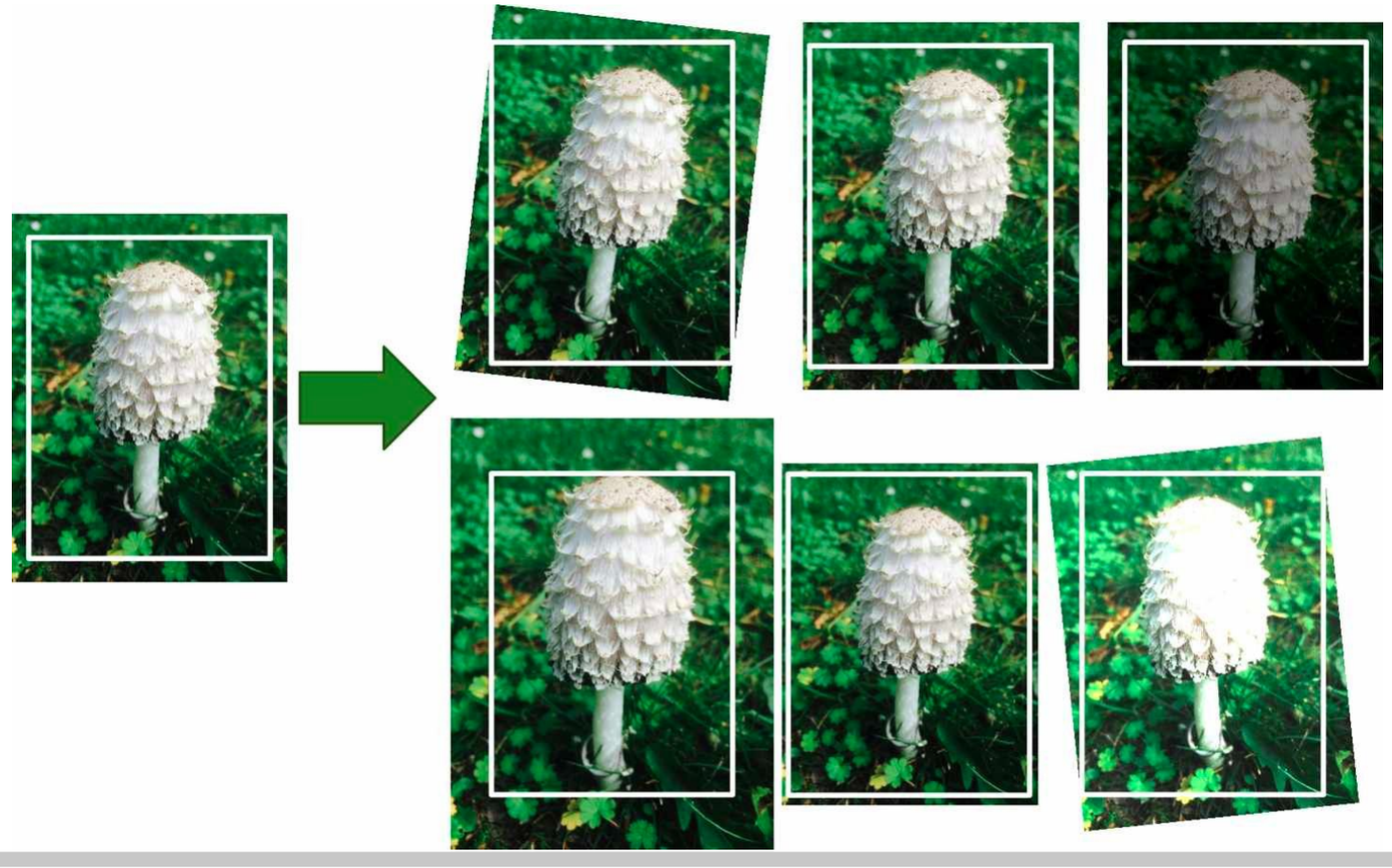
Data Augmentation

- The trick is to generate realistic training instances
- A human should not be able to tell
  - Which instances were generated and which ones were not
- Moreover the modifications we apply should be learnable



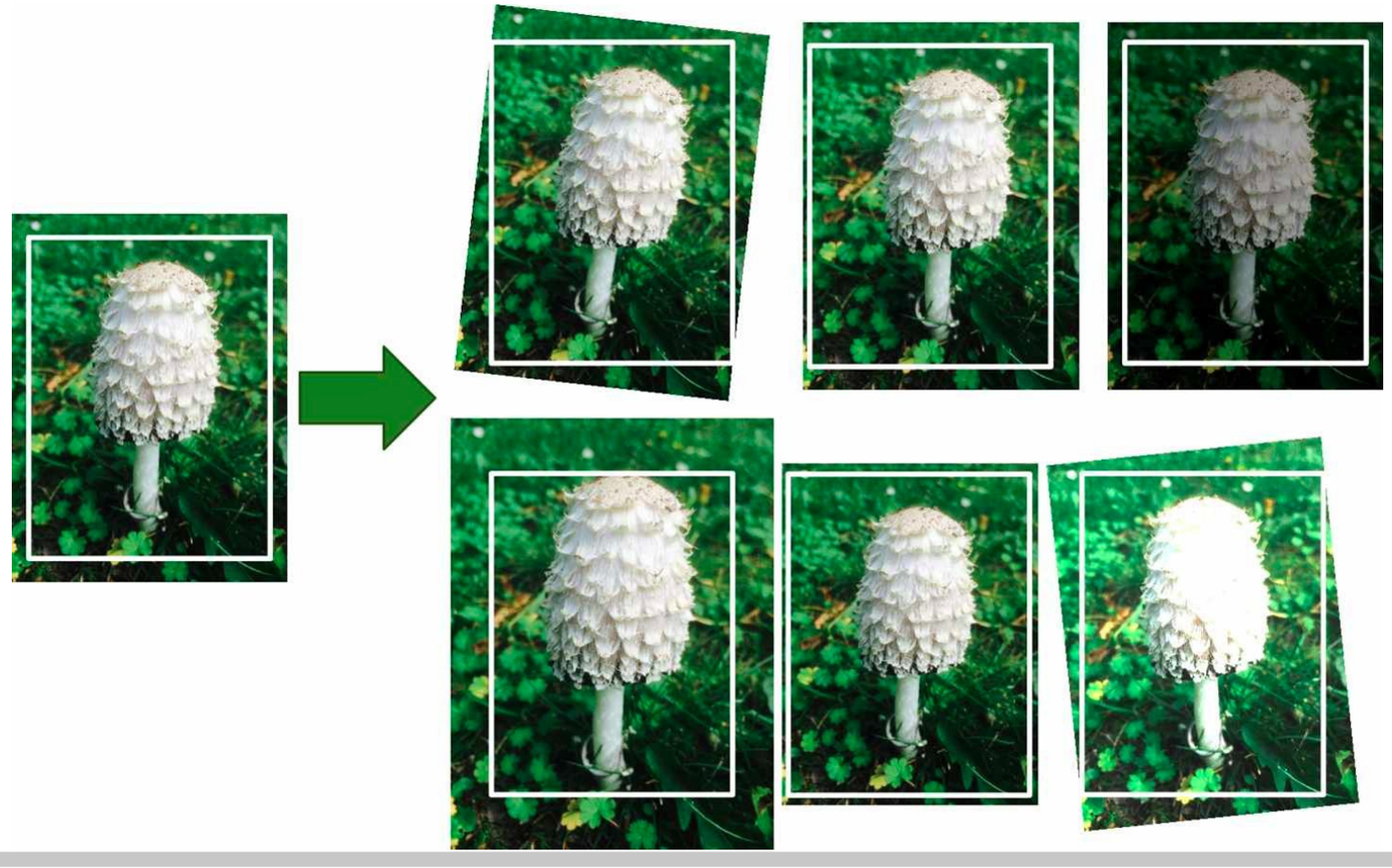
Data Augmentation

- These newly added pictures
  - Forces the model to be more tolerant to the
    - Position,
    - Orientation, and
    - Size of the mushrooms in the picture



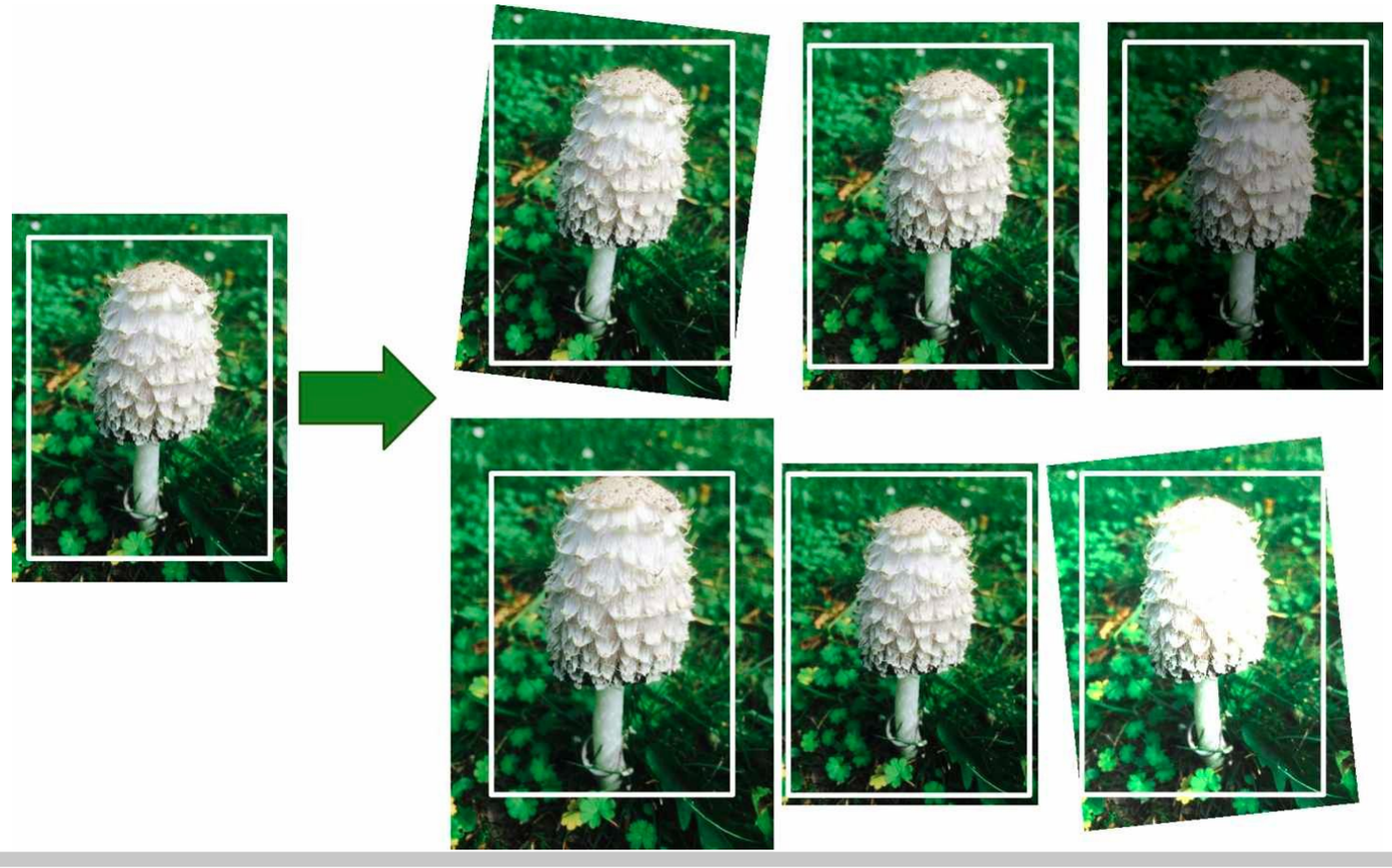
Data Augmentation

- If we want model to be more tolerant to the lightning conditions
  - We can also generate images with various contrasts and
  - Add them to the training set



Data Augmentation

- It is preferable to generate new images on the fly during training
  - Rather than wasting
    - Storage space and
    - Network bandwidth



Data Augmentation

- Keras offers several image manipulation operations such as
  - Transposing(shifting)
  - Rotating
  - Resizing
  - Flipping
  - Cropping
  - Adjusting the brightness
  - Contrast
  - Saturation and
  - Hue

Data Augmentation

- These operations makes it easy to implement data augmentation for image datasets

Practical Guidelines

Practical Guidelines

- In this topic we have covered wide range of techniques
- And common question comes on which one to use
- The configuration shown below works fine in most of the cases

Hyperparameter	Default value
Kernel initializer	He initialization
Activation function	ELU
Normalization	None if shallow; Batch Norm if deep
Regularization	Early stopping (+ℓ_2 reg. if needed)
Optimizer	Momentum optimization (or RMSProp or Nadam)
Learning rate schedule	1cycle

Default DNN Configuration

Practical Guidelines

- Also we should always look for the pretrained neural network solving the similar problem
- The default configuration which we have shown in the last slide may be tweaked as per the problem statement
    - If training set is too small then implement data augmentation

Practical Guidelines

- The default configuration which we have shown in the last slide may be tweaked as per the problem statement
  - If we can't find a good learning rate then trying adding
    - Learning schedule such as exponential decay
  - If we need a lightning fast model at run time
    - Then drop batch normalization and
    - Replace ELU with leaky ReLU

Practical Guidelines

- If we need a sparse model
  - Add some $\ell 1$ regularization
- With these guidelines
  - We can train deep neural networks
  - But if we use a single machine then
  - It make take days or months for training to complete
  - So be patient :)
  - Else train the model across many servers and GPUs

Practical Guidelines – Difference between l1, l2 (Annex)

- If we have two weights (2,1) to reduce loss, say we need to reduce weights by a total of 1.
- Should we reduce one to zero or should be reduce both by 0.5.
- (2, 1) ---> (2, 0) or (1.5, 0.5) ????

Practical Guidelines – Difference between l1, l2 (Annex)

- If we have two weights (2,1) to reduce loss, say we need to reduce weights by a total of 1.
- Should we reduce one to zero or should be reduce both by 0.5.
- (2, 1) ---> (2, 0) or (1.5, 0.5) ????

	Weights		
Regularization Loss	**2, 1**	**2, 0**	**1.5, 0.5**
L1 = |w1| + |w2|	*3*		
L2 = $w1^2 + w2^2$	*5*		

Practical Guidelines – Difference between l1, l2 (Annex)

- If we have two weights (2,1) to reduce loss, say we need to reduce weights by a total of 1.
- Should we reduce one to zero or should be reduce both by 0.5.
- (2, 1) ---> (2, 0) or (1.5, 0.5) ????

	Weights		
Regularization Loss	**2, 1**	**2, 0**	**1.5, 0.5**
L1 = |w1| + |w2|	*3*	*2*	*2*
L2 = $w1^2 + w2^2$	*5*	*4*	*2.5*

Practical Guidelines – Difference between l1, l2 (Annex)

- If we have two weights (2,1) to reduce loss, say we need to reduce weights by a total of 1.
- Should we reduce one to zero or should be reduce both by 0.5.
- (2, 1) ---> (2, 0) or (1.5, 0.5) ????

	Weights		
Regularization Loss	**2, 1**	**2, 0**	**1.5, 0.5**
L1 = |w1| + |w2|	*3*	*2*	*3*
L2 = $w1^2 + w2^2$	*5*	*4*	*2.5*

So, L1 favours some weights to disappear.

Questions?

https://discuss.cloudxlab.com

reachus@cloudxlab.com